

CM3070 Final Year Project

Deep Learning Breast Cancer Detection - Full Pipeline

Merlin Melissa Jeyabal | University of London

This project investigates whether deep learning can support breast cancer screening using mammography images from the **CBIS-DDSM mass subset**.

The task is binary classification:

- **Benign**
- **Malignant**

The experimental pipeline progresses from a baseline convolutional neural network through controlled network-scaling, regularisation and resolution experiments, before moving to transfer learning, preprocessing enhancement, model selection, held-out test evaluation, robustness testing and interpretability analysis.

The main stages are:

1. Dataset preparation and patient-level train/validation/test splitting
2. Baseline CNN at 128×128 resolution
3. Network-scaling experiment using a deeper and wider custom CNN
4. Regularisation ablation using dropout-only, augmentation-only, and combined dropout + augmentation configurations
5. Regularised/scaled CNN comparison at 128×128 and 224×224 resolution
6. Transfer learning using VGG16 and ResNet50
7. CLAHE preprocessing applied to ResNet50
8. Principal model comparison using the validation set
9. Final model selection and freezing of the validation-derived decision threshold
10. Formal evaluation of the selected model on the held-out test set
11. Multi-seed robustness analysis using seeds 42, 7 and 123
12. McNemar significance testing between ResNet50 and ResNet50 + CLAHE
13. Grad-CAM qualitative visualisation and quantitative lesion-mask localisation analysis
14. Final results and discussion

The same patient-level train/validation/test split is maintained throughout the experiments. Model development, comparison and decision-threshold selection are performed using the validation set. Following model selection, the selected model is evaluated on the held-out test set using its frozen validation-derived threshold, keeping the held-out test set separate from the primary model-selection process.

Deterministic preprocessing is kept consistent where appropriate, while ImageNet-specific preprocessing is applied to pretrained networks. Data augmentation is applied only to the

relevant regularisation experiments, and CLAHE is used only in the dedicated enhanced ResNet50 experiment.

The default random seed for the main experiments is **42**, with additional seeds **7** and **123** used for the multi-seed robustness analysis.

Section 1: Setup

This section imports the libraries required for data processing, visualisation, model development and evaluation. Random seeds are also initialised to improve reproducibility across the main experiments.

```
import os, random, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.applications import VGG16, ResNet50
from tensorflow.keras.applications.vgg16 import preprocess_input as
vgg16_preprocess
from tensorflow.keras.applications.resnet50 import (
    preprocess_input as resnet50_preprocess,
)
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    classification_report,
    ConfusionMatrixDisplay,
)
from IPython.display import display
import warnings

warnings.filterwarnings("ignore")

from google.colab import drive

drive.mount("/content/drive")

# Constants
SEED = 42
IMG_SIZE_128 = 128
IMG_SIZE_224 = 224
BATCH_SIZE = 32
EPOCHS = 30
AUTOTUNE = tf.data.AUTOTUNE
BASE = "/content/drive/MyDrive/cbis_ddsm"
```

```

JPEG = os.path.join(BASE, "jpeg")
CSV_PATH = os.path.join(BASE, "csv")
MODELS_DIR = os.path.join(BASE, "models")
os.makedirs(MODELS_DIR, exist_ok=True)

# Reproducibility
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TF :", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
print("Seed:", SEED)

Mounted at /content/drive
TF : 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]
Seed: 42

MODELS_DIR = f"{BASE}/models_patientsplit_dedup"

os.makedirs(MODELS_DIR, exist_ok=True)

print(f"Saving corrected models to: {MODELS_DIR}")

Saving corrected models to:
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup

```

Section 2: Load Dataset

The CBIS-DDSM mass-case training and test metadata are loaded and prepared for the classification pipeline. Pathology labels are converted into binary targets, where **1 = malignant** and **0 = benign**.

```

train_csv = pd.read_csv(os.path.join(CSV_PATH,
"mass_case_description_train_set.csv"))
test_csv = pd.read_csv(os.path.join(CSV_PATH,
"mass_case_description_test_set.csv"))

label_map = {"MALIGNANT": 1, "BENIGN": 0, "BENIGN_WITHOUT_CALLBACK":
0}
train_csv["label"] = train_csv["pathology"].map(label_map)
test_csv["label"] = test_csv["pathology"].map(label_map)

train_csv.dropna(subset=["label"], inplace=True)
test_csv.dropna(subset=["label"], inplace=True)

print(

```

```

    f"Train: {len(train_csv)} | Malignant:
{int(train_csv['label'].sum())} | Benign:
{int((train_csv['label']==0).sum())}"
)
print(
    f"Test : {len(test_csv)} | Malignant:
{int(test_csv['label'].sum())} | Benign:
{int((test_csv['label']==0).sum())}"
)

```

```

Train: 1318 | Malignant: 637 | Benign: 681
Test : 378 | Malignant: 147 | Benign: 231

```

Section 3: Map Image Paths

Image paths are mapped from the CBIS-DDSM metadata to the corresponding mammogram files using the available identifiers. This creates the file references required for subsequent preprocessing and dataset construction.

```

# Map mammogram paths and remove duplicates

jpeg_lookup = {
    folder: os.path.join(JPEG, folder, files[0])
    for folder in os.listdir(JPEG)
    if os.path.isdir(os.path.join(JPEG, folder))
    and (
        files := [
            f for f in os.listdir(os.path.join(JPEG, folder)) if
f.endswith(".jpg")
        ]
    )
}

print(f"Total JPEG files indexed: {len(jpeg_lookup)}")

def get_image_path(csv_path):
    return next(
        (jpeg_lookup[p] for p in str(csv_path).split("/") if p in
jpeg_lookup), None
    )

for df in [train_csv, test_csv]:
    df["full_path"] = df["image file path"].apply(get_image_path)
    df.dropna(subset=["full_path"], inplace=True)

# Preserve lesion-level metadata for later lesion-mask analysis
train_lesion_csv = train_csv.copy()

```

```

test_lesion_csv = test_csv.copy()

print("\nBEFORE DEDUPLICATION")
for name, df in [("Train", train_csv), ("Test", test_csv)]:
    conflicts = df.groupby("full_path")["label"].nunique()
    print(f"{name} rows: {len(df)} | Unique mammograms: {df['full_path'].nunique()}")
    print(
        f"{name} mammograms with conflicting lesion labels: "
        f"{int((conflicts > 1).sum())}"
    )

# Each physical mammogram must belong to one patient
for df in [train_csv, test_csv]:
    assert df.groupby("full_path")["patient_id"].nunique().max() == 1

def collapse_to_mammograms(df):
    labels = df.groupby("full_path")["label"].max()
    out = df.sort_values("label",
        ascending=False).drop_duplicates("full_path").copy()
    out["label"] = out["full_path"].map(labels).astype(int)
    return out.reset_index(drop=True)

train_csv = collapse_to_mammograms(train_csv)
test_csv = collapse_to_mammograms(test_csv)

print("\nAFTER DEDUPLICATION")
for name, df in [("Train", train_csv), ("Test", test_csv)]:
    print(f"{name} rows: {len(df)} | Unique mammograms: {df['full_path'].nunique()}")
    print(
        f"{name} malignant: {int(df['label'].sum())} | "
        f"Benign: {int((df['label'] == 0).sum())}"
    )
    assert len(df) == df["full_path"].nunique()

print("\n✓ Duplicate mammogram rows removed")
print("✓ Conflicting lesion labels collapsed to one image-level label")
print("✓ Malignant takes precedence if any lesion is malignant")

Total JPEG files indexed: 6774

BEFORE DEDUPLICATION
Train rows: 1318 | Unique mammograms: 1231
Train mammograms with conflicting lesion labels: 6
Test rows: 378 | Unique mammograms: 361
Test mammograms with conflicting lesion labels: 2

```

AFTER DEDUPLICATION

Train rows: 1231 | Unique mammograms: 1231

Train malignant: 602 | Benign: 629

Test rows: 361 | Unique mammograms: 361

Test malignant: 145 | Benign: 216

- ✓ Duplicate mammogram rows removed
- ✓ Conflicting lesion labels collapsed to one image-level label
- ✓ Malignant takes precedence if any lesion is malignant

Section 4: Train / Validation / Test Split

A patient-grouped train/validation split is created using `GroupShuffleSplit` to prevent images from the same patient appearing in both partitions. The original CBIS-DDSM test partition is retained as the held-out test set.

Validation is used for model development, comparison and threshold selection, while the held-out test set is reserved for final evaluation after model selection.

Patient-grouped train/val/test split

```
from sklearn.model_selection import GroupShuffleSplit
from sklearn.utils.class_weight import compute_class_weight

# Find split with validation prevalence closest to development set
overall_rate = train_csv["label"].mean()
best_seed, best_diff = None, np.inf

for seed in range(50):
    gss = GroupShuffleSplit(n_splits=1, test_size=0.2,
                           random_state=seed)
    _, val_idx = next(gss.split(train_csv,
                                groups=train_csv["patient_id"]))
    diff = abs(train_csv.iloc[val_idx]["label"].mean() - overall_rate)

    if diff < best_diff:
        best_seed, best_diff = seed, diff

print(f"Overall development-set malignant rate: {overall_rate*100:.1f}%")
print(f"Best seed found: {best_seed}")

# Final patient-grouped split
gss = GroupShuffleSplit(n_splits=1, test_size=0.2,
                        random_state=best_seed)
train_idx, val_idx = next(gss.split(train_csv,
                                     groups=train_csv["patient_id"]))

train_df = train_csv.iloc[train_idx].reset_index(drop=True)
```

```

val_df = train_csv.iloc[val_idx].reset_index(drop=True)

X_train = train_df["full_path"].values
y_train = train_df["label"].values.astype(np.float32)

X_val = val_df["full_path"].values
y_val = val_df["label"].values.astype(np.float32)

X_test = test_csv["full_path"].values
y_test = test_csv["label"].values.astype(np.float32)

print(f"\nTrain: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")
print(
    f"Malignant prevalence - Train: {y_train.mean()*100:.1f}% | "
    f"Val: {y_val.mean()*100:.1f}% | Test: {y_test.mean()*100:.1f}%"
)

# Leakage checks
splits = [
    ("train", train_df, X_train),
    ("val", val_df, X_val),
    ("test", test_csv, X_test),
]

patient_sets = [set(df["patient_id"]) for _, df, _ in splits]
path_sets = [set(paths) for _, _, paths in splits]

assert not (patient_sets[0] & patient_sets[1])
assert not (patient_sets[0] & patient_sets[2])
assert not (patient_sets[1] & patient_sets[2])

assert not (path_sets[0] & path_sets[1])
assert not (path_sets[0] & path_sets[2])
assert not (path_sets[1] & path_sets[2])

for _, _, paths in splits:
    assert len(paths) == len(set(paths))

print("\n✓ No patient overlap")
print("✓ No mammogram overlap")
print("✓ No duplicate mammograms within any split")

class_weights = dict(
    enumerate(compute_class_weight("balanced", classes=np.array([0, 1]), y=y_train))
)

print(f"\nClass weights: {class_weights}")

```

Overall development-set malignant rate: 48.9%
Best seed found: 8

Train: 979 | Val: 252 | Test: 361
Malignant prevalence – Train: 48.9% | Val: 48.8% | Test: 40.2%

- ✓ No patient overlap
- ✓ No mammogram overlap
- ✓ No duplicate mammograms within any split

Class weights: {0: np.float64(0.979), 1:
np.float64(1.0219206680584552)}

Section 5: Data Pipeline - 128×128 and 224×224

Mammograms are cropped to remove surrounding black regions and resized with padding to either **128×128** or **224×224** while preserving aspect ratio. The processed arrays are cached for reuse across experiments.

Data augmentation is applied only to the relevant training experiments, while validation and test data remain unaugmented.

Image pipeline with deduplicated caches

```
def load_image(path, label):
    image = tf.image.decode_jpeg(tf.io.read_file(path), channels=3)
    return image, label

def crop_to_content(image, threshold=10):
    gray = tf.image.rgb_to_grayscale(tf.cast(image, tf.float32))[:, :,
0]
    mask = gray > threshold

    row_idx = tf.where(tf.reduce_any(mask, axis=1))
    col_idx = tf.where(tf.reduce_any(mask, axis=0))

    def crop():
        top, bottom = tf.reduce_min(row_idx), tf.reduce_max(row_idx) +
1
        left, right = tf.reduce_min(col_idx), tf.reduce_max(col_idx) +
1
        return image[top:bottom, left:right, :]

    return tf.cond(
        tf.logical_and(tf.size(row_idx) > 0, tf.size(col_idx) > 0),
        crop, lambda: image
    )
```

```

def preprocess(image, label, img_size):
    image = crop_to_content(image)
    image = tf.image.resize_with_pad(image, img_size, img_size)
    return tf.cast(image, tf.float32) / 255.0, label

def augment(image, label):
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_flip_up_down(image)
    image = tf.image.random_brightness(image, 0.1)
    image = tf.image.random_contrast(image, 0.9, 1.1)
    return tf.clip_by_value(image, 0.0, 1.0), label

def build_pipeline(paths, labels, img_size):
    return (
        tf.data.Dataset.from_tensor_slices((paths, labels))
        .map(load_image, num_parallel_calls=AUTOTUNE)
        .map(lambda x, y: preprocess(x, y, img_size),
num_parallel_calls=AUTOTUNE)
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE)
    )

def cache_dataset(ds, name):
    batches = list(ds.as_numpy_iterator())
    images = np.concatenate([x for x, _ in batches])
    labels = np.concatenate([y for _, y in batches])

    np.save(f"{BASE}/{name}_images.npy", images)
    np.save(f"{BASE}/{name}_labels.npy", labels)
    print(f"Saved {name} - {len(labels)} samples")

def load_cached(name, shuffle=False, augmentation=False):
    images = np.load(f"{BASE}/{name}_images.npy")
    labels = np.load(f"{BASE}/{name}_labels.npy")

    ds = tf.data.Dataset.from_tensor_slices((images, labels))

    if shuffle:
        ds = ds.shuffle(len(labels), seed=SEED,
reshuffle_each_iteration=True)

    if augmentation:
        ds = ds.map(augment, num_parallel_calls=AUTOTUNE)

    return ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)

```

```

def get_dataset(name, paths, labels, img_size, shuffle=False,
augmentation=False):

    cache = f"{name}_{img_size}_dedup_v1"

    if not os.path.exists(f"{BASE}/{cache}_images.npy"):
        print(f"[{name}@{img_size}] building NEW deduplicated
cache...")
        cache_dataset(build_pipeline(paths, labels, img_size), cache)
    else:
        print(f"[{name}@{img_size}] reusing corrected cache")

    return load_cached(cache, shuffle, augmentation)

# 128x128
train_ds_128_plain = get_dataset("train", X_train, y_train,
IMG_SIZE_128, shuffle=True)
train_ds_128_aug = get_dataset(
    "train", X_train, y_train, IMG_SIZE_128, shuffle=True,
augmentation=True
)
val_ds_128 = get_dataset("val", X_val, y_val, IMG_SIZE_128)
test_ds_128 = get_dataset("test", X_test, y_test, IMG_SIZE_128)

# 224x224
train_ds_224_plain = get_dataset("train", X_train, y_train,
IMG_SIZE_224, shuffle=True)
train_ds_224_aug = get_dataset(
    "train", X_train, y_train, IMG_SIZE_224, shuffle=True,
augmentation=True
)
val_ds_224 = get_dataset("val", X_val, y_val, IMG_SIZE_224)
test_ds_224 = get_dataset("test", X_test, y_test, IMG_SIZE_224)

# Original downstream name
train_ds_224 = train_ds_224_aug

print("\n✓ Corrected standard pipelines ready")

[train@128] reusing corrected cache
[train@128] reusing corrected cache
[val@128] reusing corrected cache
[test@128] reusing corrected cache
[train@224] reusing corrected cache
[train@224] reusing corrected cache
[val@224] reusing corrected cache
[test@224] reusing corrected cache

```

✓ Corrected standard pipelines ready

Section 6: Evaluation and Plotting Utilities

Reusable functions are defined to ensure consistent evaluation across all experiments. Performance is measured using accuracy, sensitivity, specificity, precision, ROC-AUC and confusion matrices.

Decision thresholds are selected from validation predictions using **Youden's J statistic**, allowing the chosen threshold to be frozen when required for later evaluation.

```
# Evaluation utilities
```

```
def evaluate_model(
    model, dataset, name="Model", threshold=0.5,
    split_name="Validation", verbose=True
):
    y_prob = model.predict(dataset, verbose=0).ravel()
    y_true = np.concatenate([y.numpy() for _, y in
dataset]).astype(int)
    y_pred = (y_prob >= threshold).astype(int)

    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    results = {
        "model": name,
        "threshold": round(float(threshold), 4),
        "accuracy": round((tp + tn) / len(y_true), 4),
        "sensitivity": round(tp / (tp + fn), 4) if tp + fn else 0.0,
        "specificity": round(tn / (tn + fp), 4) if tn + fp else 0.0,
        "precision": round(tp / (tp + fp), 4) if tp + fp else 0.0,
        "auc": round(roc_auc_score(y_true, y_prob), 4),
        "cm": cm,
        "y_prob": y_prob,
        "y_true": y_true,
    }

    if verbose:
        print(
            f"\n{'='*60}\n{name} - {split_name} Results "
            f"(threshold={threshold:.3f})\n{'='*60}"
        )
        print(
            classification_report(y_true, y_pred,
target_names=["Benign", "Malignant"])
```

```

    )
    print(f"Accuracy      : {results['accuracy']}")
    print(
        f"Sensitivity : {results['sensitivity']} "
        f"(missed {fn} of {fn+tp} malignant cases)"
    )
    print(f"Specificity : {results['specificity']}")
    print(f"Precision   : {results['precision']}")
    print(f"AUC        : {results['auc']}")

    return results

def find_best_threshold(model, val_ds, verbose=True):
    y_prob = model.predict(val_ds, verbose=0).ravel()
    y_true = np.concatenate([y.numpy() for _, y in
val_ds]).astype(int)

    fpr, tpr, thresholds = roc_curve(y_true, y_prob)
    i = int(np.argmax(tpr - fpr))
    threshold = float(np.clip(thresholds[i], 0.0, 1.0))

    if verbose:
        print(
            f"Optimal threshold (Youden's J): {threshold:.3f} "
            f"(val sensitivity={tpr[i]:.3f}, "
            f"val specificity={1-fpr[i]:.3f})"
        )

    return threshold

def plot_confusion_matrix(cm, title, save_path=None):
    fig, ax = plt.subplots(figsize=(5, 5))

    ConfusionMatrixDisplay(cm, display_labels=["Benign",
"Malignant"]).plot(
        ax=ax, colorbar=False, cmap="Blues", values_format="d"
    )

    ax.set_title(title, fontweight="bold")
    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")

    plt.show()

def plot_training_curves(history, title, save_path=None):
    hist = history.history if hasattr(history, "history") else history

```

```

    metrics = [m for m in ["loss", "accuracy", "auc", "sensitivity"]
if m in hist]

    fig, axes = plt.subplots(1, len(metrics), figsize=(5 *
len(metrics), 4))
    axes = [axes] if len(metrics) == 1 else axes

    for ax, metric in zip(axes, metrics):
        ax.plot(hist[metric], label="Train")

        if f"val_{metric}" in hist:
            ax.plot(hist[f"val_{metric}"], label="Val")

        ax.set_title(metric.upper())
        ax.set_xlabel("Epoch")
        ax.legend()

plt.suptitle(title, fontweight="bold")
plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches="tight")

plt.show()

```

```
print("Evaluation utilities ready.")
```

Evaluation utilities ready.

```

def plot_roc_curve(results, title=None, save_path=None):
    """Plot the ROC curve for a single model's results dict (as
    returned by
    evaluate_model, which now also stores y_prob/y_true). This is the
    threshold-independent counterpart to the confusion matrix - reused
    for
    every transfer-learning model from here on (VGG16, ResNet50,
    etc.).
    """
    fpr, tpr, _ = roc_curve(results["y_true"], results["y_prob"])
    plt.figure(figsize=(5.5, 5.5))
    plt.plot(
        fpr, tpr, linewidth=2, label=f"{results['model']} (AUC =
{results['auc']:.3f})"
    )
    plt.plot([0, 1], [0, 1], "k--", linewidth=1, label="Chance")
    plt.xlabel("False Positive Rate (1 - Specificity)")
    plt.ylabel("True Positive Rate (Sensitivity)")
    plt.title(title or f"ROC Curve - {results['model']}",
fontweight="bold")
    plt.legend(loc="lower right")

```

```
plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
plt.show()

print("ROC-curve utility ready.")
ROC-curve utility ready.
```

Section 7: Grad-CAM Utilities

Reusable Grad-CAM functions are defined to visualise image regions contributing to model predictions. These utilities support qualitative interpretability analysis during model development.

A separate analysis later examines the final ResNet50 + CLAHE model in greater detail using both prediction cases and lesion-mask localisation.

```
def make_gradcam(img_array, model):
    last_conv = None

    for layer in model.layers:
        if isinstance(layer, tf.keras.layers.Conv2D):
            last_conv = layer.name

    if last_conv is None:
        raise ValueError("No Conv2D layer found in model.")

    if isinstance(model, tf.keras.Sequential):
        # Rebuild a simple functional graph for Sequential models
        img_input = tf.keras.Input(shape=img_array.shape[1:])
        x = img_input
        conv_output = None

        for layer in model.layers:
            if isinstance(layer, tf.keras.layers.InputLayer):
                continue
            x = layer(x)
            if layer.name == last_conv:
                conv_output = x

        grad_model = tf.keras.Model(inputs=img_input,
                                     outputs=[conv_output, x])
    else:
        # Functional transfer models already expose their graph
        grad_model = tf.keras.Model(
            inputs=model.input,
            outputs=[model.get_layer(last_conv).output, model.output],
        )
```

```

    with tf.GradientTape() as tape:
        conv_out, preds = grad_model(tf.cast(img_array, tf.float32),
training=False)
        loss = tf.reduce_sum(preds[:, 0])

        grads = tape.gradient(loss, conv_out)

        if grads is None:
            raise ValueError("Could not calculate Grad-CAM gradients.")

        pooled = tf.reduce_mean(grads, axis=(0, 1, 2))
        heatmap = conv_out[0] @ pooled[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)
        heatmap = tf.maximum(heatmap, 0)
        heatmap = heatmap / (tf.math.reduce_max(heatmap) + 1e-8)

    return heatmap.numpy()

def prepare_gradcam_image(img_path, img_size, model_type="standard"):
    # Match the preprocessing used by the training pipeline
    image = tf.io.read_file(img_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = crop_to_content(image)
    image = tf.image.resize_with_pad(image, img_size, img_size)

    display_img = tf.cast(image, tf.float32) / 255.0

    if model_type == "vgg16":
        model_img = vgg16_preprocess(tf.cast(image, tf.float32))
    elif model_type == "resnet50":
        model_img = resnet50_preprocess(tf.cast(image, tf.float32))
    else:
        model_img = display_img

    return model_img.numpy(), display_img.numpy()

# Fixed set of 4 test cases, reused for every model
mal_samples = test_csv[test_csv["label"] == 1]
["full_path"].iloc[:2].tolist()
ben_samples = test_csv[test_csv["label"] == 0]
["full_path"].iloc[:2].tolist()
GRADCAM_SAMPLES = mal_samples + ben_samples
GRADCAM_LABELS = ["MALIGNANT", "MALIGNANT", "BENIGN", "BENIGN"]

def plot_gradcam_grid(
    model, model_name, img_size, threshold=0.5, model_type="standard",
    save_path=None

```

```

):
    fig, axes = plt.subplots(4, 3, figsize=(12, 16))

    for i, (path, true) in enumerate(zip(GRADCAM_SAMPLES,
GRADCAM_LABELS)):
        model_img, display_img = prepare_gradcam_image(
            path, img_size, model_type=model_type
        )

        arr = np.expand_dims(model_img, 0).astype(np.float32)
        prob = float(model.predict(arr, verbose=0)[0][0])

        pred = "MALIGNANT" if prob >= threshold else "BENIGN"
        mark = "CORRECT" if pred == true else "WRONG"
        color = "green" if mark == "CORRECT" else "red"

        heatmap = make_gradcam(arr, model)
        hm_r = (
            tf.image.resize(heatmap[..., np.newaxis], (img_size,
img_size))
            .numpy()
            .squeeze()
        )
        colored = plt.get_cmap("jet")(hm_r)[:, :, :3]
        overlay = np.clip(colored * 0.4 + display_img * 0.6, 0, 1)

        axes[i][0].imshow(display_img, cmap="gray")
        axes[i][0].set_title(f"True: {true}", fontsize=9)
        axes[i][0].axis("off")

        axes[i][1].imshow(heatmap, cmap="jet")
        axes[i][1].set_title("Grad-CAM Heatmap", fontsize=9)
        axes[i][1].axis("off")

        axes[i][2].imshow(overlay)
        axes[i][2].set_title(
            f"Pred: {pred} ({prob:.2f}) [{mark}]", fontsize=9,
color=color
        )
        axes[i][2].axis("off")

    plt.suptitle(
        f"Grad-CAM Visualisation - {model_name}\n"
        f"Left: Input | Middle: Heatmap | Right: Overlay",
        fontsize=12,
        fontweight="bold",
    )
    plt.tight_layout()

    if save_path:

```

```
plt.savefig(save_path, dpi=150, bbox_inches="tight")

plt.show()

print("Grad-CAM utilities ready.")

Grad-CAM utilities ready.
```

Section 8: Training and Checkpointing

Training utilities use early stopping and model checkpointing to retain the best-performing model based on validation performance. Saved .keras checkpoints are then reused for subsequent evaluation.

Transfer-learning models additionally use staged training, beginning with a frozen pretrained base before selective fine-tuning.

```
def train_model(
    model,
    train_ds,
    val_ds,
    checkpoint_path,
    epochs=EPOCHS,
    patience=7,
    class_weight=None,
):
    callbacks = [
        EarlyStopping(
            monitor="val_auc",
            patience=patience,
            restore_best_weights=True,
            mode="max",
            verbose=1,
        ),
        ModelCheckpoint(
            filepath=checkpoint_path,
            monitor="val_auc",
            save_best_only=True,
            mode="max",
            verbose=1,
        ),
    ]
    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1,
```

```

        class_weight=class_weight,
    )
    best_epoch = int(np.argmax(history.history["val_auc"]) + 1)
    print(
        f"\nBest epoch: {best_epoch} | Best val_auc: "
        f"{max(history.history['val_auc']):.4f}"
    )
    return history, best_epoch

```

Section 9: Baseline CNN - Architecture

The baseline model is a **128×128 CNN** with three convolutional blocks followed by global average pooling, a dense layer and a sigmoid output. No dropout or data augmentation is applied, providing an unregularised reference model for the subsequent experiments.

```

def build_baseline_cnn(img_size=IMG_SIZE_128, name="Baseline_CNN"):
    model = tf.keras.Sequential(
        [
            layers.Conv2D(
                32, (3, 3), activation="relu", input_shape=(img_size,
img_size, 3)
            ),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(64, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(128, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.GlobalAveragePooling2D(),
            layers.Dense(128, activation="relu"),
            layers.Dense(1, activation="sigmoid"),
        ],
        name=name,
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )
    return model

baseline_model = build_baseline_cnn(IMG_SIZE_128)
baseline_model.summary()

```

Model: "Baseline_CNN"

Layer (type) Param #	Output Shape
conv2d (Conv2D) 896	(None, 126, 126, 32)
max_pooling2d (MaxPooling2D) 0	(None, 63, 63, 32)
conv2d_1 (Conv2D) 18,496	(None, 61, 61, 64)
max_pooling2d_1 (MaxPooling2D) 0	(None, 30, 30, 64)
conv2d_2 (Conv2D) 73,856	(None, 28, 28, 128)
max_pooling2d_2 (MaxPooling2D) 0	(None, 14, 14, 128)
global_average_pooling2d (GlobalAveragePooling2D) 0	(None, 128)
dense (Dense) 16,512	(None, 128)
dense_1 (Dense) 129	(None, 1)

Total params: 109,889 (429.25 KB)

Trainable params: 109,889 (429.25 KB)

Non-trainable params: 0 (0.00 B)

Section 10: Train Baseline CNN

The baseline CNN is trained on the 128×128 training set using class weighting. Early stopping and model checkpointing monitor validation AUC so that the best-performing model state is retained.

```
# Baseline uses the plain 128px training set
baseline_history, baseline_best_epoch = train_model(
    baseline_model,
    train_ds_128_plain,
    val_ds_128,
    checkpoint_path=f"{MODELS_DIR}/baseline_cnn_best.keras",
    class_weight=class_weights,
)

Epoch 1/30
28/31 _____ 0s 23ms/step - accuracy: 0.6148 - auc:
0.6721 - loss: 0.6560 - precision: 0.5875 - sensitivity: 0.7814
Epoch 1: val_auc improved from None to 0.68813, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn
n_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn
n_best.keras
31/31 _____ 1s 31ms/step - accuracy: 0.6057 - auc:
0.6437 - loss: 0.6609 - precision: 0.5917 - sensitivity: 0.6263 -
val_accuracy: 0.6548 - val_auc: 0.6881 - val_loss: 0.6441 -
val_precision: 0.6500 - val_sensitivity: 0.6341
Epoch 2/30
28/31 _____ 0s 20ms/step - accuracy: 0.6269 - auc:
0.6306 - loss: 0.6700 - precision: 0.6241 - sensitivity: 0.6875
Epoch 2: val_auc did not improve from 0.68813
31/31 _____ 1s 26ms/step - accuracy: 0.6343 - auc:
0.6531 - loss: 0.6581 - precision: 0.6179 - sensitivity: 0.6618 -
val_accuracy: 0.6548 - val_auc: 0.6877 - val_loss: 0.6435 -
val_precision: 0.6385 - val_sensitivity: 0.6748
Epoch 3/30
31/31 _____ 0s 20ms/step - accuracy: 0.6066 - auc:
0.6457 - loss: 0.6611 - precision: 0.5726 - sensitivity: 0.6747
Epoch 3: val_auc did not improve from 0.68813
31/31 _____ 1s 24ms/step - accuracy: 0.6323 - auc:
0.6632 - loss: 0.6516 - precision: 0.6108 - sensitivity: 0.6848 -
val_accuracy: 0.6429 - val_auc: 0.6837 - val_loss: 0.6425 -
val_precision: 0.6260 - val_sensitivity: 0.6667
Epoch 4/30
30/31 _____ 0s 22ms/step - accuracy: 0.6324 - auc:
```

0.6716 - loss: 0.6495 - precision: 0.6111 - sensitivity: 0.6544
Epoch 4: val_auc did not improve from 0.68813
31/31 _____ 1s 24ms/step - accuracy: 0.6302 - auc:
0.6605 - loss: 0.6528 - precision: 0.6154 - sensitivity: 0.6514 -
val_accuracy: 0.6349 - val_auc: 0.6841 - val_loss: 0.6446 -
val_precision: 0.6422 - val_sensitivity: 0.5691
Epoch 5/30
28/31 _____ 0s 19ms/step - accuracy: 0.6166 - auc:
0.6611 - loss: 0.6539 - precision: 0.5936 - sensitivity: 0.6795
Epoch 5: val_auc improved from 0.68813 to 0.69074, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn
n_best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn
n_best.keras
31/31 _____ 1s 24ms/step - accuracy: 0.6221 - auc:
0.6658 - loss: 0.6508 - precision: 0.6054 - sensitivity: 0.6534 -
val_accuracy: 0.6548 - val_auc: 0.6907 - val_loss: 0.6392 -
val_precision: 0.6385 - val_sensitivity: 0.6748
Epoch 6/30
28/31 _____ 0s 19ms/step - accuracy: 0.6012 - auc:
0.6678 - loss: 0.6509 - precision: 0.5633 - sensitivity: 0.6060
Epoch 6: val_auc did not improve from 0.69074
31/31 _____ 1s 21ms/step - accuracy: 0.6210 - auc:
0.6668 - loss: 0.6490 - precision: 0.6102 - sensitivity: 0.6242 -
val_accuracy: 0.6746 - val_auc: 0.6857 - val_loss: 0.6392 -
val_precision: 0.6667 - val_sensitivity: 0.6667
Epoch 7/30
28/31 _____ 0s 19ms/step - accuracy: 0.6644 - auc:
0.6957 - loss: 0.6322 - precision: 0.6421 - sensitivity: 0.7520
Epoch 7: val_auc did not improve from 0.69074
31/31 _____ 1s 24ms/step - accuracy: 0.6313 - auc:
0.6683 - loss: 0.6458 - precision: 0.6105 - sensitivity: 0.6806 -
val_accuracy: 0.6508 - val_auc: 0.6803 - val_loss: 0.6404 -
val_precision: 0.6400 - val_sensitivity: 0.6504
Epoch 8/30
28/31 _____ 0s 19ms/step - accuracy: 0.6393 - auc:
0.6796 - loss: 0.6409 - precision: 0.6073 - sensitivity: 0.6852
Epoch 8: val_auc did not improve from 0.69074
31/31 _____ 1s 22ms/step - accuracy: 0.6384 - auc:
0.6771 - loss: 0.6433 - precision: 0.6238 - sensitivity: 0.6576 -
val_accuracy: 0.6429 - val_auc: 0.6850 - val_loss: 0.6405 -
val_precision: 0.6571 - val_sensitivity: 0.5610
Epoch 9/30
28/31 _____ 0s 19ms/step - accuracy: 0.6085 - auc:
0.6448 - loss: 0.6578 - precision: 0.5780 - sensitivity: 0.6198
Epoch 9: val_auc improved from 0.69074 to 0.69455, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn

n_best.keras

Epoch 9: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_cn
n_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.6292 - auc:
0.6784 - loss: 0.6415 - precision: 0.6120 - sensitivity: 0.6618 -
val_accuracy: 0.6389 - val_auc: 0.6946 - val_loss: 0.6331 -
val_precision: 0.6176 - val_sensitivity: 0.6829

Epoch 10/30

28/31 ————— 0s 18ms/step - accuracy: 0.6327 - auc:
0.6660 - loss: 0.6454 - precision: 0.6196 - sensitivity: 0.6526

Epoch 10: val_auc did not improve from 0.69455

31/31 ————— 1s 21ms/step - accuracy: 0.6404 - auc:
0.6848 - loss: 0.6369 - precision: 0.6224 - sensitivity: 0.6743 -
val_accuracy: 0.6508 - val_auc: 0.6846 - val_loss: 0.6385 -
val_precision: 0.6667 - val_sensitivity: 0.5691

Epoch 11/30

28/31 ————— 0s 18ms/step - accuracy: 0.6456 - auc:
0.6924 - loss: 0.6371 - precision: 0.6630 - sensitivity: 0.6217

Epoch 11: val_auc did not improve from 0.69455

31/31 ————— 1s 21ms/step - accuracy: 0.6333 - auc:
0.6840 - loss: 0.6394 - precision: 0.6245 - sensitivity: 0.6284 -
val_accuracy: 0.6468 - val_auc: 0.6837 - val_loss: 0.6413 -
val_precision: 0.6133 - val_sensitivity: 0.7480

Epoch 12/30

28/31 ————— 0s 18ms/step - accuracy: 0.6537 - auc:
0.6789 - loss: 0.6374 - precision: 0.6355 - sensitivity: 0.6914

Epoch 12: val_auc did not improve from 0.69455

31/31 ————— 1s 23ms/step - accuracy: 0.6527 - auc:
0.6923 - loss: 0.6333 - precision: 0.6334 - sensitivity: 0.6889 -
val_accuracy: 0.6389 - val_auc: 0.6882 - val_loss: 0.6360 -
val_precision: 0.6290 - val_sensitivity: 0.6341

Epoch 13/30

28/31 ————— 0s 18ms/step - accuracy: 0.6584 - auc:
0.7003 - loss: 0.6316 - precision: 0.6473 - sensitivity: 0.6892

Epoch 13: val_auc did not improve from 0.69455

31/31 ————— 1s 21ms/step - accuracy: 0.6537 - auc:
0.6913 - loss: 0.6340 - precision: 0.6509 - sensitivity: 0.6305 -
val_accuracy: 0.6190 - val_auc: 0.6817 - val_loss: 0.6488 -
val_precision: 0.5849 - val_sensitivity: 0.7561

Epoch 14/30

28/31 ————— 0s 18ms/step - accuracy: 0.6704 - auc:
0.7214 - loss: 0.6164 - precision: 0.6653 - sensitivity: 0.6993

Epoch 14: val_auc did not improve from 0.69455

31/31 ————— 1s 21ms/step - accuracy: 0.6660 - auc:
0.7119 - loss: 0.6214 - precision: 0.6538 - sensitivity: 0.6743 -
val_accuracy: 0.6429 - val_auc: 0.6865 - val_loss: 0.6360 -
val_precision: 0.6435 - val_sensitivity: 0.6016

```

Epoch 15/30
28/31 _____ 0s 18ms/step - accuracy: 0.6595 - auc:
0.7233 - loss: 0.6142 - precision: 0.6541 - sensitivity: 0.6520
Epoch 15: val_auc did not improve from 0.69455
31/31 _____ 1s 20ms/step - accuracy: 0.6639 - auc:
0.7197 - loss: 0.6177 - precision: 0.6569 - sensitivity: 0.6555 -
val_accuracy: 0.6230 - val_auc: 0.6832 - val_loss: 0.6451 -
val_precision: 0.6795 - val_sensitivity: 0.4309
Epoch 16/30
28/31 _____ 0s 18ms/step - accuracy: 0.6620 - auc:
0.7436 - loss: 0.6036 - precision: 0.6899 - sensitivity: 0.5755
Epoch 16: val_auc did not improve from 0.69455
31/31 _____ 1s 21ms/step - accuracy: 0.6547 - auc:
0.7122 - loss: 0.6199 - precision: 0.6636 - sensitivity: 0.5971 -
val_accuracy: 0.6548 - val_auc: 0.6833 - val_loss: 0.6397 -
val_precision: 0.6233 - val_sensitivity: 0.7398
Epoch 16: early stopping
Restoring model weights from the end of the best epoch: 9.

Best epoch: 9 | Best val_auc: 0.6946

```

Section 11: Baseline CNN - Training and Validation Performance

Training curves are examined to assess learning behaviour and potential overfitting. The baseline model is then evaluated on the validation set using a threshold selected with Youden's J statistic, establishing a reference for subsequent experiments.

```

# Baseline CNN validation results (no retraining needed)

import tensorflow as tf

# Load saved best model
baseline_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/baseline_cnn_best.keras", compile=False
)

print("Loaded saved Baseline CNN model.")

# Validation threshold
baseline_threshold = find_best_threshold(baseline_model, val_ds_128)

# Validation evaluation
baseline_results = evaluate_model(
    baseline_model,

```

```

val_ds_128,
name="Baseline CNN (128px)",
threshold=baseline_threshold,
split_name="Validation",
)

# Validation confusion matrix

plot_confusion_matrix(
    baseline_results["cm"],
    "Baseline CNN - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_baseline_validation.png",
)

Loaded saved Baseline CNN model.
Optimal threshold (Youden's J): 0.531 (val sensitivity=0.634, val
specificity=0.698)

=====
Baseline CNN (128px) – Validation Results (threshold=0.531)
=====

```

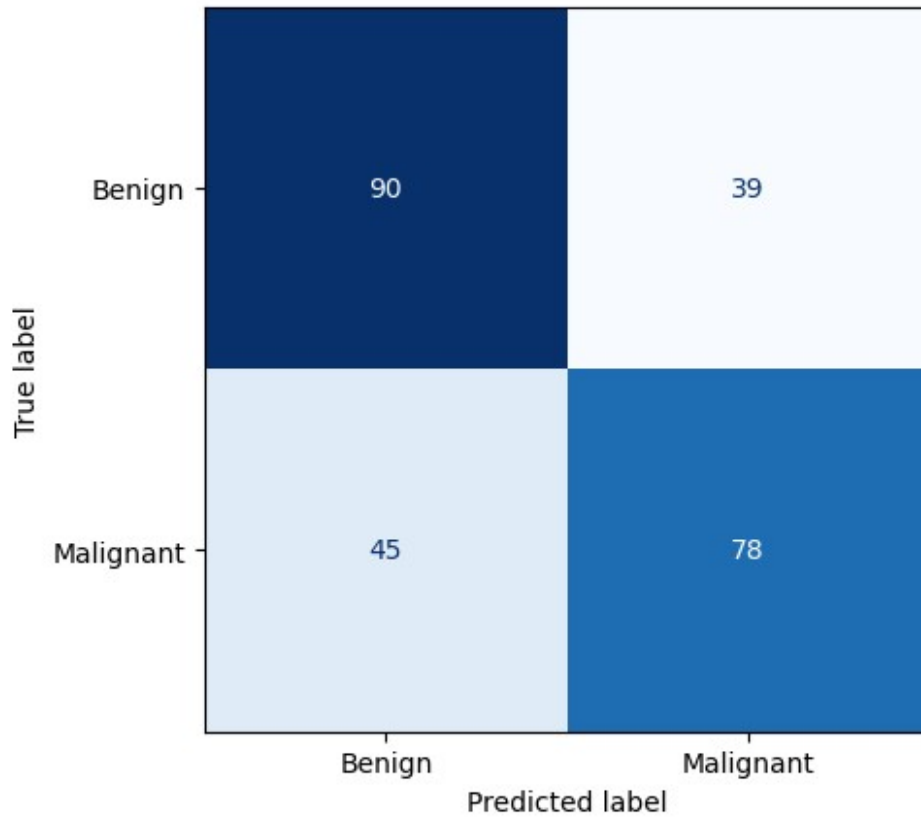
	precision	recall	f1-score	support
Benign	0.67	0.70	0.68	129
Malignant	0.67	0.63	0.65	123
accuracy			0.67	252
macro avg	0.67	0.67	0.67	252
weighted avg	0.67	0.67	0.67	252

```

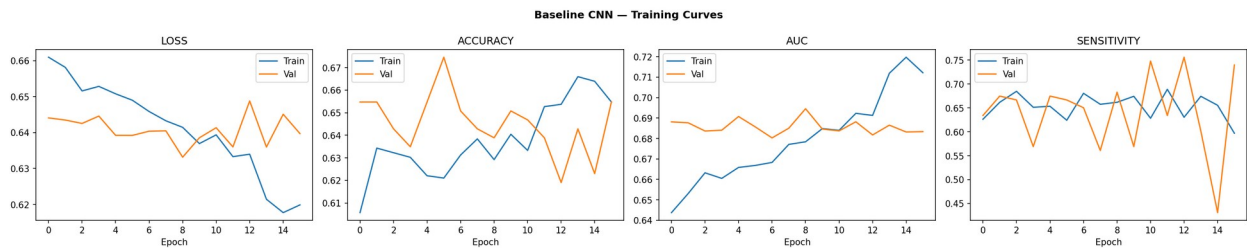
Accuracy      : 0.6667
Sensitivity   : 0.6341 (missed 45 of 123 malignant cases)
Specificity   : 0.6977
Precision     : 0.6667
AUC           : 0.6938

```

Baseline CNN — Validation Confusion Matrix

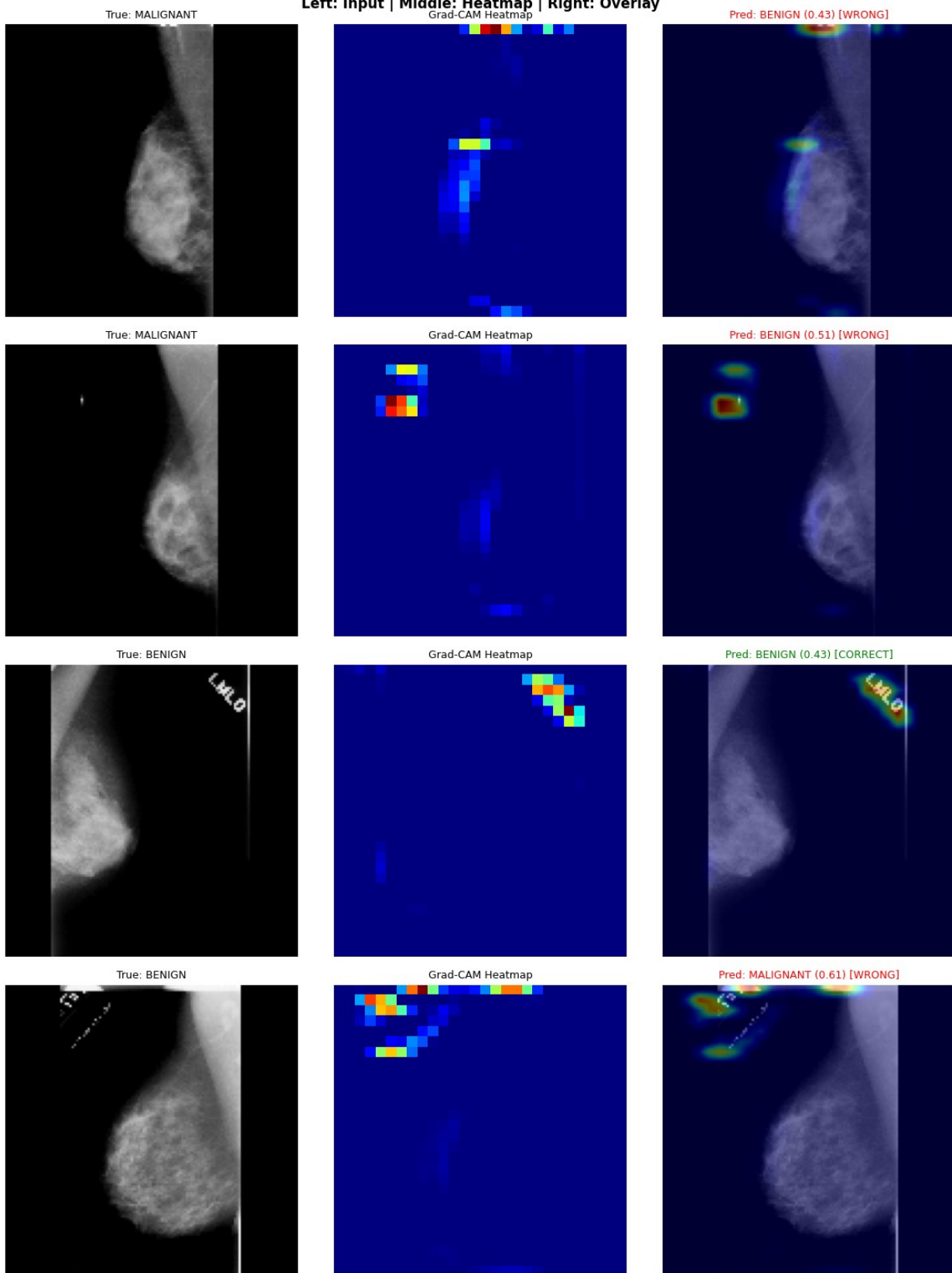


```
# Original training curves
from IPython.display import Image, display
display(Image(filename=f"{BASE}/curves_baseline.png"))
```



```
plot_gradcam_grid(
    baseline_model,
    "Baseline CNN",
    IMG_SIZE_128,
    threshold=baseline_threshold,
    save_path=f"{BASE}/gradcam_baseline.png",
)
```

Grad-CAM Visualisation — Baseline CNN Left: Input | Middle: Heatmap | Right: Overlay



Section 12: Network Scaling Experiment

The baseline CNN is made deeper and wider while retaining the **128×128 input resolution**. This controlled experiment examines whether increasing model capacity improves validation performance and generalisation.

```
# Network scaling experiment: train + validate

def make_cnn(scaled=False):
    filters = [32, 64, 128, 256] if scaled else [32, 64, 128]

    model = tf.keras.Sequential([layers.Input((128, 128, 3))])

    for f in filters:
        model.add(layers.Conv2D(f, 3, activation="relu"))
        model.add(layers.MaxPooling2D())

    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(256 if scaled else 128, activation="relu"))
    model.add(layers.Dense(1, activation="sigmoid"))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-3),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )

    return model

# Build models

baseline_scaling_model = make_cnn(scaled=False)
scaled_model = make_cnn(scaled=True)

# Train both models (data: train_ds_128_plain, monitor: val_ds_128)

baseline_scaling_history, _ = train_model(
    baseline_scaling_model,
    train_ds_128_plain,
    val_ds_128,
    f"{MODELS_DIR}/baseline_scaling_best.keras",
    class_weight=class_weights,
```

```

)

scaled_history, _ = train_model(
    scaled_model,
    train_ds_128_plain,
    val_ds_128,
    f"{MODELS_DIR}/scaled_cnn_best.keras",
    class_weight=class_weights,
)

# Select thresholds using validation set only

baseline_threshold = find_best_threshold(baseline_scaling_model,
val_ds_128)

scaled_threshold = find_best_threshold(scaled_model, val_ds_128)

# Evaluate on validation set only

baseline_scaling_results = evaluate_model(
    baseline_scaling_model,
    val_ds_128,
    name="Baseline CNN (128px)",
    threshold=baseline_threshold,
    split_name="Validation",
)

scaled_results = evaluate_model(
    scaled_model,
    val_ds_128,
    name="Scaled CNN (128px)",
    threshold=scaled_threshold,
    split_name="Validation",
)

# Validation-only network scaling comparison

print("\n" + "=" * 60)
print("NETWORK SCALING - VALIDATION COMPARISON")
print("=" * 60)

for name, r in [
    ("Baseline CNN (128px)", baseline_scaling_results),
    ("Scaled CNN (128px)", scaled_results),
]:
    print(f"\n{name}")
    print(f"Threshold      : {r['threshold']:.4f}")

```

```

print(f"Accuracy      : {r['accuracy']:.4f}")
print(f"Sensitivity   : {r['sensitivity']:.4f}")
print(f"Specificity    : {r['specificity']:.4f}")
print(f"Precision     : {r['precision']:.4f}")
print(f"AUC          : {r['auc']:.4f}")

```

```

auc_change = scaled_results["auc"] - baseline_scaling_results["auc"]

```

```

print(
    f"\nValidation AUC change: "
    f"{baseline_scaling_results['auc']:.4f} → "
    f"{scaled_results['auc']:.4f} "
    f"({auc_change:+.4f})"
)

```

Epoch 1/30

```

31/31 ————— 0s 98ms/step - accuracy: 0.4867 - auc:
0.4687 - loss: 0.6955 - precision: 0.4878 - sensitivity: 0.8367

```

Epoch 1: val_auc improved from None to 0.50000, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 1: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

```

31/31 ————— 9s 178ms/step - accuracy: 0.4709 - auc:
0.4742 - loss: 0.6946 - precision: 0.4686 - sensitivity: 0.6075 -
val_accuracy: 0.5119 - val_auc: 0.5000 - val_loss: 0.6929 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

```

Epoch 2/30

```

31/31 ————— 0s 19ms/step - accuracy: 0.4883 - auc:
0.4735 - loss: 0.6932 - precision: 0.3586 - sensitivity: 0.2342

```

Epoch 2: val_auc improved from 0.50000 to 0.55701, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 2: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

```

31/31 ————— 1s 27ms/step - accuracy: 0.4780 - auc:
0.4547 - loss: 0.6938 - precision: 0.4365 - sensitivity: 0.2296 -
val_accuracy: 0.5119 - val_auc: 0.5570 - val_loss: 0.6930 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

```

Epoch 3/30

```

28/31 ————— 0s 18ms/step - accuracy: 0.5182 - auc:
0.5024 - loss: 0.6928 - precision: 0.4943 - sensitivity: 0.5078

```

Epoch 3: val_auc improved from 0.55701 to 0.55908, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 3: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

31/31 ————— 1s 22ms/step - accuracy: 0.5015 - auc:
0.5083 - loss: 0.6932 - precision: 0.4937 - sensitivity: 0.7349 -
val_accuracy: 0.4881 - val_auc: 0.5591 - val_loss: 0.6931 -
val_precision: 0.4881 - val_sensitivity: 1.0000

Epoch 4/30

28/31 ————— 0s 18ms/step - accuracy: 0.5101 - auc:
0.5055 - loss: 0.6930 - precision: 0.4997 - sensitivity: 0.8325

Epoch 4: val_auc did not improve from 0.55908

31/31 ————— 1s 23ms/step - accuracy: 0.5046 - auc:
0.4905 - loss: 0.6936 - precision: 0.4930 - sensitivity: 0.4426 -
val_accuracy: 0.5119 - val_auc: 0.5428 - val_loss: 0.6927 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 5/30

29/31 ————— 0s 18ms/step - accuracy: 0.5551 - auc:
0.5123 - loss: 0.6918 - precision: 0.4404 - sensitivity: 0.2644

Epoch 5: val_auc improved from 0.55908 to 0.60049, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

31/31 ————— 1s 22ms/step - accuracy: 0.5322 - auc:
0.5087 - loss: 0.6930 - precision: 0.5205 - sensitivity: 0.5553 -
val_accuracy: 0.5357 - val_auc: 0.6005 - val_loss: 0.6923 -
val_precision: 0.5152 - val_sensitivity: 0.8293

Epoch 6/30

30/31 ————— 0s 18ms/step - accuracy: 0.5134 - auc:
0.5691 - loss: 0.6917 - precision: 0.4840 - sensitivity: 0.7379

Epoch 6: val_auc improved from 0.60049 to 0.64202, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

31/31 ————— 1s 22ms/step - accuracy: 0.5322 - auc:
0.5494 - loss: 0.6931 - precision: 0.5157 - sensitivity: 0.7203 -
val_accuracy: 0.5556 - val_auc: 0.6420 - val_loss: 0.6893 -
val_precision: 0.5266 - val_sensitivity: 0.8862

Epoch 7/30

29/31 ————— 0s 17ms/step - accuracy: 0.5311 - auc:
0.5603 - loss: 0.6902 - precision: 0.5075 - sensitivity: 0.4119

Epoch 7: val_auc improved from 0.64202 to 0.65157, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

31/31 ————— 1s 25ms/step - accuracy: 0.5291 - auc:
0.5482 - loss: 0.6903 - precision: 0.5203 - sensitivity: 0.4823 -
val_accuracy: 0.5675 - val_auc: 0.6516 - val_loss: 0.6835 -
val_precision: 0.5402 - val_sensitivity: 0.7642

Epoch 8/30

28/31 ————— 0s 18ms/step - accuracy: 0.5804 - auc:
0.6064 - loss: 0.6817 - precision: 0.5534 - sensitivity: 0.5931

Epoch 8: val_auc improved from 0.65157 to 0.66900, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 8: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

31/31 ————— 1s 22ms/step - accuracy: 0.5608 - auc:
0.5893 - loss: 0.6820 - precision: 0.5438 - sensitivity: 0.6347 -
val_accuracy: 0.6111 - val_auc: 0.6690 - val_loss: 0.6710 -
val_precision: 0.5668 - val_sensitivity: 0.8618

Epoch 9/30

28/31 ————— 0s 18ms/step - accuracy: 0.5767 - auc:
0.6394 - loss: 0.6692 - precision: 0.5598 - sensitivity: 0.6803

Epoch 9: val_auc did not improve from 0.66900

31/31 ————— 1s 21ms/step - accuracy: 0.5730 - auc:
0.6032 - loss: 0.6781 - precision: 0.5544 - sensitivity: 0.6493 -
val_accuracy: 0.6349 - val_auc: 0.6616 - val_loss: 0.6659 -
val_precision: 0.6703 - val_sensitivity: 0.4959

Epoch 10/30

28/31 ————— 0s 18ms/step - accuracy: 0.5866 - auc:
0.6017 - loss: 0.6770 - precision: 0.5863 - sensitivity: 0.4922

Epoch 10: val_auc did not improve from 0.66900

31/31 ————— 1s 21ms/step - accuracy: 0.5914 - auc:
0.6088 - loss: 0.6752 - precision: 0.5832 - sensitivity: 0.5783 -
val_accuracy: 0.5992 - val_auc: 0.6638 - val_loss: 0.6635 -
val_precision: 0.5632 - val_sensitivity: 0.7967

Epoch 11/30

29/31 ————— 0s 18ms/step - accuracy: 0.6124 - auc:
0.6117 - loss: 0.6754 - precision: 0.5809 - sensitivity: 0.6750

Epoch 11: val_auc did not improve from 0.66900

31/31 ————— 1s 20ms/step - accuracy: 0.5955 - auc:
0.6206 - loss: 0.6701 - precision: 0.5828 - sensitivity: 0.6096 -
val_accuracy: 0.6151 - val_auc: 0.6654 - val_loss: 0.6565 -
val_precision: 0.5942 - val_sensitivity: 0.6667

Epoch 12/30

28/31 ————— 0s 18ms/step - accuracy: 0.6040 - auc:
0.6303 - loss: 0.6663 - precision: 0.5695 - sensitivity: 0.7124

Epoch 12: val_auc did not improve from 0.66900
31/31 ————— 1s 20ms/step - accuracy: 0.6098 - auc:
0.6266 - loss: 0.6676 - precision: 0.5846 - sensitivity: 0.6994 -
val_accuracy: 0.6190 - val_auc: 0.6681 - val_loss: 0.6599 -
val_precision: 0.5789 - val_sensitivity: 0.8049
Epoch 13/30
28/31 ————— 0s 18ms/step - accuracy: 0.5920 - auc:
0.6151 - loss: 0.6724 - precision: 0.5698 - sensitivity: 0.7131
Epoch 13: val_auc improved from 0.66900 to 0.67536, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 13: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras
31/31 ————— 1s 23ms/step - accuracy: 0.6057 - auc:
0.6241 - loss: 0.6681 - precision: 0.5850 - sensitivity: 0.6681 -
val_accuracy: 0.6429 - val_auc: 0.6754 - val_loss: 0.6572 -
val_precision: 0.5965 - val_sensitivity: 0.8293
Epoch 14/30
28/31 ————— 0s 18ms/step - accuracy: 0.6292 - auc:
0.6547 - loss: 0.6567 - precision: 0.6094 - sensitivity: 0.8117
Epoch 14: val_auc did not improve from 0.67536
31/31 ————— 1s 21ms/step - accuracy: 0.5996 - auc:
0.6343 - loss: 0.6663 - precision: 0.5775 - sensitivity: 0.6764 -
val_accuracy: 0.6230 - val_auc: 0.6698 - val_loss: 0.6561 -
val_precision: 0.6296 - val_sensitivity: 0.5528
Epoch 15/30
28/31 ————— 0s 19ms/step - accuracy: 0.5951 - auc:
0.6096 - loss: 0.6732 - precision: 0.5825 - sensitivity: 0.6735
Epoch 15: val_auc did not improve from 0.67536
31/31 ————— 1s 22ms/step - accuracy: 0.5832 - auc:
0.6044 - loss: 0.6740 - precision: 0.5649 - sensitivity: 0.6451 -
val_accuracy: 0.6508 - val_auc: 0.6754 - val_loss: 0.6577 -
val_precision: 0.6882 - val_sensitivity: 0.5203
Epoch 16/30
31/31 ————— 0s 19ms/step - accuracy: 0.5728 - auc:
0.5998 - loss: 0.6791 - precision: 0.5636 - sensitivity: 0.5693
Epoch 16: val_auc improved from 0.67536 to 0.68198, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 16: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras
31/31 ————— 1s 27ms/step - accuracy: 0.6088 - auc:
0.6430 - loss: 0.6616 - precision: 0.5902 - sensitivity: 0.6555 -
val_accuracy: 0.6468 - val_auc: 0.6820 - val_loss: 0.6489 -
val_precision: 0.6371 - val_sensitivity: 0.6423

Epoch 17/30
28/31 ————— 0s 19ms/step - accuracy: 0.5798 - auc: 0.6085 - loss: 0.6762 - precision: 0.5618 - sensitivity: 0.6587
Epoch 17: val_auc did not improve from 0.68198
31/31 ————— 1s 23ms/step - accuracy: 0.6180 - auc: 0.6469 - loss: 0.6586 - precision: 0.6004 - sensitivity: 0.6555 - val_accuracy: 0.6627 - val_auc: 0.6736 - val_loss: 0.6572 - val_precision: 0.7021 - val_sensitivity: 0.5366
Epoch 18/30
31/31 ————— 0s 20ms/step - accuracy: 0.6082 - auc: 0.6357 - loss: 0.6672 - precision: 0.5918 - sensitivity: 0.6106
Epoch 18: val_auc improved from 0.68198 to 0.68362, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 18: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras
31/31 ————— 1s 29ms/step - accuracy: 0.6200 - auc: 0.6519 - loss: 0.6578 - precision: 0.6008 - sensitivity: 0.6660 - val_accuracy: 0.6151 - val_auc: 0.6836 - val_loss: 0.6527 - val_precision: 0.5756 - val_sensitivity: 0.8049
Epoch 19/30
28/31 ————— 0s 18ms/step - accuracy: 0.6179 - auc: 0.6697 - loss: 0.6476 - precision: 0.5992 - sensitivity: 0.7374
Epoch 19: val_auc improved from 0.68362 to 0.68642, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

Epoch 19: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras
31/31 ————— 1s 24ms/step - accuracy: 0.6231 - auc: 0.6621 - loss: 0.6497 - precision: 0.5989 - sensitivity: 0.6952 - val_accuracy: 0.6468 - val_auc: 0.6864 - val_loss: 0.6429 - val_precision: 0.6441 - val_sensitivity: 0.6179
Epoch 20/30
31/31 ————— 0s 18ms/step - accuracy: 0.6174 - auc: 0.6623 - loss: 0.6490 - precision: 0.6044 - sensitivity: 0.5688
Epoch 20: val_auc did not improve from 0.68642
31/31 ————— 1s 22ms/step - accuracy: 0.6282 - auc: 0.6725 - loss: 0.6436 - precision: 0.6176 - sensitivity: 0.6305 - val_accuracy: 0.6270 - val_auc: 0.6833 - val_loss: 0.6570 - val_precision: 0.5801 - val_sensitivity: 0.8537
Epoch 21/30
28/31 ————— 0s 18ms/step - accuracy: 0.6224 - auc: 0.6673 - loss: 0.6507 - precision: 0.5909 - sensitivity: 0.6849
Epoch 21: val_auc improved from 0.68642 to 0.68813, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_scaling_best.keras

aling_best.keras

Epoch 21: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.6323 - auc:
0.6750 - loss: 0.6437 - precision: 0.6151 - sensitivity: 0.6639 -
val_accuracy: 0.6468 - val_auc: 0.6881 - val_loss: 0.6397 -
val_precision: 0.6371 - val_sensitivity: 0.6423

Epoch 22/30

28/31 ————— 0s 18ms/step - accuracy: 0.5990 - auc:
0.6290 - loss: 0.6671 - precision: 0.5763 - sensitivity: 0.5457

Epoch 22: val_auc improved from 0.68813 to 0.69159, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 22: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.6261 - auc:
0.6688 - loss: 0.6461 - precision: 0.6119 - sensitivity: 0.6451 -
val_accuracy: 0.6429 - val_auc: 0.6916 - val_loss: 0.6374 -
val_precision: 0.6260 - val_sensitivity: 0.6667

Epoch 23/30

28/31 ————— 0s 18ms/step - accuracy: 0.6555 - auc:
0.6923 - loss: 0.6327 - precision: 0.6496 - sensitivity: 0.6576

Epoch 23: val_auc did not improve from 0.69159

31/31 ————— 1s 21ms/step - accuracy: 0.6343 - auc:
0.6826 - loss: 0.6376 - precision: 0.6242 - sensitivity: 0.6347 -
val_accuracy: 0.6468 - val_auc: 0.6816 - val_loss: 0.6407 -
val_precision: 0.6466 - val_sensitivity: 0.6098

Epoch 24/30

28/31 ————— 0s 18ms/step - accuracy: 0.6699 - auc:
0.7023 - loss: 0.6303 - precision: 0.6538 - sensitivity: 0.7267

Epoch 24: val_auc did not improve from 0.69159

31/31 ————— 1s 23ms/step - accuracy: 0.6558 - auc:
0.6993 - loss: 0.6277 - precision: 0.6365 - sensitivity: 0.6910 -
val_accuracy: 0.6349 - val_auc: 0.6907 - val_loss: 0.6331 -
val_precision: 0.6148 - val_sensitivity: 0.6748

Epoch 25/30

28/31 ————— 0s 18ms/step - accuracy: 0.6559 - auc:
0.7077 - loss: 0.6193 - precision: 0.6418 - sensitivity: 0.7049

Epoch 25: val_auc did not improve from 0.69159

31/31 ————— 1s 21ms/step - accuracy: 0.6394 - auc:
0.6899 - loss: 0.6298 - precision: 0.6275 - sensitivity: 0.6472 -
val_accuracy: 0.6429 - val_auc: 0.6771 - val_loss: 0.6406 -
val_precision: 0.6387 - val_sensitivity: 0.6179

Epoch 26/30

28/31 ————— 0s 18ms/step - accuracy: 0.6440 - auc:

0.6979 - loss: 0.6261 - precision: 0.6141 - sensitivity: 0.6978
Epoch 26: val_auc improved from 0.69159 to 0.69484, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

Epoch 26: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/baseline_sc
aling_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.6599 - auc:
0.7077 - loss: 0.6218 - precision: 0.6393 - sensitivity: 0.6994 -
val_accuracy: 0.6468 - val_auc: 0.6948 - val_loss: 0.6318 -
val_precision: 0.6349 - val_sensitivity: 0.6504

Epoch 27/30

28/31 ————— 0s 18ms/step - accuracy: 0.6495 - auc:
0.7115 - loss: 0.6143 - precision: 0.6308 - sensitivity: 0.6657

Epoch 27: val_auc did not improve from 0.69484

31/31 ————— 1s 21ms/step - accuracy: 0.6415 - auc:
0.6973 - loss: 0.6253 - precision: 0.6217 - sensitivity: 0.6827 -
val_accuracy: 0.6111 - val_auc: 0.6782 - val_loss: 0.6505 -
val_precision: 0.6712 - val_sensitivity: 0.3984

Epoch 28/30

28/31 ————— 0s 18ms/step - accuracy: 0.6708 - auc:
0.7350 - loss: 0.6119 - precision: 0.7093 - sensitivity: 0.6201

Epoch 28: val_auc did not improve from 0.69484

31/31 ————— 1s 21ms/step - accuracy: 0.6476 - auc:
0.7005 - loss: 0.6257 - precision: 0.6457 - sensitivity: 0.6200 -
val_accuracy: 0.6508 - val_auc: 0.6945 - val_loss: 0.6461 -
val_precision: 0.6012 - val_sensitivity: 0.8455

Epoch 29/30

28/31 ————— 0s 18ms/step - accuracy: 0.6616 - auc:
0.7366 - loss: 0.6099 - precision: 0.6185 - sensitivity: 0.7172

Epoch 29: val_auc did not improve from 0.69484

31/31 ————— 1s 23ms/step - accuracy: 0.6527 - auc:
0.7200 - loss: 0.6135 - precision: 0.6404 - sensitivity: 0.6618 -
val_accuracy: 0.6310 - val_auc: 0.6873 - val_loss: 0.6386 -
val_precision: 0.6027 - val_sensitivity: 0.7154

Epoch 30/30

28/31 ————— 0s 18ms/step - accuracy: 0.6592 - auc:
0.7384 - loss: 0.6006 - precision: 0.6202 - sensitivity: 0.6912

Epoch 30: val_auc did not improve from 0.69484

31/31 ————— 1s 21ms/step - accuracy: 0.6507 - auc:
0.7298 - loss: 0.6054 - precision: 0.6346 - sensitivity: 0.6743 -
val_accuracy: 0.5913 - val_auc: 0.6639 - val_loss: 0.6689 -
val_precision: 0.6724 - val_sensitivity: 0.3171

Restoring model weights from the end of the best epoch: 26.

Best epoch: 26 | Best val_auc: 0.6948

Epoch 1/30

31/31 ————— 0s 87ms/step - accuracy: 0.5000 - auc:

0.4845 - loss: 0.6972 - precision: 0.4686 - sensitivity: 0.5194
Epoch 1: val_auc improved from None to 0.50000, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 10s 148ms/step - accuracy: 0.5026 - auc:
0.4871 - loss: 0.6958 - precision: 0.4877 - sensitivity: 0.3299 -
val_accuracy: 0.5119 - val_auc: 0.5000 - val_loss: 0.6929 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 2/30

31/31 ————— 0s 21ms/step - accuracy: 0.4867 - auc:
0.4857 - loss: 0.6941 - precision: 0.2869 - sensitivity: 0.2073

Epoch 2: val_auc did not improve from 0.50000

31/31 ————— 1s 24ms/step - accuracy: 0.4709 - auc:
0.4633 - loss: 0.6936 - precision: 0.4646 - sensitivity: 0.5344 -
val_accuracy: 0.4881 - val_auc: 0.5000 - val_loss: 0.6932 -
val_precision: 0.4881 - val_sensitivity: 1.0000

Epoch 3/30

31/31 ————— 0s 20ms/step - accuracy: 0.4961 - auc:
0.4551 - loss: 0.6924 - precision: 0.3877 - sensitivity: 0.2059

Epoch 3: val_auc did not improve from 0.50000

31/31 ————— 1s 23ms/step - accuracy: 0.4964 - auc:
0.4751 - loss: 0.6938 - precision: 0.3871 - sensitivity: 0.0501 -
val_accuracy: 0.5119 - val_auc: 0.5000 - val_loss: 0.6930 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 4/30

31/31 ————— 0s 20ms/step - accuracy: 0.4625 - auc:
0.4699 - loss: 0.6943 - precision: 0.3826 - sensitivity: 0.3784

Epoch 4: val_auc did not improve from 0.50000

31/31 ————— 1s 23ms/step - accuracy: 0.4719 - auc:
0.4649 - loss: 0.6935 - precision: 0.4550 - sensitivity: 0.4008 -
val_accuracy: 0.5119 - val_auc: 0.5000 - val_loss: 0.6931 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 5/30

31/31 ————— 0s 20ms/step - accuracy: 0.5185 - auc:
0.5061 - loss: 0.6929 - precision: 0.0000e+00 - sensitivity:
0.0000e+00

Epoch 5: val_auc improved from 0.50000 to 0.52137, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 26ms/step - accuracy: 0.5107 - auc:
0.5050 - loss: 0.6932 - precision: 0.0000e+00 - sensitivity:

0.0000e+00 - val_accuracy: 0.5119 - val_auc: 0.5214 - val_loss: 0.6926
- val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 6/30

31/31 ————— 0s 20ms/step - accuracy: 0.4819 - auc:
0.5109 - loss: 0.6940 - precision: 0.3815 - sensitivity: 0.2990

Epoch 6: val_auc did not improve from 0.52137

31/31 ————— 1s 23ms/step - accuracy: 0.4954 - auc:
0.5231 - loss: 0.6918 - precision: 0.4815 - sensitivity: 0.4071 -
val_accuracy: 0.4683 - val_auc: 0.4985 - val_loss: 0.6920 -
val_precision: 0.3514 - val_sensitivity: 0.1057

Epoch 7/30

28/31 ————— 0s 20ms/step - accuracy: 0.5048 - auc:
0.4961 - loss: 0.6980 - precision: 0.4806 - sensitivity: 0.4700

Epoch 7: val_auc improved from 0.52137 to 0.54966, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 7: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 27ms/step - accuracy: 0.5097 - auc:
0.5209 - loss: 0.6934 - precision: 0.4991 - sensitivity: 0.5762 -
val_accuracy: 0.5278 - val_auc: 0.5497 - val_loss: 0.6887 -
val_precision: 0.5135 - val_sensitivity: 0.6179

Epoch 8/30

31/31 ————— 0s 21ms/step - accuracy: 0.5413 - auc:
0.5472 - loss: 0.6875 - precision: 0.5300 - sensitivity: 0.5746

Epoch 8: val_auc improved from 0.54966 to 0.58332, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 8: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 29ms/step - accuracy: 0.5352 - auc:
0.5435 - loss: 0.6876 - precision: 0.5225 - sensitivity: 0.5825 -
val_accuracy: 0.5000 - val_auc: 0.5833 - val_loss: 0.6932 -
val_precision: 0.4939 - val_sensitivity: 0.9919

Epoch 9/30

31/31 ————— 0s 21ms/step - accuracy: 0.5021 - auc:
0.5215 - loss: 0.6931 - precision: 0.4903 - sensitivity: 0.9374

Epoch 9: val_auc improved from 0.58332 to 0.60163, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 9: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 32ms/step - accuracy: 0.5107 - auc:
0.5351 - loss: 0.6910 - precision: 0.5000 - sensitivity: 0.8163 -

val_accuracy: 0.5635 - val_auc: 0.6016 - val_loss: 0.6858 -
val_precision: 0.5333 - val_sensitivity: 0.8455
Epoch 10/30
31/31 _____ 0s 22ms/step - accuracy: 0.5424 - auc:
0.6030 - loss: 0.6810 - precision: 0.5117 - sensitivity: 0.8686
Epoch 10: val_auc improved from 0.60163 to 0.65750, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 10: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 _____ 1s 30ms/step - accuracy: 0.5342 - auc:
0.5682 - loss: 0.6834 - precision: 0.5153 - sensitivity: 0.8100 -
val_accuracy: 0.5476 - val_auc: 0.6575 - val_loss: 0.6741 -
val_precision: 0.5217 - val_sensitivity: 0.8780

Epoch 11/30
31/31 _____ 0s 21ms/step - accuracy: 0.5446 - auc:
0.6035 - loss: 0.6799 - precision: 0.5284 - sensitivity: 0.8577
Epoch 11: val_auc improved from 0.65750 to 0.66244, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 11: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 _____ 1s 28ms/step - accuracy: 0.5455 - auc:
0.6099 - loss: 0.6802 - precision: 0.5224 - sensitivity: 0.8288 -
val_accuracy: 0.5873 - val_auc: 0.6624 - val_loss: 0.6692 -
val_precision: 0.5597 - val_sensitivity: 0.7236

Epoch 12/30
31/31 _____ 0s 20ms/step - accuracy: 0.6042 - auc:
0.6253 - loss: 0.6691 - precision: 0.5809 - sensitivity: 0.5709
Epoch 12: val_auc did not improve from 0.66244

31/31 _____ 1s 23ms/step - accuracy: 0.5894 - auc:
0.6238 - loss: 0.6681 - precision: 0.5733 - sensitivity: 0.6284 -
val_accuracy: 0.5952 - val_auc: 0.6598 - val_loss: 0.6617 -
val_precision: 0.5644 - val_sensitivity: 0.7480

Epoch 13/30
31/31 _____ 0s 20ms/step - accuracy: 0.5938 - auc:
0.6362 - loss: 0.6665 - precision: 0.5905 - sensitivity: 0.6140
Epoch 13: val_auc improved from 0.66244 to 0.67940, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 13: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 _____ 1s 26ms/step - accuracy: 0.6282 - auc:
0.6649 - loss: 0.6568 - precision: 0.6108 - sensitivity: 0.6618 -

val_accuracy: 0.6270 - val_auc: 0.6794 - val_loss: 0.6705 -
val_precision: 0.7231 - val_sensitivity: 0.3821
Epoch 14/30
31/31 _____ 0s 20ms/step - accuracy: 0.6188 - auc:
0.6674 - loss: 0.6514 - precision: 0.6301 - sensitivity: 0.5570
Epoch 14: val_auc improved from 0.67940 to 0.68163, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 14: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 _____ 1s 26ms/step - accuracy: 0.6200 - auc:
0.6613 - loss: 0.6550 - precision: 0.6213 - sensitivity: 0.5720 -
val_accuracy: 0.6032 - val_auc: 0.6816 - val_loss: 0.6545 -
val_precision: 0.5689 - val_sensitivity: 0.7724

Epoch 15/30
31/31 _____ 0s 20ms/step - accuracy: 0.6151 - auc:
0.6471 - loss: 0.6648 - precision: 0.5846 - sensitivity: 0.5978
Epoch 15: val_auc did not improve from 0.68163

31/31 _____ 1s 23ms/step - accuracy: 0.6251 - auc:
0.6777 - loss: 0.6428 - precision: 0.6124 - sensitivity: 0.6367 -
val_accuracy: 0.6230 - val_auc: 0.6727 - val_loss: 0.6676 -
val_precision: 0.6207 - val_sensitivity: 0.5854

Epoch 16/30
31/31 _____ 0s 20ms/step - accuracy: 0.6431 - auc:
0.7092 - loss: 0.6222 - precision: 0.6400 - sensitivity: 0.6185
Epoch 16: val_auc did not improve from 0.68163

31/31 _____ 1s 24ms/step - accuracy: 0.6394 - auc:
0.6973 - loss: 0.6301 - precision: 0.6270 - sensitivity: 0.6493 -
val_accuracy: 0.6310 - val_auc: 0.6771 - val_loss: 0.6647 -
val_precision: 0.6364 - val_sensitivity: 0.5691

Epoch 17/30
28/31 _____ 0s 20ms/step - accuracy: 0.6428 - auc:
0.7109 - loss: 0.6257 - precision: 0.6396 - sensitivity: 0.6300
Epoch 17: val_auc improved from 0.68163 to 0.68665, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 17: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 _____ 1s 26ms/step - accuracy: 0.6425 - auc:
0.7023 - loss: 0.6265 - precision: 0.6229 - sensitivity: 0.6827 -
val_accuracy: 0.6508 - val_auc: 0.6866 - val_loss: 0.6621 -
val_precision: 0.6036 - val_sensitivity: 0.8293

Epoch 18/30
31/31 _____ 0s 20ms/step - accuracy: 0.6646 - auc:
0.7315 - loss: 0.6104 - precision: 0.6645 - sensitivity: 0.6702
Epoch 18: val_auc did not improve from 0.68665

31/31 ————— 1s 23ms/step - accuracy: 0.6711 - auc:
0.7439 - loss: 0.5963 - precision: 0.6605 - sensitivity: 0.6743 -
val_accuracy: 0.6151 - val_auc: 0.6768 - val_loss: 0.6622 -
val_precision: 0.6327 - val_sensitivity: 0.5041

Epoch 19/30

31/31 ————— 0s 20ms/step - accuracy: 0.6882 - auc:
0.7574 - loss: 0.5860 - precision: 0.6929 - sensitivity: 0.6727

Epoch 19: val_auc improved from 0.68665 to 0.68753, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 19: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 27ms/step - accuracy: 0.6936 - auc:
0.7687 - loss: 0.5719 - precision: 0.6892 - sensitivity: 0.6806 -
val_accuracy: 0.6151 - val_auc: 0.6875 - val_loss: 0.6517 -
val_precision: 0.5985 - val_sensitivity: 0.6423

Epoch 20/30

31/31 ————— 0s 20ms/step - accuracy: 0.6770 - auc:
0.7781 - loss: 0.5577 - precision: 0.6777 - sensitivity: 0.6968

Epoch 20: val_auc did not improve from 0.68753

31/31 ————— 1s 23ms/step - accuracy: 0.6854 - auc:
0.7687 - loss: 0.5720 - precision: 0.6741 - sensitivity: 0.6910 -
val_accuracy: 0.6111 - val_auc: 0.6571 - val_loss: 0.6635 -
val_precision: 0.6437 - val_sensitivity: 0.4553

Epoch 21/30

31/31 ————— 0s 20ms/step - accuracy: 0.7314 - auc:
0.8280 - loss: 0.5231 - precision: 0.7529 - sensitivity: 0.6776

Epoch 21: val_auc improved from 0.68753 to 0.69339, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 21: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

31/31 ————— 1s 26ms/step - accuracy: 0.7150 - auc:
0.8079 - loss: 0.5328 - precision: 0.7174 - sensitivity: 0.6889 -
val_accuracy: 0.6310 - val_auc: 0.6934 - val_loss: 0.6940 -
val_precision: 0.5904 - val_sensitivity: 0.7967

Epoch 22/30

31/31 ————— 0s 20ms/step - accuracy: 0.7370 - auc:
0.8239 - loss: 0.5168 - precision: 0.7319 - sensitivity: 0.7226

Epoch 22: val_auc did not improve from 0.69339

31/31 ————— 1s 23ms/step - accuracy: 0.7395 - auc:
0.8211 - loss: 0.5131 - precision: 0.7353 - sensitivity: 0.7307 -
val_accuracy: 0.6071 - val_auc: 0.6578 - val_loss: 0.7123 -
val_precision: 0.6500 - val_sensitivity: 0.4228

Epoch 23/30

```
31/31 ————— 0s 20ms/step - accuracy: 0.7899 - auc:
0.8797 - loss: 0.4573 - precision: 0.8299 - sensitivity: 0.7286
Epoch 23: val_auc did not improve from 0.69339
31/31 ————— 1s 23ms/step - accuracy: 0.7824 - auc:
0.8568 - loss: 0.4730 - precision: 0.7854 - sensitivity: 0.7641 -
val_accuracy: 0.5794 - val_auc: 0.6795 - val_loss: 0.6991 -
val_precision: 0.5541 - val_sensitivity: 0.7073
Epoch 24/30
31/31 ————— 0s 22ms/step - accuracy: 0.7942 - auc:
0.8859 - loss: 0.4270 - precision: 0.7909 - sensitivity: 0.8078
Epoch 24: val_auc did not improve from 0.69339
31/31 ————— 1s 26ms/step - accuracy: 0.7957 - auc:
0.8807 - loss: 0.4272 - precision: 0.7925 - sensitivity: 0.7891 -
val_accuracy: 0.6071 - val_auc: 0.6577 - val_loss: 0.7758 -
val_precision: 0.6111 - val_sensitivity: 0.5366
Epoch 25/30
31/31 ————— 0s 21ms/step - accuracy: 0.8467 - auc:
0.9159 - loss: 0.3819 - precision: 0.8782 - sensitivity: 0.8091
Epoch 25: val_auc did not improve from 0.69339
31/31 ————— 1s 26ms/step - accuracy: 0.8355 - auc:
0.9009 - loss: 0.4011 - precision: 0.8313 - sensitivity: 0.8330 -
val_accuracy: 0.5913 - val_auc: 0.6424 - val_loss: 0.8462 -
val_precision: 0.6562 - val_sensitivity: 0.3415
Epoch 26/30
31/31 ————— 0s 21ms/step - accuracy: 0.8227 - auc:
0.9326 - loss: 0.3511 - precision: 0.8794 - sensitivity: 0.7623
Epoch 26: val_auc did not improve from 0.69339
31/31 ————— 1s 26ms/step - accuracy: 0.8325 - auc:
0.9240 - loss: 0.3533 - precision: 0.8508 - sensitivity: 0.7975 -
val_accuracy: 0.6032 - val_auc: 0.6657 - val_loss: 0.8013 -
val_precision: 0.5983 - val_sensitivity: 0.5691
Epoch 27/30
29/31 ————— 0s 23ms/step - accuracy: 0.8637 - auc:
0.9472 - loss: 0.3038 - precision: 0.8661 - sensitivity: 0.8594
Epoch 27: val_auc improved from 0.69339 to 0.69714, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras

Epoch 27: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/scaled_cnn_
best.keras
31/31 ————— 1s 32ms/step - accuracy: 0.8641 - auc:
0.9465 - loss: 0.3013 - precision: 0.8745 - sensitivity: 0.8434 -
val_accuracy: 0.6111 - val_auc: 0.6971 - val_loss: 0.8038 -
val_precision: 0.5862 - val_sensitivity: 0.6911
Epoch 28/30
31/31 ————— 0s 20ms/step - accuracy: 0.8749 - auc:
0.9574 - loss: 0.2713 - precision: 0.8601 - sensitivity: 0.8808
Epoch 28: val_auc did not improve from 0.69714
```

31/31 _____ 1s 23ms/step - accuracy: 0.8693 - auc: 0.9497 - loss: 0.2875 - precision: 0.8679 - sensitivity: 0.8643 - val_accuracy: 0.6032 - val_auc: 0.6554 - val_loss: 0.9485 - val_precision: 0.6139 - val_sensitivity: 0.5041

Epoch 29/30

31/31 _____ 0s 20ms/step - accuracy: 0.9020 - auc: 0.9686 - loss: 0.2387 - precision: 0.9117 - sensitivity: 0.8874

Epoch 29: val_auc did not improve from 0.69714

31/31 _____ 1s 26ms/step - accuracy: 0.8927 - auc: 0.9627 - loss: 0.2517 - precision: 0.8945 - sensitivity: 0.8852 - val_accuracy: 0.6151 - val_auc: 0.6708 - val_loss: 1.0928 - val_precision: 0.5890 - val_sensitivity: 0.6992

Epoch 30/30

31/31 _____ 0s 20ms/step - accuracy: 0.9176 - auc: 0.9775 - loss: 0.2054 - precision: 0.9256 - sensitivity: 0.8996

Epoch 30: val_auc did not improve from 0.69714

31/31 _____ 1s 24ms/step - accuracy: 0.9070 - auc: 0.9730 - loss: 0.2151 - precision: 0.9163 - sensitivity: 0.8914 - val_accuracy: 0.5794 - val_auc: 0.6387 - val_loss: 1.1645 - val_precision: 0.6104 - val_sensitivity: 0.3821

Restoring model weights from the end of the best epoch: 27.

Best epoch: 27 | Best val_auc: 0.6971

Optimal threshold (Youden's J): 0.550 (val sensitivity=0.561, val specificity=0.760)

Optimal threshold (Youden's J): 0.795 (val sensitivity=0.504, val specificity=0.814)

=====

Baseline CNN (128px) – Validation Results (threshold=0.550)

=====

	precision	recall	f1-score	support
Benign	0.64	0.76	0.70	129
Malignant	0.69	0.56	0.62	123
accuracy			0.66	252
macro avg	0.67	0.66	0.66	252
weighted avg	0.67	0.66	0.66	252

Accuracy : 0.6627

Sensitivity : 0.561 (missed 54 of 123 malignant cases)

Specificity : 0.7597

Precision : 0.69

AUC : 0.6941

=====

Scaled CNN (128px) – Validation Results (threshold=0.795)

=====

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Benign	0.63	0.81	0.71	129
Malignant	0.72	0.50	0.59	123

accuracy			0.66	252
macro avg	0.68	0.66	0.65	252
weighted avg	0.68	0.66	0.65	252

Accuracy : 0.6627
 Sensitivity : 0.5041 (missed 61 of 123 malignant cases)
 Specificity : 0.814
 Precision : 0.7209
 AUC : 0.6974

NETWORK SCALING – VALIDATION COMPARISON

Baseline CNN (128px)
 Threshold : 0.5500
 Accuracy : 0.6627
 Sensitivity : 0.5610
 Specificity : 0.7597
 Precision : 0.6900
 AUC : 0.6941

Scaled CNN (128px)
 Threshold : 0.7952
 Accuracy : 0.6627
 Sensitivity : 0.5041
 Specificity : 0.8140
 Precision : 0.7209
 AUC : 0.6974

Validation AUC change: 0.6941 → 0.6974 (+0.0033)

Section 13: Regularisation Ablation Experiment

Regularisation is investigated using the 128×128 baseline architecture. Four configurations are compared: **baseline**, **dropout only**, **augmentation only**, and **combined dropout + augmentation**.

The same data split and evaluation procedure are retained to isolate the effects of the regularisation strategies.

Regularisation ablation: dropout only vs augmentation only

```

def build_ablation_cnn(dropout=False, name="CNN"):
    model = tf.keras.Sequential(

```

```

        [
            layers.Input((128, 128, 3)),
            layers.Conv2D(32, 3, activation="relu"),
            layers.MaxPooling2D(),
            layers.Conv2D(64, 3, activation="relu"),
            layers.MaxPooling2D(),
            layers.Conv2D(128, 3, activation="relu"),
            layers.MaxPooling2D(),
            layers.GlobalAveragePooling2D(),
            layers.Dense(128, activation="relu"),
        ],
        name=name,
    )

    if dropout:
        model.add(layers.Dropout(0.5))

    model.add(layers.Dense(1, activation="sigmoid"))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )

    return model

```

Section 13.1: Dropout-Only CNN

```

# Experiment 1: dropout only

dropout_model = build_ablation_cnn(dropout=True,
name="CNN_Dropout_Only")

dropout_history, dropout_best_epoch = train_model(
    dropout_model,
    train_ds_128_plain, # NO augmentation
    val_ds_128,
    checkpoint_path=f"{MODELS_DIR}/cnn_dropout_only_best.keras",
    class_weight=class_weights,
)

# Dropout-only CNN validation results

# Load saved model

```

```

dropout_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/cnn_dropout_only_best.keras", compile=False
)

# Find threshold using VALIDATION only
dropout_threshold = find_best_threshold(dropout_model, val_ds_128)

# Evaluate on VALIDATION only
dropout_results = evaluate_model(
    dropout_model,
    val_ds_128,
    name="CNN - Dropout Only",
    threshold=dropout_threshold,
    split_name="Validation",
)

print("Dropout-only validation evaluation complete.")

Optimal threshold (Youden's J): 0.525 (val sensitivity=0.528, val
specificity=0.775)

=====
CNN – Dropout Only – Validation Results (threshold=0.525)
=====

```

	precision	recall	f1-score	support
Benign	0.63	0.78	0.70	129
Malignant	0.69	0.53	0.60	123
accuracy			0.65	252
macro avg	0.66	0.65	0.65	252
weighted avg	0.66	0.65	0.65	252

```

Accuracy      : 0.6548
Sensitivity   : 0.5285 (missed 58 of 123 malignant cases)
Specificity   : 0.7752
Precision     : 0.6915
AUC           : 0.6818
Dropout-only validation evaluation complete.

```

Section 13.2: Augmentation-Only CNN

```

# Experiment 2: augmentation only

augmentation_model = build_ablation_cnn(dropout=False,
name="CNN_Augmentation_Only")

augmentation_history, augmentation_best_epoch = train_model(
    augmentation_model,
    train_ds_128_aug, # augmentation ON

```

```

    val_ds_128,
    checkpoint_path=f"{MODELS_DIR}/cnn_augmentation_only_best.keras",
    class_weight=class_weights,
)

# Augmentation-only CNN validation results

augmentation_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/cnn_augmentation_only_best.keras", compile=False
)

augmentation_threshold = find_best_threshold(augmentation_model,
val_ds_128)

augmentation_results = evaluate_model(
    augmentation_model,
    val_ds_128,
    name="CNN - Augmentation Only",
    threshold=augmentation_threshold,
    split_name="Validation",
)

print("Augmentation-only validation evaluation complete.")

Optimal threshold (Youden's J): 0.501 (val sensitivity=0.667, val
specificity=0.636)

=====
CNN - Augmentation Only - Validation Results (threshold=0.501)
=====

```

	precision	recall	f1-score	support
Benign	0.67	0.64	0.65	129
Malignant	0.64	0.67	0.65	123
accuracy			0.65	252
macro avg	0.65	0.65	0.65	252
weighted avg	0.65	0.65	0.65	252

```

Accuracy      : 0.6508
Sensitivity   : 0.6667 (missed 41 of 123 malignant cases)
Specificity   : 0.6357
Precision     : 0.6357
AUC           : 0.6812
Augmentation-only validation evaluation complete.

```

Section 13.3: Combined Dropout + Augmentation CNN

```

def build_regularised_cnn(img_size, name="Regularised_CNN"):
    model = tf.keras.Sequential(

```

```

        [
            layers.Conv2D(
                32, (3, 3), activation="relu", input_shape=(img_size,
img_size, 3)
            ),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(64, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(128, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.GlobalAveragePooling2D(),
            layers.Dense(128, activation="relu"),
            layers.Dropout(0.5), # regularisation added here vs.
baseline
            layers.Dense(1, activation="sigmoid"),
        ],
        name=name,
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )
    return model

```

```
build_regularised_cnn(IMG_SIZE_128).summary()
```

```
Model: "Regularised_CNN"
```

Layer (type)	Output Shape
Param #	
conv2d_3 (Conv2D)	(None, 126, 126, 32)
896	
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)
0	

conv2d_4 (Conv2D)	(None, 61, 61, 64)	
18,496		
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	
0		
conv2d_5 (Conv2D)	(None, 28, 28, 128)	
73,856		
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	
0		
global_average_pooling2d_1	(None, 128)	
0		
(GlobalAveragePooling2D)		
dense_2 (Dense)	(None, 128)	
16,512		
dropout (Dropout)	(None, 128)	
0		
dense_3 (Dense)	(None, 1)	
129		

Total params: 109,889 (429.25 KB)

Trainable params: 109,889 (429.25 KB)

Non-trainable params: 0 (0.00 B)

```
regl28_model = build_regularised_cnn(IMG_SIZE_128,
name="Regularised_CNN_128")
```

Regularised CNN uses dynamic augmentation

```
regl28_history, regl28_best_epoch = train_model(
    regl28_model,
    train_ds_128_aug,
    val_ds_128,
    checkpoint_path=f"{MODELS_DIR}/regularised_cnn_128_best.keras",
```

```

    class_weight=class_weights,
)

Epoch 1/30
31/31 _____ 0s 95ms/step - accuracy: 0.4967 - auc:
0.4542 - loss: 0.6958 - precision: 0.4480 - sensitivity: 0.2303
Epoch 1: val_auc improved from None to 0.49319, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 10s 154ms/step - accuracy: 0.4770 - auc:
0.4644 - loss: 0.6953 - precision: 0.4510 - sensitivity: 0.3173 -
val_accuracy: 0.5119 - val_auc: 0.4932 - val_loss: 0.6929 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00
Epoch 2/30
30/31 _____ 0s 19ms/step - accuracy: 0.4904 - auc:
0.5091 - loss: 0.6931 - precision: 0.4886 - sensitivity: 0.4251
Epoch 2: val_auc improved from 0.49319 to 0.50000, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 2: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 27ms/step - accuracy: 0.4842 - auc:
0.4994 - loss: 0.6933 - precision: 0.4748 - sensitivity: 0.5115 -
val_accuracy: 0.5238 - val_auc: 0.5000 - val_loss: 0.6930 -
val_precision: 0.5076 - val_sensitivity: 0.8130
Epoch 3/30
29/31 _____ 0s 18ms/step - accuracy: 0.4776 - auc:
0.4773 - loss: 0.6934 - precision: 0.4680 - sensitivity: 0.4793
Epoch 3: val_auc improved from 0.50000 to 0.53154, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 3: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 23ms/step - accuracy: 0.5036 - auc:
0.4967 - loss: 0.6933 - precision: 0.4916 - sensitivity: 0.4280 -
val_accuracy: 0.4881 - val_auc: 0.5315 - val_loss: 0.6925 -
val_precision: 0.4423 - val_sensitivity: 0.1870
Epoch 4/30
30/31 _____ 0s 17ms/step - accuracy: 0.4748 - auc:
0.4807 - loss: 0.6936 - precision: 0.4587 - sensitivity: 0.3192
Epoch 4: val_auc improved from 0.53154 to 0.55924, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised

```

_cnn_128_best.keras

Epoch 4: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 22ms/step - accuracy: 0.4872 - auc:
0.5054 - loss: 0.6931 - precision: 0.4657 - sensitivity: 0.3257 -
val_accuracy: 0.5119 - val_auc: 0.5592 - val_loss: 0.6915 -
val_precision: 0.0000e+00 - val_sensitivity: 0.0000e+00

Epoch 5/30

29/31 ————— 0s 18ms/step - accuracy: 0.5237 - auc:
0.5349 - loss: 0.6917 - precision: 0.5684 - sensitivity: 0.1811

Epoch 5: val_auc did not improve from 0.55924

31/31 ————— 1s 21ms/step - accuracy: 0.5179 - auc:
0.5106 - loss: 0.6929 - precision: 0.5092 - sensitivity: 0.4029 -
val_accuracy: 0.5357 - val_auc: 0.5412 - val_loss: 0.6918 -
val_precision: 0.5227 - val_sensitivity: 0.5610

Epoch 6/30

28/31 ————— 0s 17ms/step - accuracy: 0.5211 - auc:
0.5435 - loss: 0.6918 - precision: 0.5156 - sensitivity: 0.4901

Epoch 6: val_auc improved from 0.55924 to 0.60698, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.5179 - auc:
0.5160 - loss: 0.6925 - precision: 0.5060 - sensitivity: 0.6180 -
val_accuracy: 0.5317 - val_auc: 0.6070 - val_loss: 0.6890 -
val_precision: 0.8571 - val_sensitivity: 0.0488

Epoch 7/30

31/31 ————— 0s 17ms/step - accuracy: 0.5483 - auc:
0.5357 - loss: 0.6899 - precision: 0.5422 - sensitivity: 0.0986

Epoch 7: val_auc improved from 0.60698 to 0.63512, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.5403 - auc:
0.5521 - loss: 0.6917 - precision: 0.5721 - sensitivity: 0.2401 -
val_accuracy: 0.5317 - val_auc: 0.6351 - val_loss: 0.6899 -
val_precision: 0.5113 - val_sensitivity: 0.9187

Epoch 8/30

28/31 ————— 0s 19ms/step - accuracy: 0.5202 - auc:
0.5287 - loss: 0.6917 - precision: 0.4947 - sensitivity: 0.7969

Epoch 8: val_auc did not improve from 0.63512

31/31 ————— 1s 23ms/step - accuracy: 0.5557 - auc:

0.5508 - loss: 0.6885 - precision: 0.5382 - sensitivity: 0.6472 -
val_accuracy: 0.5556 - val_auc: 0.6348 - val_loss: 0.6831 -
val_precision: 0.5369 - val_sensitivity: 0.6504

Epoch 9/30

30/31 ————— 0s 19ms/step - accuracy: 0.5516 - auc:
0.5749 - loss: 0.6888 - precision: 0.5215 - sensitivity: 0.6671

Epoch 9: val_auc improved from 0.63512 to 0.65312, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 9: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 26ms/step - accuracy: 0.5516 - auc:
0.5757 - loss: 0.6878 - precision: 0.5303 - sensitivity: 0.7307 -
val_accuracy: 0.5833 - val_auc: 0.6531 - val_loss: 0.6776 -
val_precision: 0.5682 - val_sensitivity: 0.6098

Epoch 10/30

30/31 ————— 0s 20ms/step - accuracy: 0.5513 - auc:
0.5691 - loss: 0.6889 - precision: 0.5507 - sensitivity: 0.5749

Epoch 10: val_auc did not improve from 0.65312

31/31 ————— 1s 24ms/step - accuracy: 0.5526 - auc:
0.5800 - loss: 0.6859 - precision: 0.5362 - sensitivity: 0.6347 -
val_accuracy: 0.5595 - val_auc: 0.6335 - val_loss: 0.6773 -
val_precision: 0.5366 - val_sensitivity: 0.7154

Epoch 11/30

29/31 ————— 0s 20ms/step - accuracy: 0.5556 - auc:
0.5732 - loss: 0.6842 - precision: 0.5184 - sensitivity: 0.4355

Epoch 11: val_auc did not improve from 0.65312

31/31 ————— 1s 24ms/step - accuracy: 0.5628 - auc:
0.5839 - loss: 0.6837 - precision: 0.5560 - sensitivity: 0.5282 -
val_accuracy: 0.5516 - val_auc: 0.6442 - val_loss: 0.6816 -
val_precision: 0.5253 - val_sensitivity: 0.8455

Epoch 12/30

28/31 ————— 0s 18ms/step - accuracy: 0.5718 - auc:
0.6353 - loss: 0.6759 - precision: 0.5314 - sensitivity: 0.7782

Epoch 12: val_auc improved from 0.65312 to 0.66266, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 12: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 23ms/step - accuracy: 0.5792 - auc:
0.6043 - loss: 0.6801 - precision: 0.5552 - sensitivity: 0.7035 -
val_accuracy: 0.6270 - val_auc: 0.6627 - val_loss: 0.6649 -
val_precision: 0.5924 - val_sensitivity: 0.7561

Epoch 13/30

29/31 ————— 0s 18ms/step - accuracy: 0.5717 - auc:

```

0.5956 - loss: 0.6803 - precision: 0.5531 - sensitivity: 0.7342
Epoch 13: val_auc did not improve from 0.66266
31/31 _____ 1s 21ms/step - accuracy: 0.5557 - auc:
0.5857 - loss: 0.6829 - precision: 0.5367 - sensitivity: 0.6722 -
val_accuracy: 0.5675 - val_auc: 0.6495 - val_loss: 0.6718 -
val_precision: 0.5432 - val_sensitivity: 0.7154
Epoch 14/30
30/31 _____ 0s 17ms/step - accuracy: 0.5846 - auc:
0.6273 - loss: 0.6763 - precision: 0.5645 - sensitivity: 0.7265
Epoch 14: val_auc did not improve from 0.66266
31/31 _____ 1s 20ms/step - accuracy: 0.5802 - auc:
0.6144 - loss: 0.6775 - precision: 0.5609 - sensitivity: 0.6534 -
val_accuracy: 0.6389 - val_auc: 0.6570 - val_loss: 0.6622 -
val_precision: 0.6270 - val_sensitivity: 0.6423
Epoch 15/30
31/31 _____ 0s 17ms/step - accuracy: 0.5816 - auc:
0.6095 - loss: 0.6772 - precision: 0.5768 - sensitivity: 0.6273
Epoch 15: val_auc did not improve from 0.66266
31/31 _____ 1s 21ms/step - accuracy: 0.5751 - auc:
0.6055 - loss: 0.6784 - precision: 0.5576 - sensitivity: 0.6367 -
val_accuracy: 0.6468 - val_auc: 0.6607 - val_loss: 0.6641 -
val_precision: 0.6441 - val_sensitivity: 0.6179
Epoch 16/30
28/31 _____ 0s 18ms/step - accuracy: 0.6040 - auc:
0.6366 - loss: 0.6665 - precision: 0.5834 - sensitivity: 0.6556
Epoch 16: val_auc improved from 0.66266 to 0.66865, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 16: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 23ms/step - accuracy: 0.5955 - auc:
0.6191 - loss: 0.6740 - precision: 0.5835 - sensitivity: 0.6054 -
val_accuracy: 0.5992 - val_auc: 0.6687 - val_loss: 0.6673 -
val_precision: 0.5625 - val_sensitivity: 0.8049
Epoch 17/30
28/31 _____ 0s 17ms/step - accuracy: 0.6188 - auc:
0.6553 - loss: 0.6655 - precision: 0.5855 - sensitivity: 0.7007
Epoch 17: val_auc improved from 0.66865 to 0.66928, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 17: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 22ms/step - accuracy: 0.5935 - auc:
0.6118 - loss: 0.6774 - precision: 0.5860 - sensitivity: 0.5762 -
val_accuracy: 0.6706 - val_auc: 0.6693 - val_loss: 0.6601 -

```

```

val_precision: 0.6724 - val_sensitivity: 0.6341
Epoch 18/30
28/31 _____ 0s 17ms/step - accuracy: 0.6010 - auc:
0.6308 - loss: 0.6691 - precision: 0.5969 - sensitivity: 0.6618
Epoch 18: val_auc did not improve from 0.66928
31/31 _____ 1s 20ms/step - accuracy: 0.5771 - auc:
0.6120 - loss: 0.6746 - precision: 0.5588 - sensitivity: 0.6451 -
val_accuracy: 0.6270 - val_auc: 0.6335 - val_loss: 0.6729 -
val_precision: 0.6559 - val_sensitivity: 0.4959
Epoch 19/30
28/31 _____ 0s 18ms/step - accuracy: 0.5916 - auc:
0.6127 - loss: 0.6786 - precision: 0.6170 - sensitivity: 0.5741
Epoch 19: val_auc improved from 0.66928 to 0.67228, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 19: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 23ms/step - accuracy: 0.5965 - auc:
0.6118 - loss: 0.6769 - precision: 0.5827 - sensitivity: 0.6180 -
val_accuracy: 0.6746 - val_auc: 0.6723 - val_loss: 0.6602 -
val_precision: 0.6847 - val_sensitivity: 0.6179
Epoch 20/30
30/31 _____ 0s 18ms/step - accuracy: 0.6083 - auc:
0.6421 - loss: 0.6675 - precision: 0.6109 - sensitivity: 0.6485
Epoch 20: val_auc did not improve from 0.67228
31/31 _____ 1s 21ms/step - accuracy: 0.5965 - auc:
0.6201 - loss: 0.6740 - precision: 0.5875 - sensitivity: 0.5887 -
val_accuracy: 0.6310 - val_auc: 0.6520 - val_loss: 0.6664 -
val_precision: 0.6531 - val_sensitivity: 0.5203
Epoch 21/30
30/31 _____ 0s 17ms/step - accuracy: 0.5980 - auc:
0.6196 - loss: 0.6700 - precision: 0.5690 - sensitivity: 0.5902
Epoch 21: val_auc improved from 0.67228 to 0.68876, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 21: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
31/31 _____ 1s 23ms/step - accuracy: 0.5843 - auc:
0.6139 - loss: 0.6734 - precision: 0.5634 - sensitivity: 0.6681 -
val_accuracy: 0.6389 - val_auc: 0.6888 - val_loss: 0.6541 -
val_precision: 0.6026 - val_sensitivity: 0.7642
Epoch 22/30
28/31 _____ 0s 18ms/step - accuracy: 0.5573 - auc:
0.5847 - loss: 0.6838 - precision: 0.5474 - sensitivity: 0.5818
Epoch 22: val_auc did not improve from 0.68876

```

31/31 ————— 1s 21ms/step - accuracy: 0.5904 - auc:
0.6209 - loss: 0.6719 - precision: 0.5725 - sensitivity: 0.6430 -
val_accuracy: 0.6548 - val_auc: 0.6793 - val_loss: 0.6499 -
val_precision: 0.6667 - val_sensitivity: 0.5854

Epoch 23/30

28/31 ————— 0s 18ms/step - accuracy: 0.6126 - auc:
0.6345 - loss: 0.6679 - precision: 0.6236 - sensitivity: 0.5528

Epoch 23: val_auc did not improve from 0.68876

31/31 ————— 1s 21ms/step - accuracy: 0.6016 - auc:
0.6183 - loss: 0.6713 - precision: 0.5937 - sensitivity: 0.5887 -
val_accuracy: 0.6587 - val_auc: 0.6873 - val_loss: 0.6484 -
val_precision: 0.6637 - val_sensitivity: 0.6098

Epoch 24/30

28/31 ————— 0s 17ms/step - accuracy: 0.5849 - auc:
0.6356 - loss: 0.6613 - precision: 0.5617 - sensitivity: 0.5882

Epoch 24: val_auc improved from 0.68876 to 0.70278, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

Epoch 24: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras

31/31 ————— 1s 24ms/step - accuracy: 0.5863 - auc:
0.6233 - loss: 0.6694 - precision: 0.5725 - sensitivity: 0.6096 -
val_accuracy: 0.6310 - val_auc: 0.7028 - val_loss: 0.6454 -
val_precision: 0.5949 - val_sensitivity: 0.7642

Epoch 25/30

30/31 ————— 0s 20ms/step - accuracy: 0.6187 - auc:
0.6649 - loss: 0.6575 - precision: 0.6049 - sensitivity: 0.6960

Epoch 25: val_auc did not improve from 0.70278

31/31 ————— 1s 23ms/step - accuracy: 0.6139 - auc:
0.6491 - loss: 0.6623 - precision: 0.6081 - sensitivity: 0.5929 -
val_accuracy: 0.6389 - val_auc: 0.6937 - val_loss: 0.6445 -
val_precision: 0.6290 - val_sensitivity: 0.6341

Epoch 26/30

28/31 ————— 0s 19ms/step - accuracy: 0.5819 - auc:
0.6080 - loss: 0.6772 - precision: 0.5808 - sensitivity: 0.6813

Epoch 26: val_auc did not improve from 0.70278

31/31 ————— 1s 23ms/step - accuracy: 0.6006 - auc:
0.6270 - loss: 0.6690 - precision: 0.5840 - sensitivity: 0.6388 -
val_accuracy: 0.6587 - val_auc: 0.6847 - val_loss: 0.6462 -
val_precision: 0.6581 - val_sensitivity: 0.6260

Epoch 27/30

30/31 ————— 0s 20ms/step - accuracy: 0.5984 - auc:
0.6263 - loss: 0.6715 - precision: 0.5711 - sensitivity: 0.5972

Epoch 27: val_auc did not improve from 0.70278

31/31 ————— 1s 25ms/step - accuracy: 0.6027 - auc:
0.6344 - loss: 0.6678 - precision: 0.5907 - sensitivity: 0.6117 -
val_accuracy: 0.6706 - val_auc: 0.6860 - val_loss: 0.6479 -

```
val_precision: 0.7041 - val_sensitivity: 0.5610
Epoch 28/30
29/31 _____ 0s 20ms/step - accuracy: 0.6251 - auc:
0.6547 - loss: 0.6554 - precision: 0.6209 - sensitivity: 0.5743
Epoch 28: val_auc improved from 0.70278 to 0.70297, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
```

```
Epoch 28: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/regularised
_cnn_128_best.keras
```

```
31/31 _____ 1s 29ms/step - accuracy: 0.6129 - auc:
0.6408 - loss: 0.6604 - precision: 0.6055 - sensitivity: 0.5992 -
val_accuracy: 0.6548 - val_auc: 0.7030 - val_loss: 0.6407 -
val_precision: 0.6268 - val_sensitivity: 0.7236
```

```
Epoch 29/30
```

```
28/31 _____ 0s 18ms/step - accuracy: 0.5977 - auc:
0.6403 - loss: 0.6611 - precision: 0.5733 - sensitivity: 0.6924
```

```
Epoch 29: val_auc did not improve from 0.70297
```

```
31/31 _____ 1s 21ms/step - accuracy: 0.6088 - auc:
0.6362 - loss: 0.6659 - precision: 0.5886 - sensitivity: 0.6660 -
val_accuracy: 0.6627 - val_auc: 0.7028 - val_loss: 0.6414 -
val_precision: 0.6319 - val_sensitivity: 0.7398
```

```
Epoch 30/30
```

```
28/31 _____ 0s 18ms/step - accuracy: 0.6057 - auc:
0.6557 - loss: 0.6582 - precision: 0.5660 - sensitivity: 0.6437
```

```
Epoch 30: val_auc did not improve from 0.70297
```

```
31/31 _____ 1s 21ms/step - accuracy: 0.5986 - auc:
0.6388 - loss: 0.6631 - precision: 0.5896 - sensitivity: 0.5908 -
val_accuracy: 0.6587 - val_auc: 0.6943 - val_loss: 0.6424 -
val_precision: 0.6762 - val_sensitivity: 0.5772
```

```
Restoring model weights from the end of the best epoch: 28.
```

```
Best epoch: 28 | Best val_auc: 0.7030
```

```
# Regularised CNN (128px) validation results
```

```
regl28_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/regularised_cnn_128_best.keras", compile=False
)
```

```
regl28_threshold = find_best_threshold(regl28_model, val_ds_128)
```

```
regl28_results = evaluate_model(
    regl28_model,
    val_ds_128,
    name="Regularised CNN (128px)",
    threshold=regl28_threshold,
    split_name="Validation",
)
```

```

plot_confusion_matrix(
    reg128_results["cm"],
    "Regularised CNN @128x128 - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_reg128_validation.png",
)

```

```

print("Regularised CNN validation evaluation complete.")

```

Optimal threshold (Youden's J): 0.538 (val sensitivity=0.610, val specificity=0.760)

```

=====
Regularised CNN (128px) – Validation Results (threshold=0.538)
=====

```

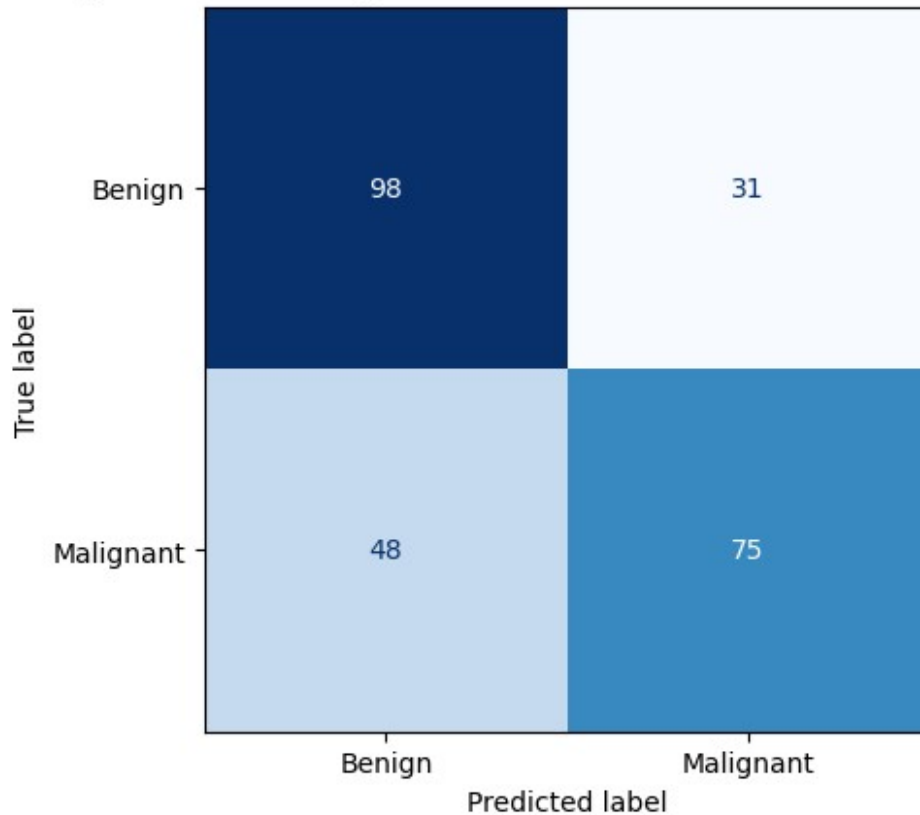
	precision	recall	f1-score	support
Benign	0.67	0.76	0.71	129
Malignant	0.71	0.61	0.66	123
accuracy			0.69	252
macro avg	0.69	0.68	0.68	252
weighted avg	0.69	0.69	0.68	252

```

Accuracy      : 0.6865
Sensitivity    : 0.6098 (missed 48 of 123 malignant cases)
Specificity    : 0.7597
Precision     : 0.7075
AUC           : 0.7032

```

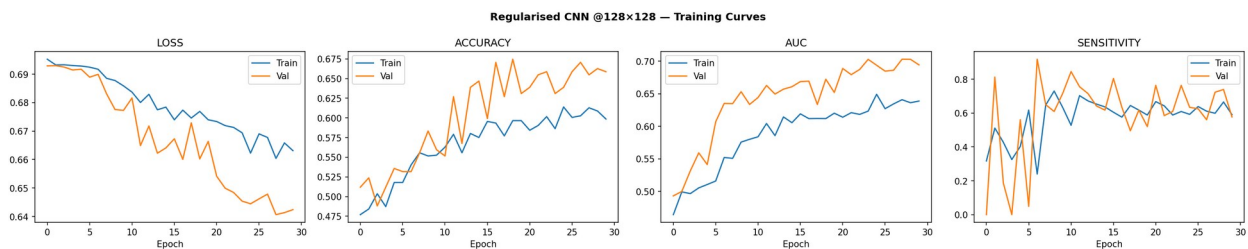
Regularised CNN @128×128 — Validation Confusion Matrix



Regularised CNN validation evaluation complete.

Original training curves

```
from IPython.display import Image, display  
display(Image(filename=f"{BASE}/curves_reg128.png"))
```



Section 13.4: Regularisation Ablation Comparison

The four 128×128 configurations are compared using validation performance. This comparison shows the individual and combined effects of dropout and data augmentation across the evaluation metrics.

```

# Validation ablation comparison

import os
import numpy as np
import pandas as pd
import tensorflow as tf

from sklearn.metrics import (
    roc_auc_score,
    roc_curve,
    accuracy_score,
    recall_score,
    precision_score,
    confusion_matrix,
)

# Best saved model checkpoints

model_paths = {
    "Baseline": os.path.join(MODELS_DIR, "baseline_cnn_best.keras"),
    "Dropout Only": os.path.join(MODELS_DIR,
    "cnn_dropout_only_best.keras"),
    "Augmentation Only": os.path.join(MODELS_DIR,
    "cnn_augmentation_only_best.keras"),
    "Dropout + Augmentation": os.path.join(
        MODELS_DIR, "regularised_cnn_128_best.keras"
    ),
}

# Load saved models

models = {
    name: tf.keras.models.load_model(path, compile=False)
    for name, path in model_paths.items()
}

print("All ablation models loaded.")

# Validation labels

y_val = np.concatenate([y.numpy().reshape(-1) for _, y in
val_ds_128]).astype(int)

# Validation evaluation

results = []

for name, model in models.items():

```

```

y_prob = model.predict(val_ds_128, verbose=0).reshape(-1)

# Threshold selected using VALIDATION only
fpr, tpr, thresholds = roc_curve(y_val, y_prob)

j_scores = tpr - fpr

best_idx = np.argmax(j_scores)

threshold = float(thresholds[best_idx])

y_pred = (y_prob >= threshold).astype(int)

tn, fp, fn, tp = confusion_matrix(y_val, y_pred).ravel()

results.append(
    {
        "Configuration": name,
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_val, y_pred),
        "Sensitivity": recall_score(y_val, y_pred),
        "Specificity": (tn / (tn + fp)),
        "Precision": precision_score(y_val, y_pred,
zero_division=0),
        "AUC": roc_auc_score(y_val, y_prob),
    }
)

```

Validation results table

```
validation_results = pd.DataFrame(results)
```

```
print("\nVALIDATION ABLATION COMPARISON")
```

```
print("=" * 60)
```

```
display(validation_results.round(4))
```

All ablation models loaded.

WARNING:tensorflow:5 out of the last 17 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bb5e7181300> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and

https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:5 out of the last 17 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7bb5e7180b80> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

VALIDATION ABLATION COMPARISON

```
=====
{"summary": "{\n  \"name\": \"\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"Configuration\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Dropout Only\",\n          \"Dropout + Augmentation\",\n          \"Baseline\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Threshold\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.016157634934193397,\n        \"min\": 0.501,\n        \"max\": 0.5384,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          0.5249,\n          0.5384,\n          0.5308\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.01602560451277889,\n        \"min\": 0.6508,\n        \"max\": 0.6865,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          0.6548,\n          0.6865,\n          0.6667\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Sensitivity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.058985273585870565,\n        \"min\": 0.5285,\n        \"max\": 0.6667,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          0.5285,\n          0.6098,\n          0.6341\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Specificity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.06375130717614083,\n        \"min\": 0.6357,\n        \"max\": 0.7752,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          0.7752,\n          0.7597,\n          0.6977\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Precision\",\n
```

```

{"properties": {"dtype": "number", "std": 0.0313123511307173, "min": 0.6357, "max": 0.7075, "num_unique_values": 4, "samples": [0.6915, 0.7075, 0.6667]}, "semantic_type": "", "description": "", "column": "AUC", "properties": {"dtype": "number", "std": 0.01054134716248357, "min": 0.6812, "max": 0.7032, "num_unique_values": 4, "samples": [0.6818, 0.7032, 0.6938]}, "semantic_type": "", "description": ""}, {"type": "dataframe"}

```

Section 14: Input Resolution Experiment - 128×128 vs 224×224

The regularised/scaled custom CNN is evaluated at both **128×128** and **224×224** resolution using the same modelling strategy. This experiment examines whether retaining greater image detail through increased input resolution improves validation performance.

The 224×224 configuration did not outperform the 128×128 model, indicating that increased resolution alone was not beneficial in this experiment.

Regularised/scaled CNN architecture (32-64-128-256 filters, dense 256, dropout 0.5)

```

def build_regularised_scaled_cnn(img_size,
name="Regularised_Scaled_CNN"):

    model = tf.keras.Sequential(
        [
            layers.Input(shape=(img_size, img_size, 3)),
            layers.Conv2D(32, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(64, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.Conv2D(128, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            # Added scaled-up convolutional block
            layers.Conv2D(256, (3, 3), activation="relu"),
            layers.MaxPooling2D((2, 2)),
            layers.GlobalAveragePooling2D(),
            # Increased dense representation
            layers.Dense(256, activation="relu"),
            # Regularisation
            layers.Dropout(0.5),
            layers.Dense(1, activation="sigmoid"),

```

```

    ],
    name=name,
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        tf.keras.metrics.AUC(name="auc"),
        tf.keras.metrics.Recall(name="sensitivity"),
        tf.keras.metrics.Precision(name="precision"),
    ],
)

return model

```

```
build_regularised_scaled_cnn(IMG_SIZE_128).summary()
```

Model: "Regularised_Scaled_CNN"

Layer (type)	Output Shape	
Param #		
conv2d_3 (Conv2D)	(None, 126, 126, 32)	
896		
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	
0		
conv2d_4 (Conv2D)	(None, 61, 61, 64)	
18,496		
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	
0		
conv2d_5 (Conv2D)	(None, 28, 28, 128)	
73,856		
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	
0		

conv2d_6 (Conv2D)	(None, 12, 12, 256)	
295,168		
max_pooling2d_6 (MaxPooling2D)	(None, 6, 6, 256)	
0		
global_average_pooling2d_1	(None, 256)	
0		
(GlobalAveragePooling2D)		
dense_2 (Dense)	(None, 256)	
65,792		
dropout (Dropout)	(None, 256)	
0		
dense_3 (Dense)	(None, 1)	
257		

Total params: 454,465 (1.73 MB)

Trainable params: 454,465 (1.73 MB)

Non-trainable params: 0 (0.00 B)

Regularised/scaled CNN at 128x128

```
regl28_model = build_regularised_scaled_cnn(
    IMG_SIZE_128, name="Regularised_Scaled_CNN_128"
)
```

```
regl28_history, regl28_best_epoch = train_model(
    regl28_model,
    train_ds_128_aug,
    val_ds_128,
```

```
checkpoint_path=(f"{MODELS_DIR}/regularised_scaled_cnn_128_best.keras"
),
    class_weight=class_weights,
)
```

```

# Resolution experiment: regularised/scaled CNN at 224x224

reg224_model = build_regularised_scaled_cnn(
    IMG_SIZE_224, name="Regularised_Scaled_CNN_224"
)

reg224_history, reg224_best_epoch = train_model(
    reg224_model,
    train_ds_224_aug,
    val_ds_224,

checkpoint_path=(f"{MODELS_DIR}/regularised_scaled_cnn_224_best.keras"
),
    class_weight=class_weights,
)

# Resolution comparison on validation set (no retraining needed)

import pandas as pd
import tensorflow as tf

# Load best saved models

reg128_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/regularised_scaled_cnn_128_best.keras",
    compile=False
)

reg224_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/regularised_scaled_cnn_224_best.keras",
    compile=False
)

print("Regularised scaled CNN models loaded.")

# 128x128 validation evaluation

reg128_threshold = find_best_threshold(reg128_model, val_ds_128)

reg128_results = evaluate_model(
    reg128_model,
    val_ds_128,
    name="Regularised Scaled CNN (128px)",
    threshold=reg128_threshold,
    split_name="Validation",
)

plot_confusion_matrix(
    reg128_results["cm"],
    "Regularised Scaled CNN @128x128 - Validation Confusion Matrix",

```

```

    save_path=f"{BASE}/cm_reg_scaled_128_validation.png",
)

# 224x224 validation evaluation

reg224_threshold = find_best_threshold(reg224_model, val_ds_224)

reg224_results = evaluate_model(
    reg224_model,
    val_ds_224,
    name="Regularised Scaled CNN (224px)",
    threshold=reg224_threshold,
    split_name="Validation",
)

plot_confusion_matrix(
    reg224_results["cm"],
    "Regularised Scaled CNN @224x224 - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_reg_scaled_224_validation.png",
)

# Resolution comparison

resolution_comparison = pd.DataFrame([reg128_results, reg224_results])
[
    ["model", "threshold", "accuracy", "sensitivity", "specificity",
"precision", "auc"]
]

print("\nVALIDATION RESOLUTION COMPARISON")
print("=" * 60)

display(resolution_comparison.round(4))

Regularised scaled CNN models loaded.
Optimal threshold (Youden's J): 0.557 (val sensitivity=0.862, val
specificity=0.535)

```

```

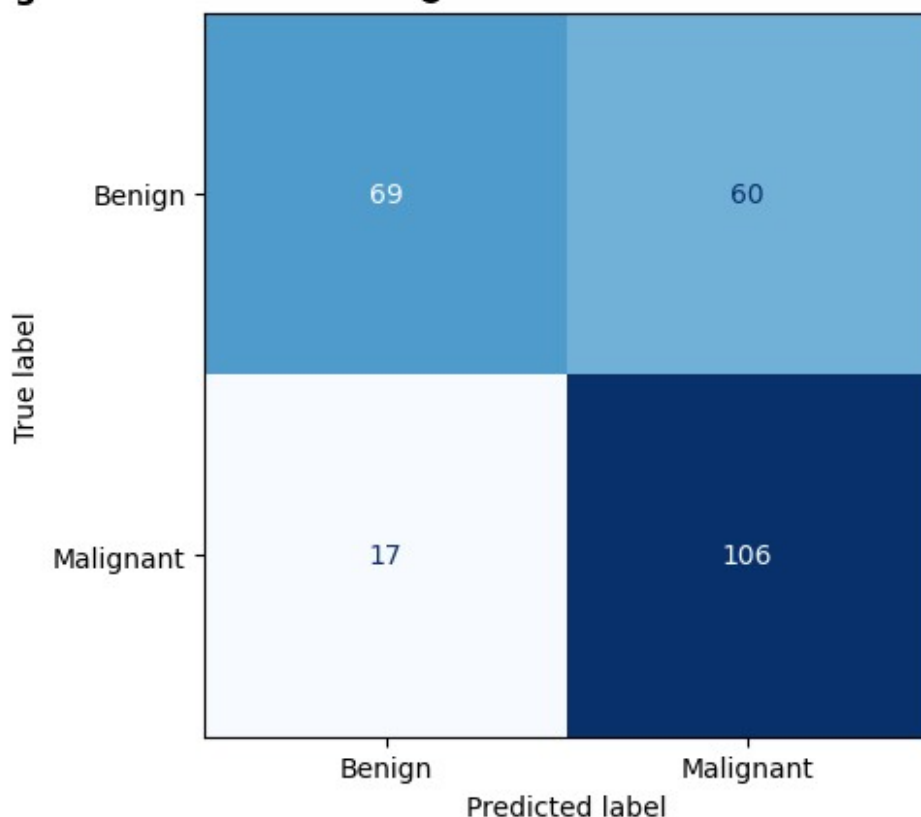
=====
Regularised Scaled CNN (128px) – Validation Results (threshold=0.557)
=====

```

	precision	recall	f1-score	support
Benign	0.80	0.53	0.64	129
Malignant	0.64	0.86	0.73	123
accuracy			0.69	252
macro avg	0.72	0.70	0.69	252
weighted avg	0.72	0.69	0.69	252

Accuracy : 0.6944
 Sensitivity : 0.8618 (missed 17 of 123 malignant cases)
 Specificity : 0.5349
 Precision : 0.6386
 AUC : 0.7353

Regularised Scaled CNN @128×128 — Validation Confusion Matrix



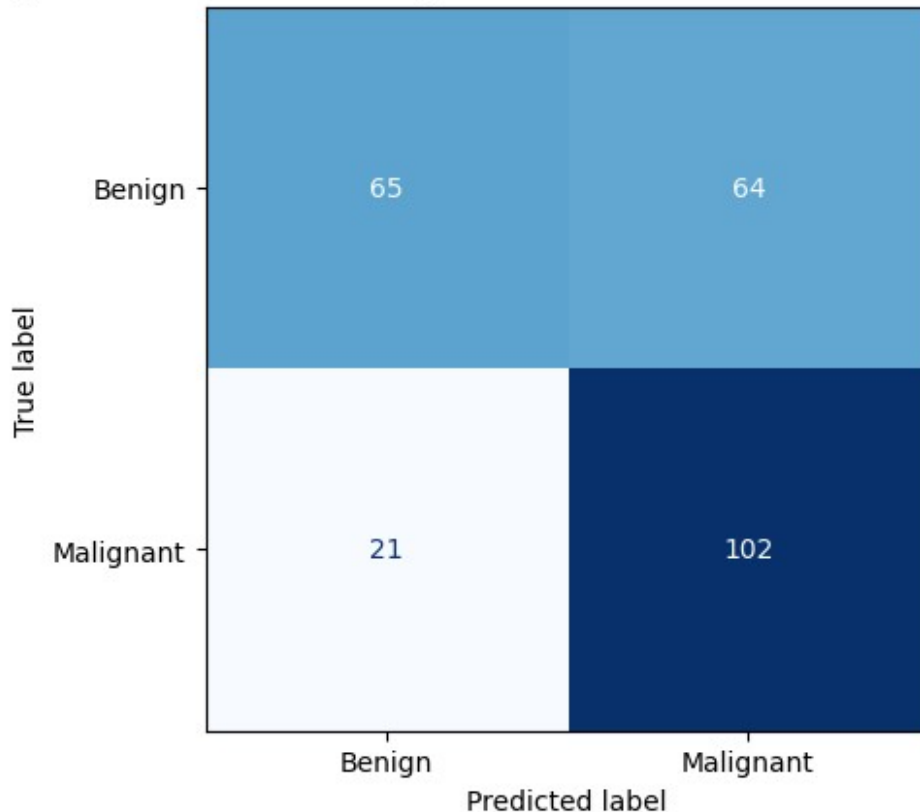
Optimal threshold (Youden's J): 0.469 (val sensitivity=0.829, val specificity=0.504)

Regularised Scaled CNN (224px) – Validation Results (threshold=0.469)

	precision	recall	f1-score	support
Benign	0.76	0.50	0.60	129
Malignant	0.61	0.83	0.71	123
accuracy			0.66	252
macro avg	0.69	0.67	0.66	252
weighted avg	0.69	0.66	0.65	252

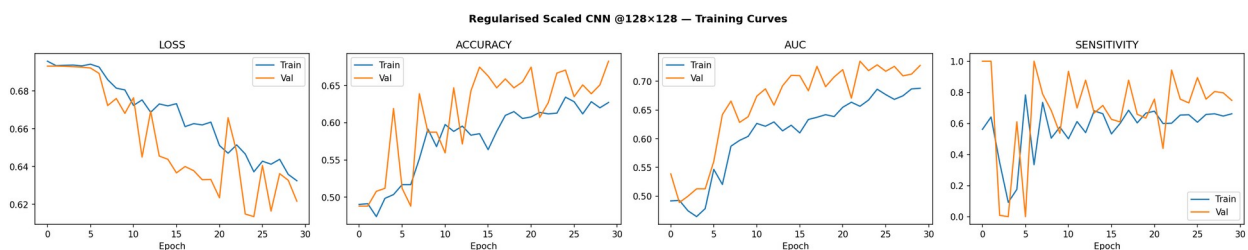
Accuracy : 0.6627
Sensitivity : 0.8293 (missed 21 of 123 malignant cases)
Specificity : 0.5039
Precision : 0.6145
AUC : 0.6856

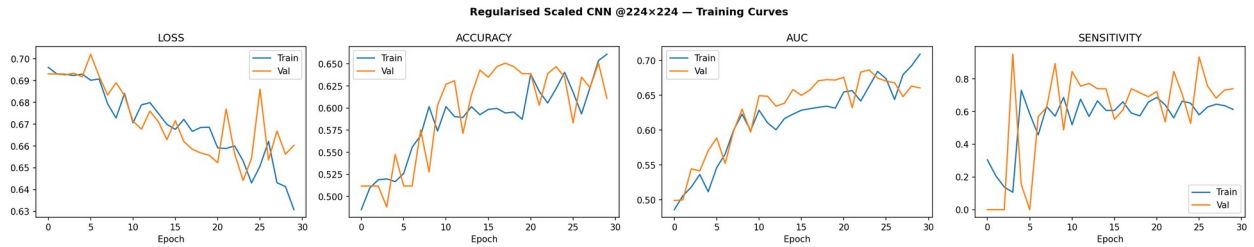
Regularised Scaled CNN @224x224 — Validation Confusion Matrix



VALIDATION RESOLUTION COMPARISON

```
=====
{"summary":{"\n  \"name\": \"\")\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Regularised Scaled CNN (224px)\",\n          \"Regularised Scaled CNN (128px)\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"threshold\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.06201326471006022,\n        \"min\": 0.469,\n        \"max\": 0.5567,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.469,\n          0.5567\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}}
```





Section 15: Grad-CAM - Regularised/Scaled CNNs

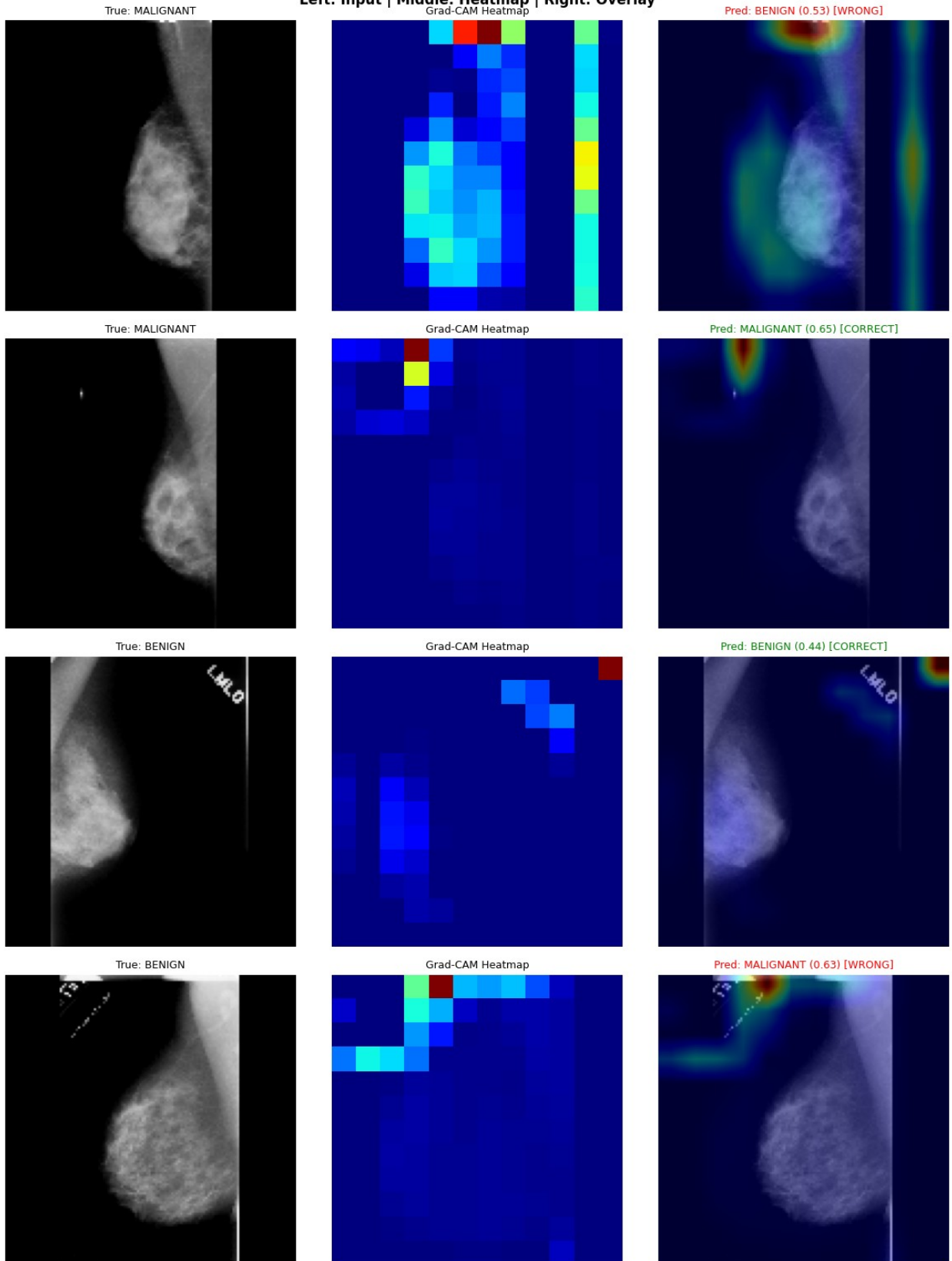
Grad-CAM visualisations are generated for the regularised/scaled CNN at both input resolutions. These provide a qualitative view of the image regions influencing predictions and complement the validation-based resolution comparison.

```
# Grad-CAM: regularised/scaled CNN

plot_gradcam_grid(
    reg128_model,
    "Regularised Scaled CNN @128x128",
    IMG_SIZE_128,
    threshold=reg128_threshold,
    save_path=f"{BASE}/gradcam_reg_scaled_128.png",
)

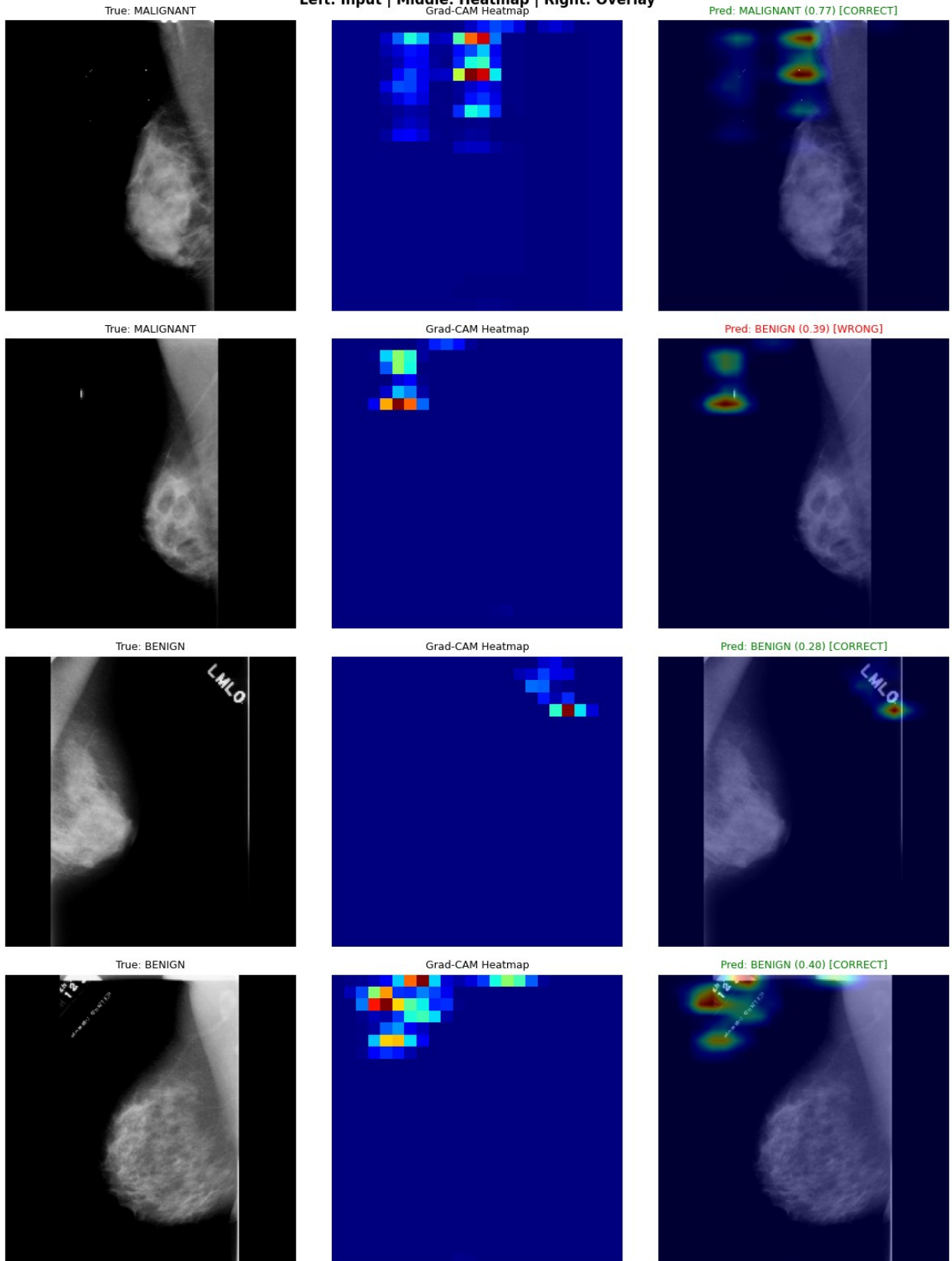
plot_gradcam_grid(
    reg224_model,
    "Regularised Scaled CNN @224x224",
    IMG_SIZE_224,
    threshold=reg224_threshold,
    save_path=f"{BASE}/gradcam_reg_scaled_224.png",
)
```

Grad-CAM Visualisation — Regularised Scaled CNN @128×128
Left: Input | Middle: Heatmap | Right: Overlay



Grad-CAM Visualisation — Regularised Scaled CNN @224×224

Left: Input | Middle: Heatmap | Right: Overlay



Section 16: Custom CNN Stage Comparison

The main custom CNN configurations are compared using validation performance: the **baseline CNN**, **regularised/scaled CNN at 128×128**, and **regularised/scaled CNN at 224×224**.

This summarises the development of the custom CNN before progressing to pretrained transfer-learning models.

```
# Stage comparison: baseline vs regularised CNN (128 vs 224)

# Reload the principal baseline model
baseline_principal_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/baseline_cnn_best.keras", compile=False
)

# Validation-derived threshold
baseline_principal_threshold =
find_best_threshold(baseline_principal_model, val_ds_128)

# Validation evaluation
baseline_principal_results = evaluate_model(
    baseline_principal_model,
    val_ds_128,
    name="Baseline CNN (128px)",
    threshold=baseline_principal_threshold,
    split_name="Validation",
    verbose=False,
)

# Compare development stages using VALIDATION only
stage_comparison = pd.DataFrame(
    [baseline_principal_results, reg128_results, reg224_results]
)[["model", "accuracy", "sensitivity", "specificity", "precision",
"auc"]]

display(stage_comparison.round(4))

Optimal threshold (Youden's J): 0.531 (val sensitivity=0.634, val
specificity=0.698)

{"summary": "{\n  \"name\": \"display(stage_comparison\", \n  \"rows\":
3, \n  \"fields\": [\n    {\n      \"column\": \"model\", \n
\"properties\": {\n        \"dtype\": \"string\", \n
\"num_unique_values\": 3, \n        \"samples\": [\n
\"Baseline CNN (128px)\", \n        \"Regularised Scaled CNN
(128px)\", \n        \"Regularised Scaled CNN (224px)\", \n
], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n
}, \n        {\n          \"column\": \"accuracy\", \n          \"properties\":
{\n            \"dtype\": \"number\", \n            \"std\":
0.01726354540643379, \n            \"min\": 0.6627, \n            \"max\":
```

```

0.6944,\n          \"num_unique_values\": 3,\n          \"samples\": [\n0.6667,\n          0.6944,\n          0.6627\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"sensitivity\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.1231574737209778,\n          \"min\": 0.6341,\n          \"max\": 0.8618,\n          \"num_unique_values\": 3,\n          \"samples\": [\n0.6341,\n          0.8618,\n          0.8293\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"specificity\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.10410193722180837,\n          \"min\": 0.5039,\n          \"max\": 0.6977,\n          \"num_unique_values\": 3,\n          \"samples\": [\n0.6977,\n          0.5349,\n          0.5039\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"precision\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.026125530297648143,\n          \"min\": 0.6145,\n          \"max\": 0.6667,\n          \"num_unique_values\": 3,\n          \"samples\": [\n0.6667,\n          0.6386,\n          0.6145\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"auc\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.026644511629977368,\n          \"min\": 0.6856,\n          \"max\": 0.7353,\n          \"num_unique_values\": 3,\n          \"samples\": [\n          0.6938,\n          0.7353,\n          0.6856\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n          }\n          }\n          ],\n          \"type\": \"dataframe\"}

```

```

# Stage comparison bar chart

```

```

fig, ax = plt.subplots(figsize=(9, 5))

metrics_to_plot = ["accuracy", "sensitivity", "specificity", "auc"]

x = np.arange(len(metrics_to_plot))
width = 0.25

for i, row in stage_comparison.iterrows():

    ax.bar(x + i * width, [row[m] for m in metrics_to_plot], width,
label=row["model"])

ax.set_xticks(x + width)

ax.set_xticklabels([m.capitalize() for m in metrics_to_plot])

ax.set_ylim(0, 1)

ax.set_title("Baseline vs Regularised CNN (128px & 224px)",

```

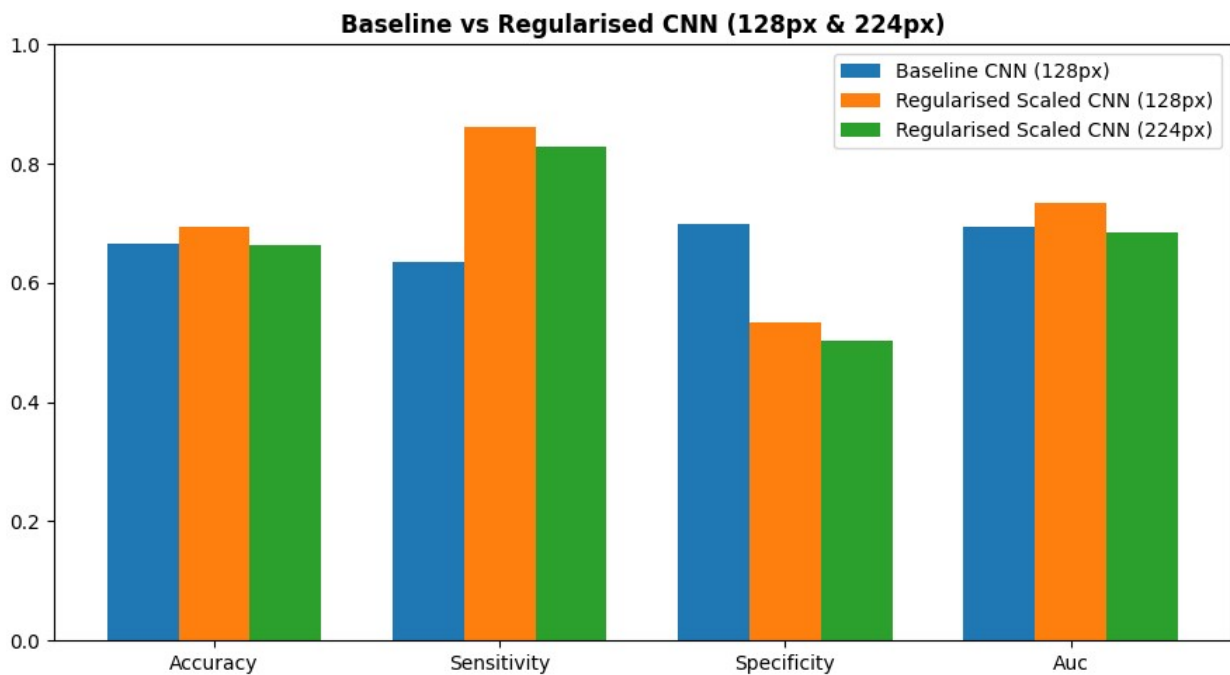
```

fontweight="bold")
ax.legend()
plt.tight_layout()

plt.savefig(f"{BASE}/comparison_stage2.png", dpi=150,
bbox_inches="tight")

plt.show()

```



Section 17: Transfer Learning - Build and Fine-Tune Strategy

Transfer learning is investigated using **VGG16** and **ResNet50** pretrained on ImageNet. Each model uses a classification head with global average pooling, a dense layer, dropout and a sigmoid output.

Training follows two stages: initial feature extraction with the pretrained base frozen, followed by selective fine-tuning of the upper layers using a lower learning rate. Model-specific ImageNet preprocessing is applied to each architecture.

```

def prepare_vgg_dataset(ds):
    # VGG16 expects its ImageNet preprocessing, not plain 0-1 inputs
    return ds.map(
        lambda images, labels: (vgg16_preprocess(images * 255.0),
labels),

```

```

        num_parallel_calls=AUTOTUNE,
    ).prefetch(AUTOTUNE)

def prepare_resnet_dataset(ds):
    # ResNet50 uses its own ImageNet preprocessing
    return ds.map(
        lambda images, labels: (resnet50_preprocess(images * 255.0),
labels),
        num_parallel_calls=AUTOTUNE,
    ).prefetch(AUTOTUNE)

# Model-specific datasets keep the same samples and labels
vgg_train_ds = prepare_vgg_dataset(train_ds_224_aug)
vgg_val_ds = prepare_vgg_dataset(val_ds_224)
vgg_test_ds = prepare_vgg_dataset(test_ds_224)

resnet_train_ds = prepare_resnet_dataset(train_ds_224_aug)
resnet_val_ds = prepare_resnet_dataset(val_ds_224)
resnet_test_ds = prepare_resnet_dataset(test_ds_224)

def build_transfer_model(base_class, img_size=IMG_SIZE_224,
name="transfer_model"):
    base_model = base_class(
        weights="imagenet", include_top=False, input_shape=(img_size,
img_size, 3)
    )
    base_model.trainable = False # frozen for Stage A

    x = base_model.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = tf.keras.Model(inputs=base_model.input, outputs=outputs,
name=name)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )

```

```

    return model, base_model

def fine_tune(model, base_model, unfreeze_fraction=0.3):
    # Unfreeze the top part of the pretrained model
    base_model.trainable = True
    n_layers = len(base_model.layers)
    freeze_until = int(n_layers * (1 - unfreeze_fraction))

    for layer in base_model.layers[:freeze_until]:
        layer.trainable = False

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.AUC(name="auc"),
            tf.keras.metrics.Recall(name="sensitivity"),
            tf.keras.metrics.Precision(name="precision"),
        ],
    )

    print(f"Unfroze top {n_layers - freeze_until} of {n_layers} base layers.")
    return model

def run_transfer_learning(
    base_class, model_name, checkpoint_prefix, train_ds, val_ds,
    class_weight=None
):
    print(f"\n{'='*60}\n{model_name} - Stage A: frozen head (10 epochs)\n{'='*60}")

    model, base_model = build_transfer_model(base_class, IMG_SIZE_224,
name=model_name)

    head_history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=10,
        verbose=1,
        class_weight=class_weight,
    )

    print(f"\n{'='*60}\n{model_name} - Stage B: fine-tuning (lr=1e-5)\n{'='*60}")

    model = fine_tune(model, base_model)

```

```

ft_callbacks = [
    EarlyStopping(
        monitor="val_auc",
        patience=7,
        restore_best_weights=True,
        mode="max",
        verbose=1,
    ),
    ModelCheckpoint(
        filepath=f"{MODELS_DIR}/{checkpoint_prefix}_best.keras",
        monitor="val_auc",
        save_best_only=True,
        mode="max",
        verbose=1,
    ),
]

ft_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=ft_callbacks,
    verbose=1,
    class_weight=class_weight,
)

combined_history = {
    k: head_history.history[k] + ft_history.history[k] for k in
head_history.history
}

model.save(f"{MODELS_DIR}/{checkpoint_prefix}_final.keras")
return model, combined_history

print("Transfer learning helpers ready.")
Transfer learning helpers ready.

```

Section 18: Transfer Learning - VGG16

VGG16 is trained using ImageNet-pretrained weights and VGG16-specific preprocessing. Following staged training and fine-tuning, the best saved checkpoint is evaluated on the validation set using a validation-derived decision threshold.

```

# Train VGG16 using VGG16 ImageNet preprocessing
vgg_model, vgg_history = run_transfer_learning(
    VGG16,

```

```

"VGG16",
"vgg16",
train_ds=vgg_train_ds,
val_ds=vgg_val_ds,
class_weight=class_weights,
)

```

```

=====
VGG16 – Stage A: frozen head (10 epochs)
=====

```

Epoch 1/10

```

31/31 ————— 15s 382ms/step - accuracy: 0.5128 - auc:
0.4968 - loss: 1.2022 - precision: 0.5021 - sensitivity: 0.5094 -
val_accuracy: 0.5357 - val_auc: 0.6256 - val_loss: 0.7978 -
val_precision: 0.5133 - val_sensitivity: 0.9431

```

Epoch 2/10

```

31/31 ————— 7s 225ms/step - accuracy: 0.5434 - auc:
0.5696 - loss: 0.7851 - precision: 0.5308 - sensitivity: 0.5762 -
val_accuracy: 0.6071 - val_auc: 0.6529 - val_loss: 0.6633 -
val_precision: 0.6463 - val_sensitivity: 0.4309

```

Epoch 3/10

```

31/31 ————— 7s 232ms/step - accuracy: 0.5914 - auc:
0.6195 - loss: 0.6759 - precision: 0.5738 - sensitivity: 0.6409 -
val_accuracy: 0.5794 - val_auc: 0.6438 - val_loss: 0.6607 -
val_precision: 0.5876 - val_sensitivity: 0.4634

```

Epoch 4/10

```

31/31 ————— 7s 234ms/step - accuracy: 0.6067 - auc:
0.6581 - loss: 0.6598 - precision: 0.6049 - sensitivity: 0.5658 -
val_accuracy: 0.6389 - val_auc: 0.6948 - val_loss: 0.6542 -
val_precision: 0.5889 - val_sensitivity: 0.8618

```

Epoch 5/10

```

31/31 ————— 8s 241ms/step - accuracy: 0.6251 - auc:
0.6656 - loss: 0.6508 - precision: 0.6061 - sensitivity: 0.6681 -
val_accuracy: 0.6508 - val_auc: 0.7119 - val_loss: 0.6415 -
val_precision: 0.6061 - val_sensitivity: 0.8130

```

Epoch 6/10

```

31/31 ————— 8s 244ms/step - accuracy: 0.6588 - auc:
0.7091 - loss: 0.6225 - precision: 0.6489 - sensitivity: 0.6597 -
val_accuracy: 0.6548 - val_auc: 0.6897 - val_loss: 0.6554 -
val_precision: 0.6154 - val_sensitivity: 0.7805

```

Epoch 7/10

```

31/31 ————— 8s 237ms/step - accuracy: 0.6650 - auc:
0.7225 - loss: 0.6167 - precision: 0.6519 - sensitivity: 0.6764 -
val_accuracy: 0.6349 - val_auc: 0.6983 - val_loss: 0.6454 -
val_precision: 0.6148 - val_sensitivity: 0.6748

```

Epoch 8/10

```

31/31 ————— 7s 238ms/step - accuracy: 0.6793 - auc:
0.7331 - loss: 0.6054 - precision: 0.6653 - sensitivity: 0.6931 -
val_accuracy: 0.6071 - val_auc: 0.7108 - val_loss: 0.6453 -

```

```
val_precision: 0.6500 - val_sensitivity: 0.4228
Epoch 9/10
31/31 _____ 7s 230ms/step - accuracy: 0.6547 - auc:
0.7321 - loss: 0.6066 - precision: 0.6402 - sensitivity: 0.6722 -
val_accuracy: 0.6151 - val_auc: 0.7038 - val_loss: 0.6321 -
val_precision: 0.6204 - val_sensitivity: 0.5447
Epoch 10/10
31/31 _____ 7s 229ms/step - accuracy: 0.6782 - auc:
0.7494 - loss: 0.5893 - precision: 0.6559 - sensitivity: 0.7203 -
val_accuracy: 0.6468 - val_auc: 0.7186 - val_loss: 0.6236 -
val_precision: 0.6604 - val_sensitivity: 0.5691
```

```
=====
VGG16 - Stage B: fine-tuning (lr=1e-5)
=====
```

Unfroze top 6 of 19 base layers.

```
Epoch 1/20
31/31 _____ 0s 332ms/step - accuracy: 0.6922 - auc:
0.7578 - loss: 0.5831 - precision: 0.6819 - sensitivity: 0.6746
Epoch 1: val_auc improved from None to 0.75484, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras
```

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras

```
31/31 _____ 20s 474ms/step - accuracy: 0.7130 - auc:
0.7799 - loss: 0.5682 - precision: 0.7012 - sensitivity: 0.7203 -
val_accuracy: 0.6746 - val_auc: 0.7548 - val_loss: 0.5915 -
val_precision: 0.6814 - val_sensitivity: 0.6260
```

```
Epoch 2/20
31/31 _____ 0s 221ms/step - accuracy: 0.7247 - auc:
0.8006 - loss: 0.5446 - precision: 0.7119 - sensitivity: 0.7668
Epoch 2: val_auc improved from 0.75484 to 0.77182, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras
```

Epoch 2: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras

```
31/31 _____ 16s 528ms/step - accuracy: 0.7314 - auc:
0.7982 - loss: 0.5469 - precision: 0.7151 - sensitivity: 0.7495 -
val_accuracy: 0.7063 - val_auc: 0.7718 - val_loss: 0.5832 -
val_precision: 0.7025 - val_sensitivity: 0.6911
```

```
Epoch 3/20
31/31 _____ 0s 228ms/step - accuracy: 0.7560 - auc:
0.8312 - loss: 0.5136 - precision: 0.7687 - sensitivity: 0.7284
Epoch 3: val_auc improved from 0.77182 to 0.78663, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras
```

Epoch 3: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras

31/31 ————— 12s 398ms/step - accuracy: 0.7375 - auc:
0.8171 - loss: 0.5286 - precision: 0.7275 - sensitivity: 0.7411 -
val_accuracy: 0.6944 - val_auc: 0.7866 - val_loss: 0.5752 -
val_precision: 0.7447 - val_sensitivity: 0.5691

Epoch 4/20

31/31 ————— 0s 233ms/step - accuracy: 0.7676 - auc:
0.8607 - loss: 0.4816 - precision: 0.7774 - sensitivity: 0.7528

Epoch 4: val_auc did not improve from 0.78663

31/31 ————— 9s 290ms/step - accuracy: 0.7600 - auc:
0.8335 - loss: 0.5066 - precision: 0.7574 - sensitivity: 0.7495 -
val_accuracy: 0.7302 - val_auc: 0.7858 - val_loss: 0.5642 -
val_precision: 0.7165 - val_sensitivity: 0.7398

Epoch 5/20

31/31 ————— 0s 236ms/step - accuracy: 0.7259 - auc:
0.8270 - loss: 0.5055 - precision: 0.6901 - sensitivity: 0.7811

Epoch 5: val_auc did not improve from 0.78663

31/31 ————— 9s 290ms/step - accuracy: 0.7477 - auc:
0.8464 - loss: 0.4905 - precision: 0.7257 - sensitivity: 0.7787 -
val_accuracy: 0.6984 - val_auc: 0.7798 - val_loss: 0.5876 -
val_precision: 0.6577 - val_sensitivity: 0.7967

Epoch 6/20

31/31 ————— 0s 235ms/step - accuracy: 0.7654 - auc:
0.8598 - loss: 0.4705 - precision: 0.7442 - sensitivity: 0.7608

Epoch 6: val_auc did not improve from 0.78663

31/31 ————— 9s 289ms/step - accuracy: 0.7753 - auc:
0.8602 - loss: 0.4661 - precision: 0.7738 - sensitivity: 0.7641 -
val_accuracy: 0.6944 - val_auc: 0.7809 - val_loss: 0.5883 -
val_precision: 0.7674 - val_sensitivity: 0.5366

Epoch 7/20

31/31 ————— 0s 229ms/step - accuracy: 0.7896 - auc:
0.8705 - loss: 0.4498 - precision: 0.7817 - sensitivity: 0.7746

Epoch 7: val_auc improved from 0.78663 to 0.79120, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.
keras

31/31 ————— 16s 504ms/step - accuracy: 0.7896 - auc:
0.8758 - loss: 0.4455 - precision: 0.7826 - sensitivity: 0.7891 -
val_accuracy: 0.6984 - val_auc: 0.7912 - val_loss: 0.5667 -
val_precision: 0.6643 - val_sensitivity: 0.7724

Epoch 8/20

31/31 ————— 0s 219ms/step - accuracy: 0.7952 - auc:
0.8887 - loss: 0.4282 - precision: 0.7813 - sensitivity: 0.8322

Epoch 8: val_auc improved from 0.79120 to 0.79908, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 8: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 14s 462ms/step - accuracy: 0.7937 - auc: 0.8863 - loss: 0.4298 - precision: 0.7787 - sensitivity: 0.8079 - val_accuracy: 0.7183 - val_auc: 0.7991 - val_loss: 0.5598 - val_precision: 0.7097 - val_sensitivity: 0.7154

Epoch 9/20

31/31 ————— 0s 220ms/step - accuracy: 0.7956 - auc: 0.8911 - loss: 0.4176 - precision: 0.7985 - sensitivity: 0.7782

Epoch 9: val_auc improved from 0.79908 to 0.80677, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 9: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 10s 304ms/step - accuracy: 0.8141 - auc: 0.9039 - loss: 0.4036 - precision: 0.8126 - sensitivity: 0.8058 - val_accuracy: 0.7421 - val_auc: 0.8068 - val_loss: 0.5478 - val_precision: 0.7339 - val_sensitivity: 0.7398

Epoch 10/20

31/31 ————— 0s 226ms/step - accuracy: 0.8258 - auc: 0.9176 - loss: 0.3783 - precision: 0.8045 - sensitivity: 0.8497

Epoch 10: val_auc did not improve from 0.80677

31/31 ————— 10s 313ms/step - accuracy: 0.8264 - auc: 0.9148 - loss: 0.3781 - precision: 0.8147 - sensitivity: 0.8351 - val_accuracy: 0.7183 - val_auc: 0.7933 - val_loss: 0.5922 - val_precision: 0.7766 - val_sensitivity: 0.5935

Epoch 11/20

31/31 ————— 0s 232ms/step - accuracy: 0.8428 - auc: 0.9236 - loss: 0.3673 - precision: 0.8541 - sensitivity: 0.8257

Epoch 11: val_auc did not improve from 0.80677

31/31 ————— 9s 285ms/step - accuracy: 0.8519 - auc: 0.9227 - loss: 0.3623 - precision: 0.8538 - sensitivity: 0.8413 - val_accuracy: 0.7341 - val_auc: 0.8000 - val_loss: 0.5740 - val_precision: 0.7545 - val_sensitivity: 0.6748

Epoch 12/20

31/31 ————— 0s 236ms/step - accuracy: 0.8217 - auc: 0.9127 - loss: 0.3768 - precision: 0.8108 - sensitivity: 0.8415

Epoch 12: val_auc did not improve from 0.80677

31/31 ————— 9s 295ms/step - accuracy: 0.8315 - auc: 0.9169 - loss: 0.3694 - precision: 0.8230 - sensitivity: 0.8351 - val_accuracy: 0.7421 - val_auc: 0.7956 - val_loss: 0.5845 - val_precision: 0.7589 - val_sensitivity: 0.6911

Epoch 13/20

31/31 ————— 0s 231ms/step - accuracy: 0.8754 - auc: 0.9443 - loss: 0.3244 - precision: 0.8565 - sensitivity: 0.8908

Epoch 13: val_auc did not improve from 0.80677

31/31 ————— 9s 287ms/step - accuracy: 0.8662 - auc: 0.9353 - loss: 0.3384 - precision: 0.8537 - sensitivity: 0.8768 - val_accuracy: 0.7183 - val_auc: 0.8013 - val_loss: 0.5701 - val_precision: 0.7097 - val_sensitivity: 0.7154

Epoch 14/20

31/31 ————— 0s 227ms/step - accuracy: 0.8554 - auc: 0.9365 - loss: 0.3336 - precision: 0.8427 - sensitivity: 0.8773

Epoch 14: val_auc improved from 0.80677 to 0.81058, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 14: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 13s 406ms/step - accuracy: 0.8519 - auc: 0.9402 - loss: 0.3218 - precision: 0.8465 - sensitivity: 0.8518 - val_accuracy: 0.7183 - val_auc: 0.8106 - val_loss: 0.5717 - val_precision: 0.6806 - val_sensitivity: 0.7967

Epoch 15/20

31/31 ————— 0s 222ms/step - accuracy: 0.8446 - auc: 0.9378 - loss: 0.3232 - precision: 0.8202 - sensitivity: 0.8513

Epoch 15: val_auc did not improve from 0.81058

31/31 ————— 9s 276ms/step - accuracy: 0.8437 - auc: 0.9331 - loss: 0.3305 - precision: 0.8327 - sensitivity: 0.8518 - val_accuracy: 0.6984 - val_auc: 0.8062 - val_loss: 0.5710 - val_precision: 0.6942 - val_sensitivity: 0.6829

Epoch 16/20

31/31 ————— 0s 224ms/step - accuracy: 0.8603 - auc: 0.9391 - loss: 0.3229 - precision: 0.8702 - sensitivity: 0.8546

Epoch 16: val_auc did not improve from 0.81058

31/31 ————— 9s 279ms/step - accuracy: 0.8631 - auc: 0.9391 - loss: 0.3227 - precision: 0.8586 - sensitivity: 0.8622 - val_accuracy: 0.7143 - val_auc: 0.8050 - val_loss: 0.5869 - val_precision: 0.6835 - val_sensitivity: 0.7724

Epoch 17/20

31/31 ————— 0s 224ms/step - accuracy: 0.8667 - auc: 0.9409 - loss: 0.3176 - precision: 0.8702 - sensitivity: 0.8531

Epoch 17: val_auc did not improve from 0.81058

31/31 ————— 9s 277ms/step - accuracy: 0.8846 - auc: 0.9491 - loss: 0.2987 - precision: 0.8861 - sensitivity: 0.8768 - val_accuracy: 0.7183 - val_auc: 0.8063 - val_loss: 0.5773 - val_precision: 0.7364 - val_sensitivity: 0.6585

Epoch 18/20

31/31 ————— 0s 227ms/step - accuracy: 0.8618 - auc: 0.9528 - loss: 0.2893 - precision: 0.8419 - sensitivity: 0.8771

Epoch 18: val_auc improved from 0.81058 to 0.81553, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 18: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 15s 475ms/step - accuracy: 0.8672 - auc: 0.9511 - loss: 0.2905 - precision: 0.8613 - sensitivity: 0.8685 - val_accuracy: 0.7222 - val_auc: 0.8155 - val_loss: 0.5715 - val_precision: 0.7524 - val_sensitivity: 0.6423

Epoch 19/20

31/31 ————— 0s 226ms/step - accuracy: 0.8922 - auc: 0.9628 - loss: 0.2636 - precision: 0.8900 - sensitivity: 0.8943

Epoch 19: val_auc improved from 0.81553 to 0.82410, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 19: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 16s 533ms/step - accuracy: 0.8897 - auc: 0.9606 - loss: 0.2677 - precision: 0.8922 - sensitivity: 0.8810 - val_accuracy: 0.7500 - val_auc: 0.8241 - val_loss: 0.5446 - val_precision: 0.7459 - val_sensitivity: 0.7398

Epoch 20/20

31/31 ————— 0s 229ms/step - accuracy: 0.9177 - auc: 0.9725 - loss: 0.2391 - precision: 0.8982 - sensitivity: 0.9363

Epoch 20: val_auc improved from 0.82410 to 0.82517, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

Epoch 20: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/vgg16_best.keras

31/31 ————— 16s 529ms/step - accuracy: 0.8948 - auc: 0.9635 - loss: 0.2623 - precision: 0.8790 - sensitivity: 0.9102 - val_accuracy: 0.7262 - val_auc: 0.8252 - val_loss: 0.5754 - val_precision: 0.7935 - val_sensitivity: 0.5935

Restoring model weights from the end of the best epoch: 20.

```
vgg_model =  
tf.keras.models.load_model(f"{MODELS_DIR}/vgg16_best.keras",  
                           compile=False)
```

```
vgg_threshold = find_best_threshold(vgg_model, vgg_val_ds)
```

```
vgg_results = evaluate_model(  
    vgg_model,  
    vgg_val_ds,
```

```

    name="VGG16",
    threshold=vgg_threshold,
    split_name="Validation",
)

plot_confusion_matrix(
    vgg_results["cm"],
    "VGG16 - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_vgg16_validation.png",
)

```

Optimal threshold (Youden's J): 0.317 (val sensitivity=0.764, val specificity=0.744)

```

=====
VGG16 - Validation Results (threshold=0.317)
=====

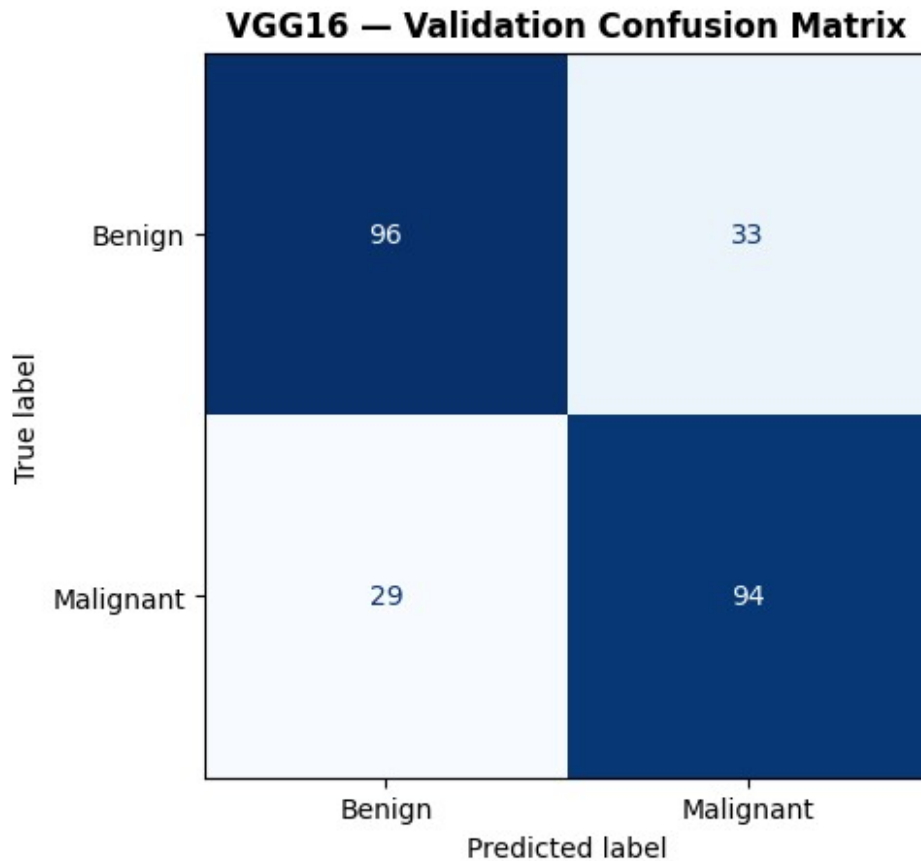
```

	precision	recall	f1-score	support
Benign	0.77	0.74	0.76	129
Malignant	0.74	0.76	0.75	123
accuracy			0.75	252
macro avg	0.75	0.75	0.75	252
weighted avg	0.75	0.75	0.75	252

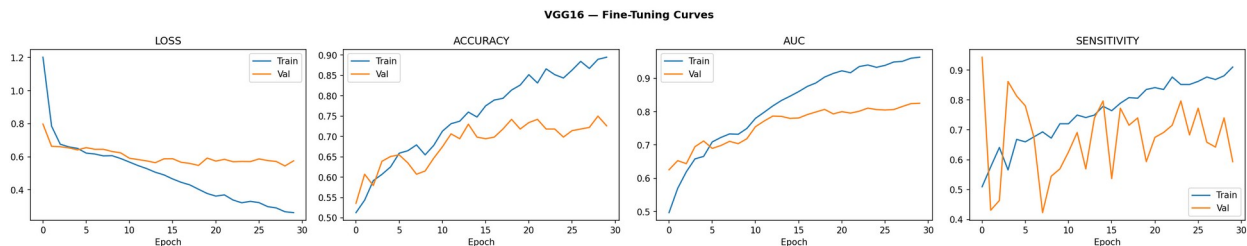
```

Accuracy      : 0.754
Sensitivity   : 0.7642 (missed 29 of 123 malignant cases)
Specificity   : 0.7442
Precision     : 0.7402
AUC           : 0.8254

```



```
# Original training curves
from IPython.display import Image, display
display(Image(filename=f"{BASE}/curves_vgg16.png"))
```



Section 19: Transfer Learning - ResNet50

ResNet50 is trained using ImageNet-pretrained weights and ResNet50-specific preprocessing. The same two-stage transfer-learning strategy is used before evaluating the best checkpoint on the validation set.

```
# Train ResNet50 using ResNet50 ImageNet preprocessing
resnet_model, resnet_history = run_transfer_learning(
```

```
ResNet50,  
"ResNet50",  
"resnet50",  
train_ds=resnet_train_ds,  
val_ds=resnet_val_ds,  
class_weight=class_weights,  
)
```

```
=====  
ResNet50 – Stage A: frozen head (10 epochs)  
=====
```

Epoch 1/10

31/31 ————— 27s 510ms/step - accuracy: 0.4944 - auc:
0.4874 - loss: 0.7759 - precision: 0.4822 - sensitivity: 0.4530 -
val_accuracy: 0.5278 - val_auc: 0.6024 - val_loss: 0.6925 -
val_precision: 1.0000 - val_sensitivity: 0.0325

Epoch 2/10

31/31 ————— 4s 119ms/step - accuracy: 0.5465 - auc:
0.5662 - loss: 0.6864 - precision: 0.5308 - sensitivity: 0.6305 -
val_accuracy: 0.5476 - val_auc: 0.5935 - val_loss: 0.6830 -
val_precision: 0.6800 - val_sensitivity: 0.1382

Epoch 3/10

31/31 ————— 4s 121ms/step - accuracy: 0.5536 - auc:
0.5721 - loss: 0.6857 - precision: 0.5469 - sensitivity: 0.5115 -
val_accuracy: 0.5476 - val_auc: 0.6157 - val_loss: 0.6824 -
val_precision: 0.6667 - val_sensitivity: 0.1463

Epoch 4/10

31/31 ————— 5s 151ms/step - accuracy: 0.5424 - auc:
0.5718 - loss: 0.6844 - precision: 0.5336 - sensitivity: 0.5136 -
val_accuracy: 0.5516 - val_auc: 0.5838 - val_loss: 0.6812 -
val_precision: 0.6000 - val_sensitivity: 0.2439

Epoch 5/10

31/31 ————— 4s 122ms/step - accuracy: 0.6027 - auc:
0.6448 - loss: 0.6600 - precision: 0.5886 - sensitivity: 0.6242 -
val_accuracy: 0.6270 - val_auc: 0.6766 - val_loss: 0.6571 -
val_precision: 0.6883 - val_sensitivity: 0.4309

Epoch 6/10

31/31 ————— 4s 124ms/step - accuracy: 0.5924 - auc:
0.6351 - loss: 0.6617 - precision: 0.5980 - sensitivity: 0.5094 -
val_accuracy: 0.6111 - val_auc: 0.6569 - val_loss: 0.6735 -
val_precision: 0.5604 - val_sensitivity: 0.9431

Epoch 7/10

31/31 ————— 4s 132ms/step - accuracy: 0.5914 - auc:
0.6289 - loss: 0.6644 - precision: 0.5678 - sensitivity: 0.6910 -
val_accuracy: 0.6111 - val_auc: 0.6792 - val_loss: 0.6585 -
val_precision: 0.6437 - val_sensitivity: 0.4553

Epoch 8/10

31/31 ————— 4s 132ms/step - accuracy: 0.6170 - auc:
0.6601 - loss: 0.6560 - precision: 0.6198 - sensitivity: 0.5616 -

val_accuracy: 0.5992 - val_auc: 0.6647 - val_loss: 0.6526 -
val_precision: 0.5917 - val_sensitivity: 0.5772
Epoch 9/10

31/31 ————— 5s 122ms/step - accuracy: 0.6006 - auc:
0.6418 - loss: 0.6591 - precision: 0.5884 - sensitivity: 0.6117 -
val_accuracy: 0.5794 - val_auc: 0.6837 - val_loss: 0.6470 -
val_precision: 0.5691 - val_sensitivity: 0.5691
Epoch 10/10

31/31 ————— 4s 122ms/step - accuracy: 0.5884 - auc:
0.6461 - loss: 0.6609 - precision: 0.5815 - sensitivity: 0.5658 -
val_accuracy: 0.6032 - val_auc: 0.7004 - val_loss: 0.6680 -
val_precision: 0.5530 - val_sensitivity: 0.9756

=====
ResNet50 – Stage B: fine-tuning (lr=1e-5)
=====

Unfroze top 53 of 175 base layers.

Epoch 1/20
31/31 ————— 0s 472ms/step - accuracy: 0.5282 - auc:
0.6089 - loss: 0.7401 - precision: 0.5182 - sensitivity: 0.9419

Epoch 1: val_auc improved from None to 0.70609, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

31/31 ————— 57s 1s/step - accuracy: 0.5322 - auc:
0.6299 - loss: 0.7248 - precision: 0.5120 - sensitivity: 0.9353 -
val_accuracy: 0.6548 - val_auc: 0.7061 - val_loss: 0.6445 -
val_precision: 0.6098 - val_sensitivity: 0.8130

Epoch 2/20
31/31 ————— 0s 154ms/step - accuracy: 0.5771 - auc:
0.6615 - loss: 0.6860 - precision: 0.5282 - sensitivity: 0.8555
Epoch 2: val_auc did not improve from 0.70609

31/31 ————— 6s 187ms/step - accuracy: 0.5986 - auc:
0.6613 - loss: 0.6643 - precision: 0.5614 - sensitivity: 0.8205 -
val_accuracy: 0.6230 - val_auc: 0.7008 - val_loss: 0.6379 -
val_precision: 0.6321 - val_sensitivity: 0.5447

Epoch 3/20
31/31 ————— 0s 154ms/step - accuracy: 0.6327 - auc:
0.7187 - loss: 0.6300 - precision: 0.5840 - sensitivity: 0.7599
Epoch 3: val_auc did not improve from 0.70609

31/31 ————— 6s 198ms/step - accuracy: 0.6609 - auc:
0.7304 - loss: 0.6160 - precision: 0.6252 - sensitivity: 0.7662 -
val_accuracy: 0.6230 - val_auc: 0.7014 - val_loss: 0.6328 -
val_precision: 0.6429 - val_sensitivity: 0.5122

Epoch 4/20
31/31 ————— 0s 160ms/step - accuracy: 0.6739 - auc:

0.7359 - loss: 0.6045 - precision: 0.6509 - sensitivity: 0.7276
Epoch 4: val_auc improved from 0.70609 to 0.71107, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

Epoch 4: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

31/31 ————— 17s 539ms/step - accuracy: 0.6691 - auc:
0.7409 - loss: 0.6003 - precision: 0.6460 - sensitivity: 0.7161 -
val_accuracy: 0.6270 - val_auc: 0.7111 - val_loss: 0.6225 -
val_precision: 0.6283 - val_sensitivity: 0.5772

Epoch 5/20

31/31 ————— 0s 157ms/step - accuracy: 0.6770 - auc:
0.7603 - loss: 0.5871 - precision: 0.6721 - sensitivity: 0.6958

Epoch 5: val_auc improved from 0.71107 to 0.72890, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

31/31 ————— 20s 666ms/step - accuracy: 0.6823 - auc:
0.7639 - loss: 0.5827 - precision: 0.6641 - sensitivity: 0.7098 -
val_accuracy: 0.6706 - val_auc: 0.7289 - val_loss: 0.6090 -
val_precision: 0.6587 - val_sensitivity: 0.6748

Epoch 6/20

31/31 ————— 0s 156ms/step - accuracy: 0.7138 - auc:
0.8086 - loss: 0.5524 - precision: 0.7003 - sensitivity: 0.7703

Epoch 6: val_auc improved from 0.72890 to 0.73959, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

31/31 ————— 16s 539ms/step - accuracy: 0.7120 - auc:
0.8032 - loss: 0.5518 - precision: 0.6862 - sensitivity: 0.7578 -
val_accuracy: 0.6746 - val_auc: 0.7396 - val_loss: 0.5978 -
val_precision: 0.6541 - val_sensitivity: 0.7073

Epoch 7/20

31/31 ————— 0s 162ms/step - accuracy: 0.7286 - auc:
0.8309 - loss: 0.5229 - precision: 0.7063 - sensitivity: 0.7498

Epoch 7: val_auc improved from 0.73959 to 0.75131, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_be
st.keras

31/31 ————— 20s 671ms/step - accuracy: 0.7109 - auc: 0.8203 - loss: 0.5281 - precision: 0.6960 - sensitivity: 0.7265 - val_accuracy: 0.6786 - val_auc: 0.7513 - val_loss: 0.5864 - val_precision: 0.6567 - val_sensitivity: 0.7154

Epoch 8/20

31/31 ————— 0s 155ms/step - accuracy: 0.7821 - auc: 0.8653 - loss: 0.4891 - precision: 0.7667 - sensitivity: 0.7855

Epoch 8: val_auc improved from 0.75131 to 0.76404, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

Epoch 8: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

31/31 ————— 17s 556ms/step - accuracy: 0.7732 - auc: 0.8599 - loss: 0.4917 - precision: 0.7575 - sensitivity: 0.7891 - val_accuracy: 0.6905 - val_auc: 0.7640 - val_loss: 0.5844 - val_precision: 0.6452 - val_sensitivity: 0.8130

Epoch 9/20

31/31 ————— 0s 158ms/step - accuracy: 0.7933 - auc: 0.8798 - loss: 0.4685 - precision: 0.7776 - sensitivity: 0.8254

Epoch 9: val_auc did not improve from 0.76404

31/31 ————— 6s 194ms/step - accuracy: 0.7947 - auc: 0.8794 - loss: 0.4664 - precision: 0.7694 - sensitivity: 0.8288 - val_accuracy: 0.6746 - val_auc: 0.7620 - val_loss: 0.5861 - val_precision: 0.6323 - val_sensitivity: 0.7967

Epoch 10/20

31/31 ————— 0s 165ms/step - accuracy: 0.8257 - auc: 0.9071 - loss: 0.4385 - precision: 0.8089 - sensitivity: 0.8451

Epoch 10: val_auc improved from 0.76404 to 0.76609, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

Epoch 10: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

31/31 ————— 22s 713ms/step - accuracy: 0.8192 - auc: 0.8988 - loss: 0.4403 - precision: 0.8044 - sensitivity: 0.8330 - val_accuracy: 0.6905 - val_auc: 0.7661 - val_loss: 0.5843 - val_precision: 0.6552 - val_sensitivity: 0.7724

Epoch 11/20

31/31 ————— 0s 153ms/step - accuracy: 0.8239 - auc: 0.9008 - loss: 0.4250 - precision: 0.8385 - sensitivity: 0.8023

Epoch 11: val_auc did not improve from 0.76609

31/31 ————— 6s 187ms/step - accuracy: 0.8304 - auc: 0.9115 - loss: 0.4120 - precision: 0.8309 - sensitivity: 0.8205 - val_accuracy: 0.6944 - val_auc: 0.7619 - val_loss: 0.6086 - val_precision: 0.6494 - val_sensitivity: 0.8130

Epoch 12/20

31/31 ————— 0s 160ms/step - accuracy: 0.8386 - auc: 0.9197 - loss: 0.3928 - precision: 0.8434 - sensitivity: 0.8458
Epoch 12: val_auc did not improve from 0.76609
31/31 ————— 6s 206ms/step - accuracy: 0.8498 - auc: 0.9254 - loss: 0.3847 - precision: 0.8320 - sensitivity: 0.8685 - val_accuracy: 0.6667 - val_auc: 0.7587 - val_loss: 0.6230 - val_precision: 0.6226 - val_sensitivity: 0.8049
Epoch 13/20
31/31 ————— 0s 169ms/step - accuracy: 0.8586 - auc: 0.9257 - loss: 0.3742 - precision: 0.8681 - sensitivity: 0.8313
Epoch 13: val_auc did not improve from 0.76609
31/31 ————— 10s 200ms/step - accuracy: 0.8713 - auc: 0.9336 - loss: 0.3585 - precision: 0.8896 - sensitivity: 0.8413 - val_accuracy: 0.6825 - val_auc: 0.7659 - val_loss: 0.6535 - val_precision: 0.6335 - val_sensitivity: 0.8293
Epoch 14/20
31/31 ————— 0s 157ms/step - accuracy: 0.8758 - auc: 0.9440 - loss: 0.3370 - precision: 0.8597 - sensitivity: 0.8790
Epoch 14: val_auc improved from 0.76609 to 0.76675, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

Epoch 14: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras
31/31 ————— 18s 599ms/step - accuracy: 0.8754 - auc: 0.9466 - loss: 0.3295 - precision: 0.8758 - sensitivity: 0.8685 - val_accuracy: 0.6905 - val_auc: 0.7667 - val_loss: 0.6988 - val_precision: 0.6271 - val_sensitivity: 0.9024
Epoch 15/20
31/31 ————— 0s 156ms/step - accuracy: 0.8886 - auc: 0.9547 - loss: 0.3019 - precision: 0.8820 - sensitivity: 0.8912
Epoch 15: val_auc improved from 0.76675 to 0.77491, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras

Epoch 15: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_best.keras
31/31 ————— 15s 490ms/step - accuracy: 0.8989 - auc: 0.9588 - loss: 0.2926 - precision: 0.8893 - sensitivity: 0.9061 - val_accuracy: 0.6905 - val_auc: 0.7749 - val_loss: 0.6673 - val_precision: 0.6471 - val_sensitivity: 0.8049
Epoch 16/20
31/31 ————— 0s 161ms/step - accuracy: 0.9177 - auc: 0.9734 - loss: 0.2510 - precision: 0.9186 - sensitivity: 0.9090
Epoch 16: val_auc did not improve from 0.77491
31/31 ————— 6s 190ms/step - accuracy: 0.9142 - auc: 0.9680 - loss: 0.2629 - precision: 0.9229 - sensitivity: 0.8998 -

```

val_accuracy: 0.6746 - val_auc: 0.7622 - val_loss: 0.7773 -
val_precision: 0.6171 - val_sensitivity: 0.8780
Epoch 17/20
31/31 ━━━━━━━━━━━ 0s 160ms/step - accuracy: 0.9314 - auc:
0.9765 - loss: 0.2308 - precision: 0.9338 - sensitivity: 0.9248
Epoch 17: val_auc did not improve from 0.77491
31/31 ━━━━━━━━━━━ 6s 194ms/step - accuracy: 0.9346 - auc:
0.9807 - loss: 0.2228 - precision: 0.9443 - sensitivity: 0.9207 -
val_accuracy: 0.6944 - val_auc: 0.7594 - val_loss: 0.7410 -
val_precision: 0.6474 - val_sensitivity: 0.8211
Epoch 18/20
31/31 ━━━━━━━━━━━ 0s 174ms/step - accuracy: 0.9581 - auc:
0.9902 - loss: 0.1765 - precision: 0.9564 - sensitivity: 0.9586
Epoch 18: val_auc did not improve from 0.77491
31/31 ━━━━━━━━━━━ 7s 222ms/step - accuracy: 0.9479 - auc:
0.9875 - loss: 0.1861 - precision: 0.9440 - sensitivity: 0.9499 -
val_accuracy: 0.6627 - val_auc: 0.7562 - val_loss: 0.8194 -
val_precision: 0.6187 - val_sensitivity: 0.8049
Epoch 19/20
31/31 ━━━━━━━━━━━ 0s 157ms/step - accuracy: 0.9608 - auc:
0.9896 - loss: 0.1668 - precision: 0.9779 - sensitivity: 0.9447
Epoch 19: val_auc did not improve from 0.77491
31/31 ━━━━━━━━━━━ 9s 188ms/step - accuracy: 0.9622 - auc:
0.9879 - loss: 0.1711 - precision: 0.9702 - sensitivity: 0.9520 -
val_accuracy: 0.6746 - val_auc: 0.7655 - val_loss: 0.7574 -
val_precision: 0.6565 - val_sensitivity: 0.6992
Epoch 20/20
31/31 ━━━━━━━━━━━ 0s 166ms/step - accuracy: 0.9523 - auc:
0.9903 - loss: 0.1589 - precision: 0.9535 - sensitivity: 0.9433
Epoch 20: val_auc did not improve from 0.77491
31/31 ━━━━━━━━━━━ 6s 194ms/step - accuracy: 0.9520 - auc:
0.9901 - loss: 0.1587 - precision: 0.9675 - sensitivity: 0.9332 -
val_accuracy: 0.6667 - val_auc: 0.7633 - val_loss: 0.8179 -
val_precision: 0.6423 - val_sensitivity: 0.7154
Restoring model weights from the end of the best epoch: 15.

resnet_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_best.keras", compile=False
)

resnet_threshold = find_best_threshold(resnet_model, resnet_val_ds)

resnet_results = evaluate_model(
    resnet_model,
    resnet_val_ds,
    name="ResNet50",
    threshold=resnet_threshold,
    split_name="Validation",
)

```

```

plot_confusion_matrix(
    resnet_results["cm"],
    "ResNet50 - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_resnet50_validation.png",
)

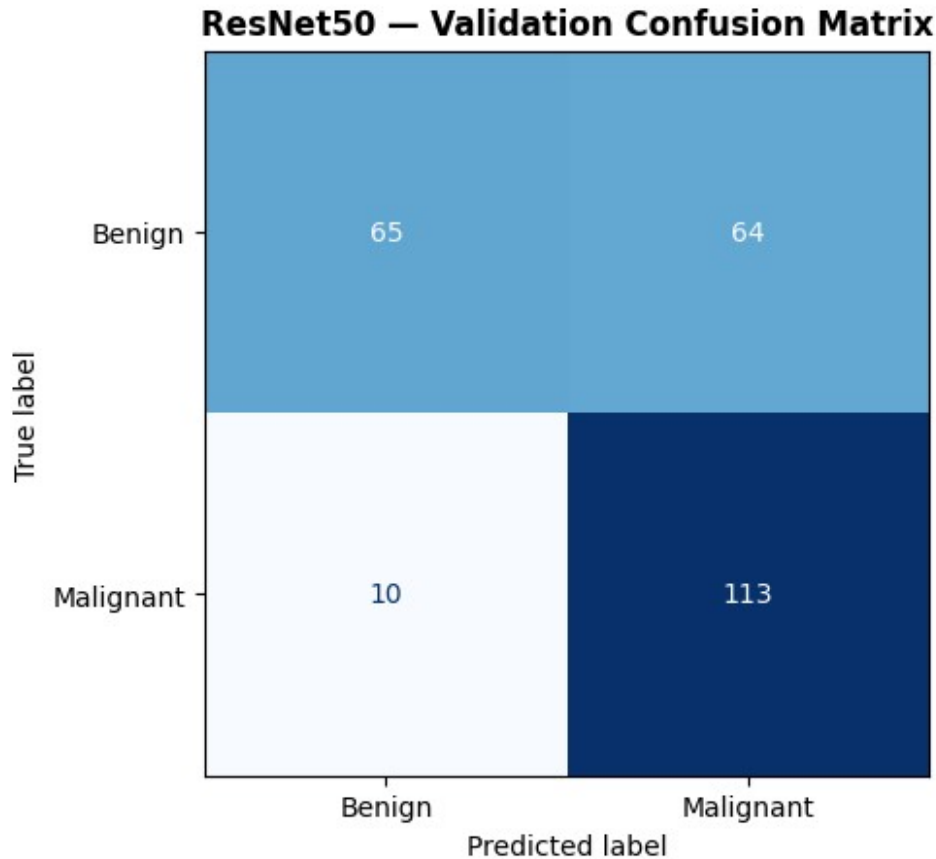
```

Optimal threshold (Youden's J): 0.381 (val sensitivity=0.919, val specificity=0.504)

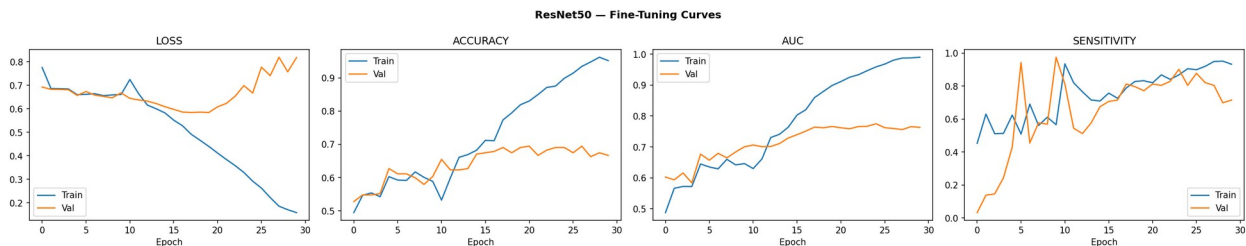
=====
ResNet50 – Validation Results (threshold=0.381)
=====

	precision	recall	f1-score	support
Benign	0.87	0.50	0.64	129
Malignant	0.64	0.92	0.75	123
accuracy			0.71	252
macro avg	0.75	0.71	0.70	252
weighted avg	0.76	0.71	0.69	252

Accuracy : 0.7063
Sensitivity : 0.9187 (missed 10 of 123 malignant cases)
Specificity : 0.5039
Precision : 0.6384
AUC : 0.7751



```
# Original training curves
from IPython.display import Image, display
display(Image(filename=f"{BASE}/curves_resnet50.png"))
```



Section 20: Grad-CAM - VGG16 and ResNet50

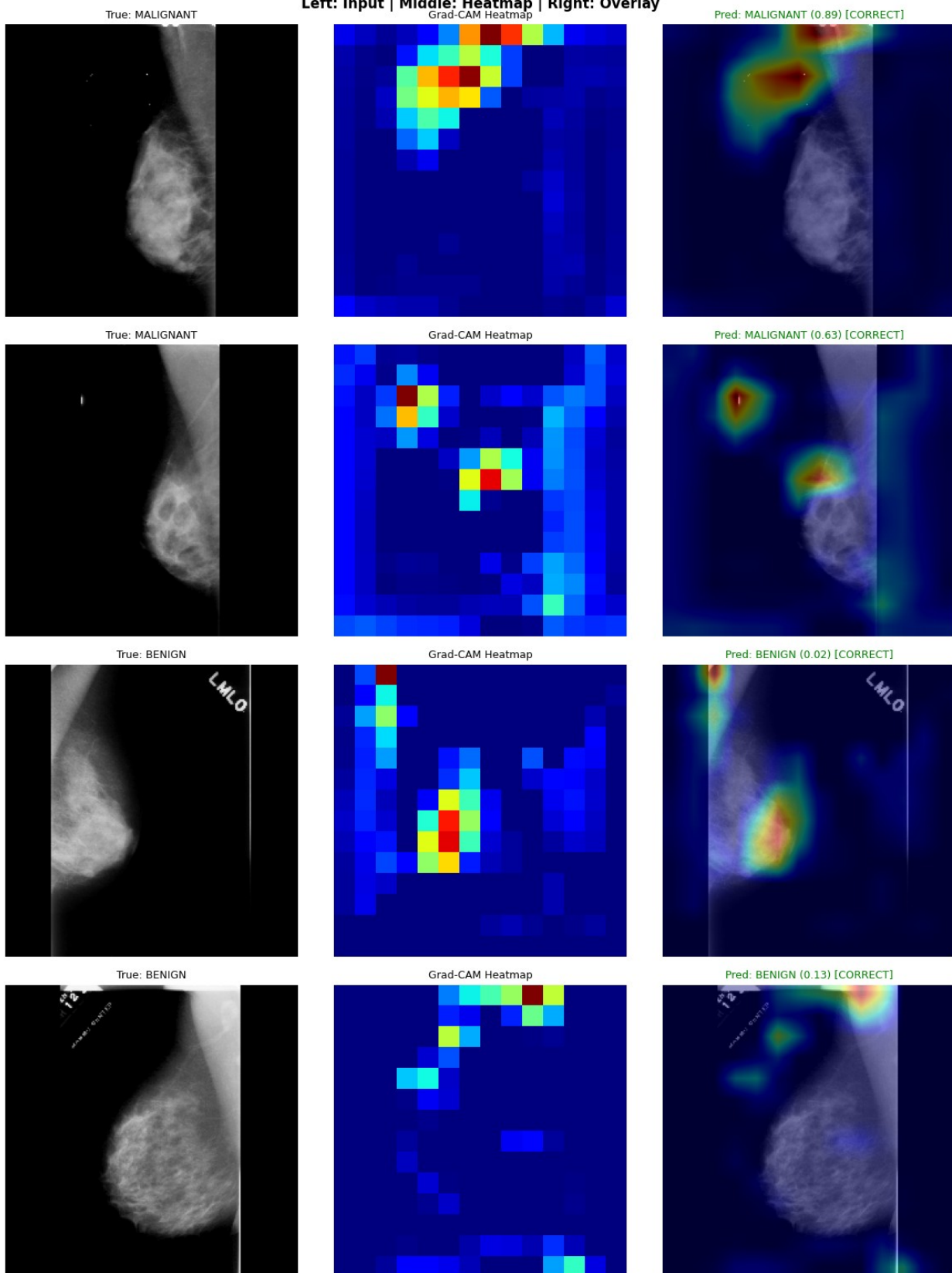
Grad-CAM visualisations are generated for VGG16 and ResNet50 to examine the image regions contributing to their predictions. This provides qualitative interpretability evidence alongside their validation performance.

```
# Grad-CAM uses the same VGG16 preprocessing as evaluation
plot_gradcam_grid()
```

```
vgg_model,  
"VGG16",  
IMG_SIZE_224,  
threshold=vgg_threshold,  
model_type="vgg16",  
save_path=f"{BASE}/gradcam_vgg16.png",  
)
```

Grad-CAM Visualisation — VGG16

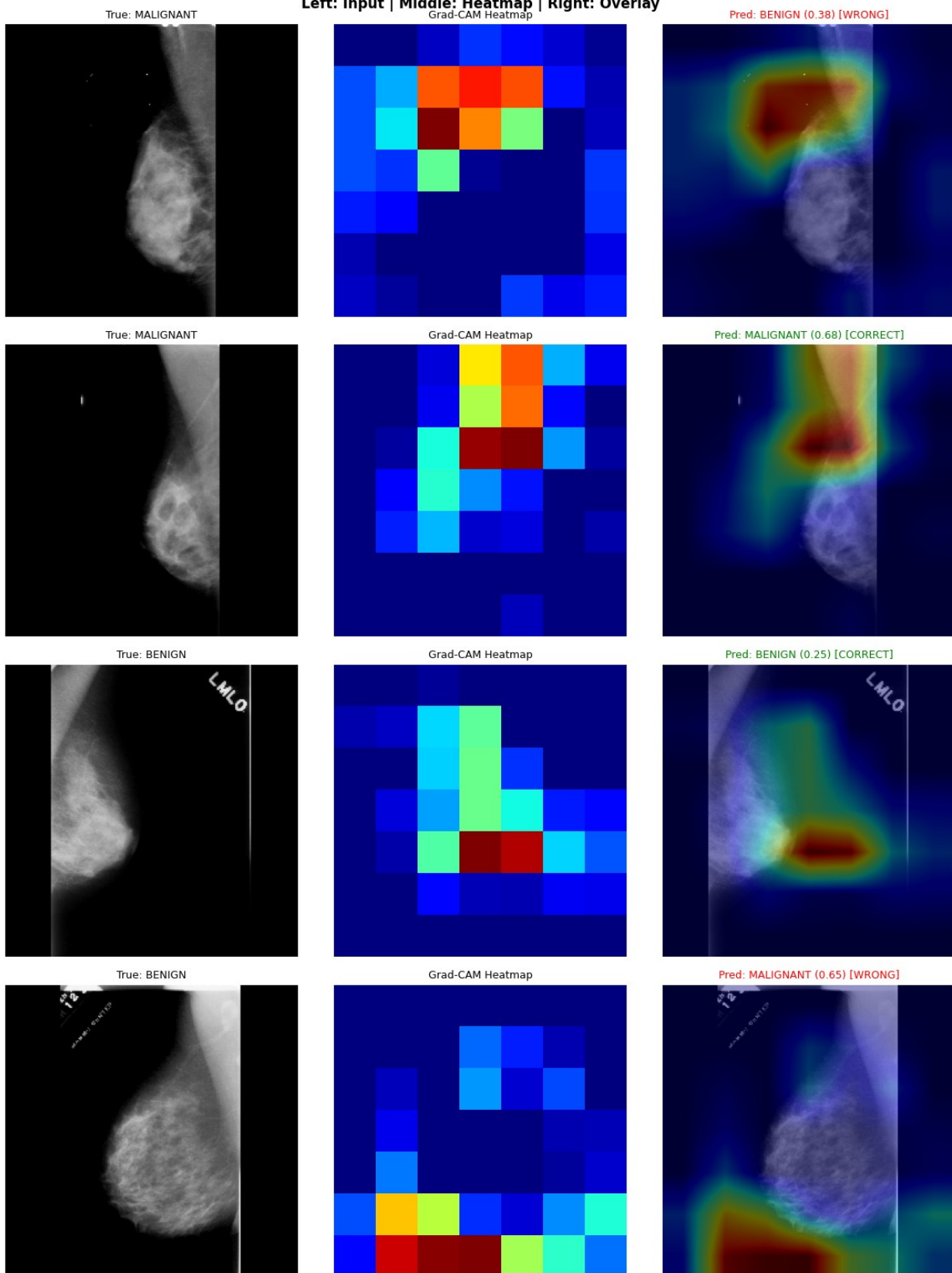
Left: Input | Middle: Heatmap | Right: Overlay



```
# Grad-CAM uses the same ResNet50 preprocessing as evaluation
plot_gradcam_grid(
    resnet_model,
    "ResNet50",
    IMG_SIZE_224,
    threshold=resnet_threshold,
    model_type="resnet50",
    save_path=f"{BASE}/gradcam_resnet50.png",
)
```

Grad-CAM Visualisation — ResNet50

Left: Input | Middle: Heatmap | Right: Overlay



Section 21: Pre-CLAHE Model Comparison

The models developed so far are compared using validation metrics before introducing CLAHE preprocessing. This provides a common reference for assessing the custom CNN and transfer-learning approaches.

```
# Principal model comparison (uses the baseline model, not the scaling one)
```

```
all_results = pd.DataFrame(
    [
        baseline_principal_results,
        reg128_results,
        reg224_results,
        vgg_results,
        resnet_results,
    ]
)[["model", "threshold", "accuracy", "sensitivity", "specificity",
  "precision", "auc"]]
```

```
all_results = all_results.sort_values("sensitivity",
ascending=False).reset_index(
    drop=True
)
```

```
display(all_results)
```

```
{
  "summary": {
    "name": "all_results",
    "rows": 5,
    "fields": [
      {
        "column": "model",
        "properties": {
          "dtype": "string",
          "num_unique_values": 5,
          "samples": [
            "Regularised Scaled CNN (128px)",
            "Baseline CNN (128px)",
            "Regularised Scaled CNN (224px)"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "threshold",
        "properties": {
          "dtype": "number",
          "std": 0.10076101428628038,
          "min": 0.3172,
          "max": 0.5567,
          "num_unique_values": 5,
          "samples": [
            0.5567,
            0.5308,
            0.469
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "accuracy",
        "properties": {
          "dtype": "number",
          "std": 0.03686688215729669,
          "min": 0.6627,
          "max": 0.754,
          "num_unique_values": 5,
          "samples": [
            0.6944,
            0.6667,
            0.6627
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "sensitivity",
        "properties": {
          "dtype": "number",
          "std": 0.10903984134251113,
          "min": 0.6341,
          "max": 0.9187,
          "num unique values": 5,
          "samples": [

```

```
0.8618,\n                0.6341,\n                0.8293\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"specificity\",\n            \"dtype\": \"number\",\n            \"std\": 0.11510856614518311,\n            \"min\": 0.5039,\n            \"max\": 0.7442,\n            \"num_unique_values\": 4,\n            \"samples\": [\n                0.5349,\n                0.6977,\n                0.5039\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"precision\",\n            \"dtype\": \"number\",\n            \"std\": 0.04866001438553012,\n            \"min\": 0.6145,\n            \"max\": 0.7402,\n            \"num_unique_values\": 5,\n            \"samples\": [\n                0.6386,\n                0.6667,\n                0.6145\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"auc\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0.058298481969945,\n                \"min\": 0.6856,\n                \"max\": 0.8254,\n                \"num_unique_values\": 5,\n                \"samples\": [\n                    0.6938,\n                    0.6856,\n                    0.7353,\n                    0.6856,\n                    0.6856\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            }\n        },\n        \"type\": \"dataframe\", \"variable name\": \"all results\"}
```

Confusion matrices for principal models

```
cms = [
    baseline_principal_results,
    reg128_results,
    reg224_results,
    vgg_results,
    resnet_results,
]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

for ax, res in zip(axes.flat, cms):
    ConfusionMatrixDisplay(res["cm"], display_labels=["Benign",
"Malignant"]).plot(
        ax=ax, colorbar=False, cmap="Blues", values_format="d"
    )

    ax.set_title(res["model"], fontsize=10, fontweight="bold")

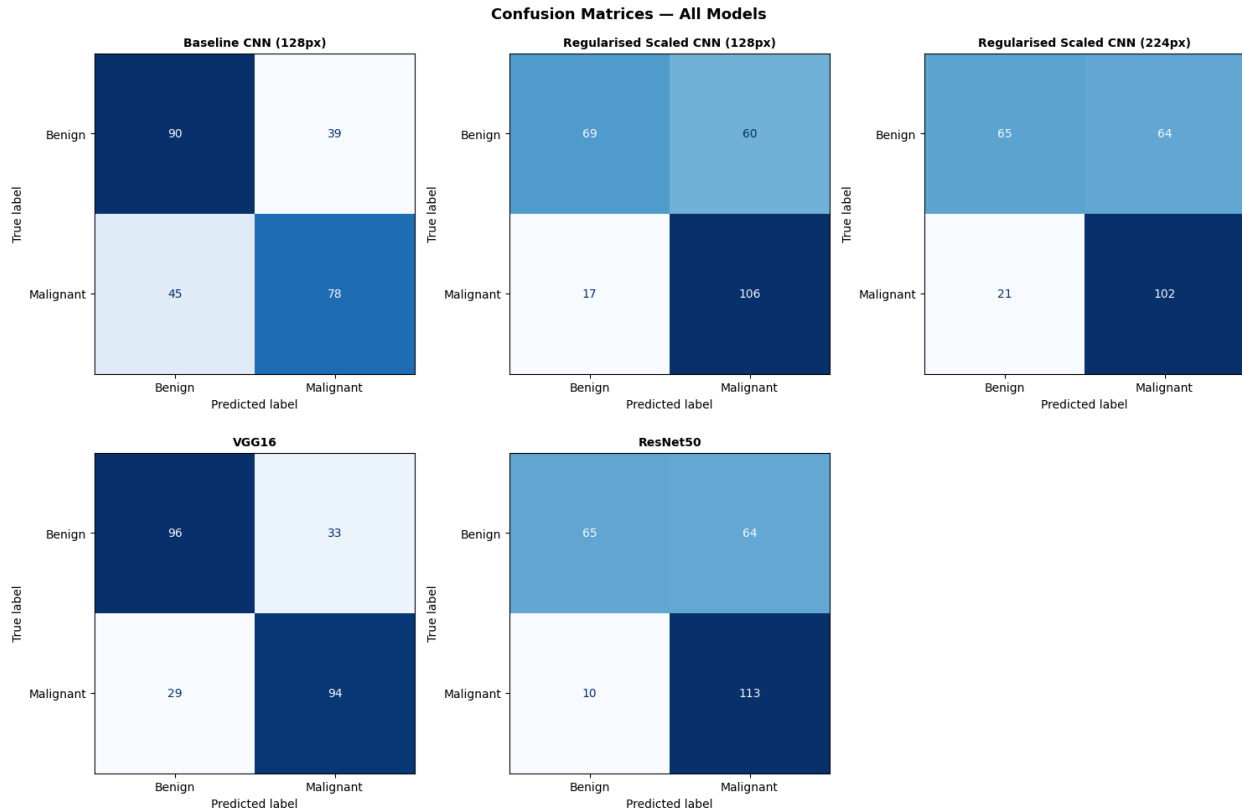
axes.flat[-1].axis("off")

plt.suptitle("Confusion Matrices - All Models", fontweight="bold",
fontsize=13)

plt.tight_layout()
```

```
plt.savefig(f"{BASE}/confusion_matrices_all.png", dpi=150,
bbox_inches="tight")

plt.show()
```



```
fig, ax = plt.subplots(figsize=(11, 5))

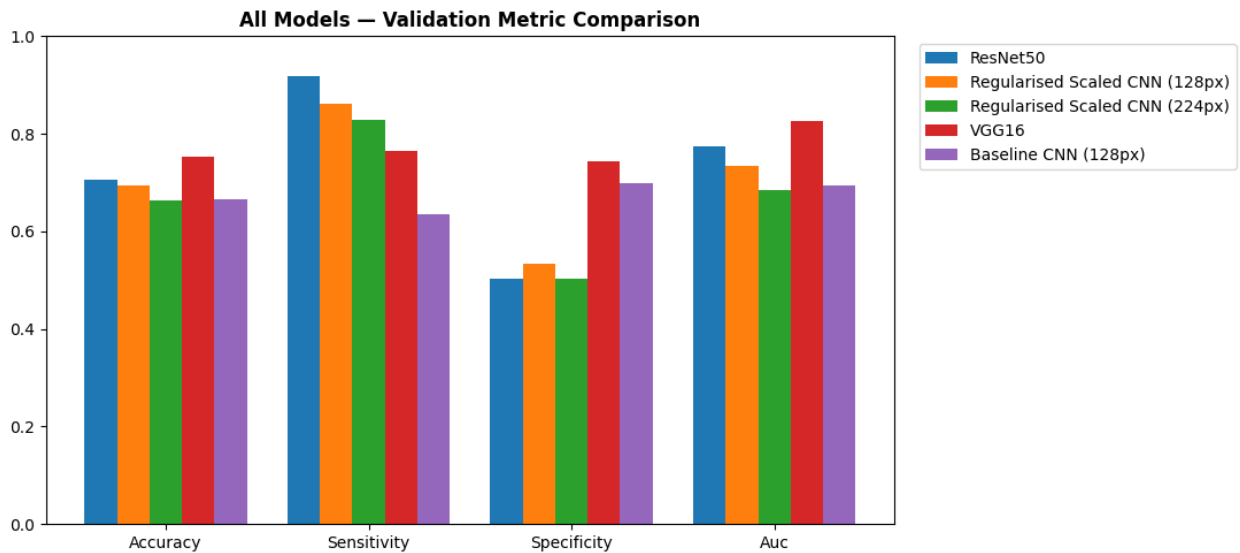
metrics_to_plot = ["accuracy", "sensitivity", "specificity", "auc"]
x = np.arange(len(metrics_to_plot))
width = 0.8 / len(all_results)

for i, row in all_results.iterrows():
    ax.bar(x + i * width, [row[m] for m in metrics_to_plot], width,
label=row["model"])

ax.set_xticks(x + width * (len(all_results) - 1) / 2)
ax.set_xticklabels([m.capitalize() for m in metrics_to_plot])
ax.set_ylim(0, 1)
ax.set_title("All Models - Validation Metric Comparison",
fontweight="bold")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")

plt.tight_layout()
plt.savefig(
f"{BASE}/comparison_all_models_validation.png", dpi=150,
```

```
bbox_inches="tight"
)
plt.show()
```



Section 22: CLAHE Preprocessing for ResNet50

Contrast Limited Adaptive Histogram Equalisation (**CLAHE**) is introduced as a dedicated preprocessing enhancement for ResNet50. CLAHE is applied after cropping to improve local contrast before resizing and ResNet50-specific ImageNet preprocessing.

The original ResNet50 pipeline is otherwise retained to support a controlled comparison.

```
# CLAHE pipeline with deduplicated caches

import cv2

def apply_clahe(image):
    def _clahe(img):
        img = img.numpy().astype(np.uint8)
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        gray = cv2.createCLAHE(2.0, (8, 8)).apply(gray)
        return cv2.cvtColor(gray,
            cv2.COLOR_GRAY2RGB).astype(np.float32)

    out = tf.py_function(_clahe, [image], tf.float32)
    out.set_shape([None, None, 3])
    return out

def preprocess_clahe(image, label, img_size):
    image = apply_clahe(crop_to_content(image))
```

```

        image = tf.image.resize_with_pad(image, img_size, img_size)
        return tf.cast(image, tf.float32) / 255.0, label

def build_pipeline_clahe(paths, labels, img_size):
    return (
        tf.data.Dataset.from_tensor_slices((paths, labels))
        .map(load_image, num_parallel_calls=AUTOTUNE)
        .map(lambda x, y: preprocess_clahe(x, y, img_size),
num_parallel_calls=AUTOTUNE)
        .batch(BATCH_SIZE)
        .prefetch(AUTOTUNE)
    )

def get_dataset_clahe(name, paths, labels, img_size, shuffle=False,
augmentation=False):

    cache = f"{name}_{img_size}_clahe_dedup_v1"

    if not os.path.exists(f"{BASE}/{cache}_images.npy"):
        print(f"[{name}@{img_size} CLAHE] building NEW deduplicated
cache...")
        cache_dataset(build_pipeline_clahe(paths, labels, img_size),
cache)
    else:
        print(f"[{name}@{img_size} CLAHE] reusing corrected cache")

    return load_cached(cache, shuffle, augmentation)

train_ds_224_clahe_plain = get_dataset_clahe(
    "train", X_train, y_train, IMG_SIZE_224, shuffle=True
)

train_ds_224_clahe = get_dataset_clahe(
    "train", X_train, y_train, IMG_SIZE_224, shuffle=True,
augmentation=True
)

val_ds_224_clahe = get_dataset_clahe("val", X_val, y_val,
IMG_SIZE_224)

test_ds_224_clahe = get_dataset_clahe("test", X_test, y_test,
IMG_SIZE_224)

print("\n✓ Corrected CLAHE pipelines ready")
print("✓ NEW CLAHE caches used for train, validation and test")

[train@224 CLAHE] reusing corrected cache
[train@224 CLAHE] reusing corrected cache

```

```

[val@224 CLAHE] reusing corrected cache
[test@224 CLAHE] reusing corrected cache

✓ Corrected CLAHE pipelines ready
✓ NEW CLAHE caches used for train, validation and test

sample_plain = np.load(f"{BASE}/train_224_dedup_v1_images.npy")[0]
sample_clahe = np.load(f"{BASE}/train_224_clahe_dedup_v1_images.npy")
[0]

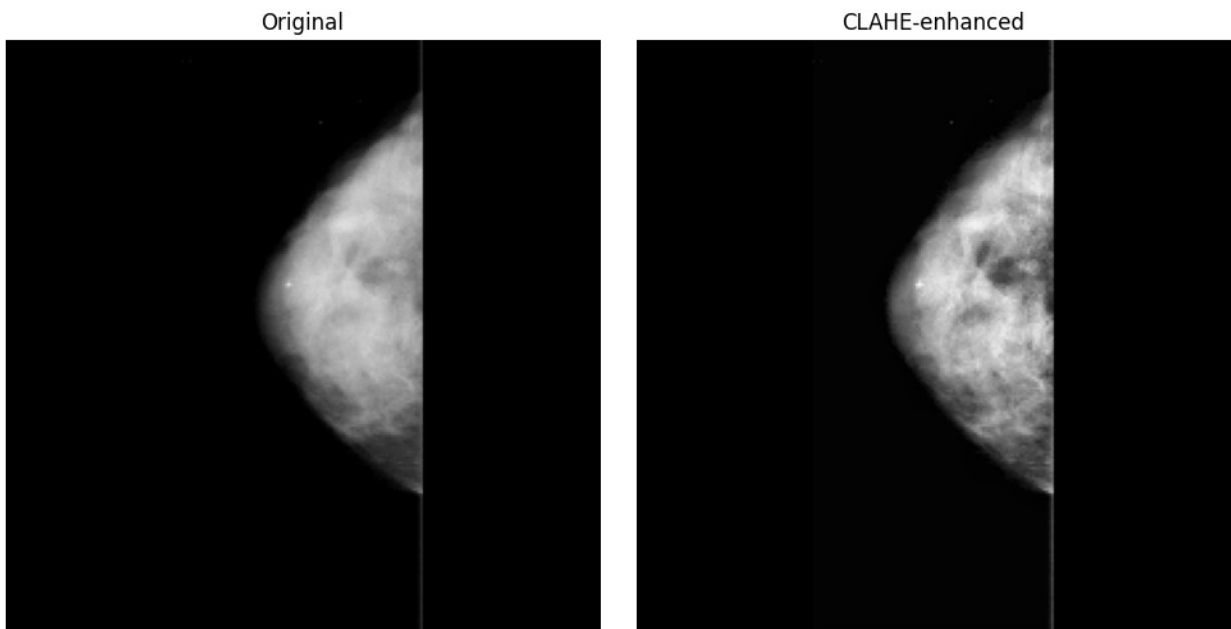
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(sample_plain)
axes[0].set_title("Original")
axes[0].axis("off")

axes[1].imshow(sample_clahe)
axes[1].set_title("CLAHE-enhanced")
axes[1].axis("off")

plt.tight_layout()
plt.show()

```



Section 23: ResNet50 + CLAHE - Training and Validation Evaluation

ResNet50 is retrained using CLAHE-enhanced mammograms with the same two-stage transfer-learning strategy. The best checkpoint is evaluated on the validation set using a validation-derived decision threshold.

Performance is then compared with plain ResNet50 to assess the effect of CLAHE preprocessing.

```
# ResNet50 preprocessing after CLAHE
resnet_clahe_train_ds = prepare_resnet_dataset(train_ds_224_clahe)
resnet_clahe_val_ds = prepare_resnet_dataset(val_ds_224_clahe)
resnet_clahe_test_ds = prepare_resnet_dataset(test_ds_224_clahe)

def run_transfer_learning_resnet_clahe(class_weight=None):
    # Same ResNet50 recipe, but with CLAHE inputs
    print(f"\n{'='*60}\nResNet50 + CLAHE - Stage A: frozen head (10
epochs)\n{'='*60}")

    model, base_model = build_transfer_model(
        ResNet50, IMG_SIZE_224, name="ResNet50_CLAHE"
    )

    head_history = model.fit(
        resnet_clahe_train_ds,
        validation_data=resnet_clahe_val_ds,
        epochs=10,
        verbose=1,
        class_weight=class_weight,
    )

    print(f"\n{'='*60}\nResNet50 + CLAHE - Stage B: fine-tuning
(lr=1e-5)\n{'='*60}")

    model = fine_tune(model, base_model)

    ft_callbacks = [
        EarlyStopping(
            monitor="val_auc",
            patience=7,
            restore_best_weights=True,
            mode="max",
            verbose=1,
        ),
        ModelCheckpoint(
            filepath=f"{MODELS_DIR}/resnet50_clahe_best.keras",
            monitor="val_auc",
```

```

        save_best_only=True,
        mode="max",
        verbose=1,
    ),
]

ft_history = model.fit(
    resnet_clahe_train_ds,
    validation_data=resnet_clahe_val_ds,
    epochs=20,
    callbacks=ft_callbacks,
    verbose=1,
    class_weight=class_weight,
)

combined_history = {
    k: head_history.history[k] + ft_history.history[k] for k in
head_history.history
}

model.save(f"{MODELS_DIR}/resnet50_clahe_final.keras")

return model, combined_history

resnet_clahe_model, resnet_clahe_history =
run_transfer_learning_resnet_clahe(
    class_weight=class_weights
)

```

```

=====
ResNet50 + CLAHE – Stage A: frozen head (10 epochs)
=====
Epoch 1/10
31/31 ————— 32s 573ms/step - accuracy: 0.5322 - auc:
0.5365 - loss: 0.8608 - precision: 0.5232 - sensitivity: 0.4948 -
val_accuracy: 0.4881 - val_auc: 0.5623 - val_loss: 0.7365 -
val_precision: 0.4881 - val_sensitivity: 1.0000
Epoch 2/10
31/31 ————— 4s 120ms/step - accuracy: 0.5209 - auc:
0.5325 - loss: 0.6949 - precision: 0.5102 - sensitivity: 0.5219 -
val_accuracy: 0.5119 - val_auc: 0.5715 - val_loss: 0.6894 -
val_precision: 0.5000 - val_sensitivity: 0.2520
Epoch 3/10
31/31 ————— 5s 121ms/step - accuracy: 0.5169 - auc:
0.5446 - loss: 0.6903 - precision: 0.5066 - sensitivity: 0.4823 -
val_accuracy: 0.5714 - val_auc: 0.5919 - val_loss: 0.6866 -
val_precision: 0.6829 - val_sensitivity: 0.2276
Epoch 4/10

```

31/31 ————— 4s 130ms/step - accuracy: 0.5403 - auc:
0.5619 - loss: 0.6891 - precision: 0.5509 - sensitivity: 0.3278 -
val_accuracy: 0.5754 - val_auc: 0.6398 - val_loss: 0.6795 -
val_precision: 0.5784 - val_sensitivity: 0.4797

Epoch 5/10

31/31 ————— 4s 130ms/step - accuracy: 0.5628 - auc:
0.5888 - loss: 0.6814 - precision: 0.5594 - sensitivity: 0.5010 -
val_accuracy: 0.5913 - val_auc: 0.6390 - val_loss: 0.6814 -
val_precision: 0.5515 - val_sensitivity: 0.8699

Epoch 6/10

31/31 ————— 4s 124ms/step - accuracy: 0.5761 - auc:
0.6103 - loss: 0.6748 - precision: 0.5613 - sensitivity: 0.6117 -
val_accuracy: 0.5556 - val_auc: 0.6303 - val_loss: 0.6710 -
val_precision: 0.5902 - val_sensitivity: 0.2927

Epoch 7/10

31/31 ————— 4s 124ms/step - accuracy: 0.5659 - auc:
0.6068 - loss: 0.6715 - precision: 0.5597 - sensitivity: 0.5282 -
val_accuracy: 0.6071 - val_auc: 0.6606 - val_loss: 0.6659 -
val_precision: 0.5968 - val_sensitivity: 0.6016

Epoch 8/10

31/31 ————— 5s 146ms/step - accuracy: 0.5659 - auc:
0.6072 - loss: 0.6734 - precision: 0.5587 - sensitivity: 0.5365 -
val_accuracy: 0.6230 - val_auc: 0.6641 - val_loss: 0.6589 -
val_precision: 0.6111 - val_sensitivity: 0.6260

Epoch 9/10

31/31 ————— 4s 127ms/step - accuracy: 0.5863 - auc:
0.6066 - loss: 0.6784 - precision: 0.5717 - sensitivity: 0.6159 -
val_accuracy: 0.6270 - val_auc: 0.6606 - val_loss: 0.6673 -
val_precision: 0.6090 - val_sensitivity: 0.6585

Epoch 10/10

31/31 ————— 4s 126ms/step - accuracy: 0.5669 - auc:
0.6109 - loss: 0.6732 - precision: 0.5530 - sensitivity: 0.5992 -
val_accuracy: 0.6230 - val_auc: 0.6781 - val_loss: 0.6607 -
val_precision: 0.6148 - val_sensitivity: 0.6098

=====
ResNet50 + CLAHE – Stage B: fine-tuning (lr=1e-5)
=====

Unfroze top 53 of 175 base layers.

Epoch 1/20

31/31 ————— 0s 484ms/step - accuracy: 0.5959 - auc:
0.6540 - loss: 0.6627 - precision: 0.5495 - sensitivity: 0.8016

Epoch 1: val_auc improved from None to 0.65699, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 84s 2s/step - accuracy: 0.5955 - auc:
0.6649 - loss: 0.6560 - precision: 0.5628 - sensitivity: 0.7766 -
val_accuracy: 0.6190 - val_auc: 0.6570 - val_loss: 0.6598 -
val_precision: 0.6392 - val_sensitivity: 0.5041

Epoch 2/20

31/31 ————— 0s 152ms/step - accuracy: 0.6222 - auc:
0.6955 - loss: 0.6433 - precision: 0.5851 - sensitivity: 0.7428

Epoch 2: val_auc improved from 0.65699 to 0.66676, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 2: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 41s 1s/step - accuracy: 0.6445 - auc:
0.7154 - loss: 0.6345 - precision: 0.6076 - sensitivity: 0.7724 -
val_accuracy: 0.6111 - val_auc: 0.6668 - val_loss: 0.6510 -
val_precision: 0.6344 - val_sensitivity: 0.4797

Epoch 3/20

31/31 ————— 0s 152ms/step - accuracy: 0.6564 - auc:
0.7304 - loss: 0.6224 - precision: 0.6174 - sensitivity: 0.7783

Epoch 3: val_auc improved from 0.66676 to 0.67792, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 3: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 41s 1s/step - accuracy: 0.6486 - auc:
0.7205 - loss: 0.6208 - precision: 0.6138 - sensitivity: 0.7599 -
val_accuracy: 0.5913 - val_auc: 0.6779 - val_loss: 0.6399 -
val_precision: 0.6042 - val_sensitivity: 0.4715

Epoch 4/20

31/31 ————— 0s 159ms/step - accuracy: 0.6872 - auc:
0.7697 - loss: 0.5966 - precision: 0.6254 - sensitivity: 0.8080

Epoch 4: val_auc improved from 0.67792 to 0.69632, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 4: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 57s 2s/step - accuracy: 0.6823 - auc:
0.7548 - loss: 0.6018 - precision: 0.6429 - sensitivity: 0.7891 -
val_accuracy: 0.6508 - val_auc: 0.6963 - val_loss: 0.6275 -
val_precision: 0.6471 - val_sensitivity: 0.6260

Epoch 5/20

31/31 ————— 0s 153ms/step - accuracy: 0.6760 - auc:
0.7676 - loss: 0.5922 - precision: 0.6550 - sensitivity: 0.7535

Epoch 5: val_auc improved from 0.69632 to 0.71138, saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 56s 2s/step - accuracy: 0.6854 - auc:
0.7811 - loss: 0.5833 - precision: 0.6477 - sensitivity: 0.7829 -
val_accuracy: 0.6786 - val_auc: 0.7114 - val_loss: 0.6174 -
val_precision: 0.6479 - val_sensitivity: 0.7480

Epoch 6/20

31/31 ————— 0s 156ms/step - accuracy: 0.6772 - auc:
0.7721 - loss: 0.5779 - precision: 0.6273 - sensitivity: 0.7974

Epoch 6: val_auc improved from 0.71138 to 0.72351, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 53s 2s/step - accuracy: 0.6966 - auc:
0.7946 - loss: 0.5625 - precision: 0.6558 - sensitivity: 0.7996 -
val_accuracy: 0.6905 - val_auc: 0.7235 - val_loss: 0.6103 -
val_precision: 0.6471 - val_sensitivity: 0.8049

Epoch 7/20

31/31 ————— 0s 162ms/step - accuracy: 0.7228 - auc:
0.8039 - loss: 0.5510 - precision: 0.6746 - sensitivity: 0.7998

Epoch 7: val_auc improved from 0.72351 to 0.73032, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 55s 2s/step - accuracy: 0.7222 - auc:
0.8137 - loss: 0.5433 - precision: 0.6838 - sensitivity: 0.8038 -
val_accuracy: 0.6905 - val_auc: 0.7303 - val_loss: 0.6104 -
val_precision: 0.6433 - val_sensitivity: 0.8211

Epoch 8/20

31/31 ————— 0s 152ms/step - accuracy: 0.7350 - auc:
0.8381 - loss: 0.5230 - precision: 0.6880 - sensitivity: 0.8396

Epoch 8: val_auc improved from 0.73032 to 0.73558, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 8: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 55s 2s/step - accuracy: 0.7344 - auc:
0.8377 - loss: 0.5189 - precision: 0.6918 - sensitivity: 0.8246 -

val_accuracy: 0.6944 - val_auc: 0.7356 - val_loss: 0.6067 -
val_precision: 0.6513 - val_sensitivity: 0.8049
Epoch 9/20
31/31 ————— 0s 158ms/step - accuracy: 0.7580 - auc:
0.8347 - loss: 0.5179 - precision: 0.7351 - sensitivity: 0.8197
Epoch 9: val_auc improved from 0.73558 to 0.73977, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 9: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 55s 2s/step - accuracy: 0.7692 - auc:
0.8514 - loss: 0.5078 - precision: 0.7338 - sensitivity: 0.8288 -
val_accuracy: 0.6865 - val_auc: 0.7398 - val_loss: 0.6135 -
val_precision: 0.6392 - val_sensitivity: 0.8211

Epoch 10/20
31/31 ————— 0s 157ms/step - accuracy: 0.7697 - auc:
0.8689 - loss: 0.4888 - precision: 0.7289 - sensitivity: 0.8466

Epoch 10: val_auc did not improve from 0.73977

31/31 ————— 6s 188ms/step - accuracy: 0.7855 - auc:
0.8771 - loss: 0.4801 - precision: 0.7523 - sensitivity: 0.8372 -
val_accuracy: 0.6984 - val_auc: 0.7371 - val_loss: 0.6402 -
val_precision: 0.6424 - val_sensitivity: 0.8618

Epoch 11/20

31/31 ————— 0s 156ms/step - accuracy: 0.8162 - auc:
0.8883 - loss: 0.4572 - precision: 0.8029 - sensitivity: 0.8350

Epoch 11: val_auc improved from 0.73977 to 0.74100, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 11: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 48s 2s/step - accuracy: 0.8192 - auc:
0.9001 - loss: 0.4421 - precision: 0.7938 - sensitivity: 0.8518 -
val_accuracy: 0.7024 - val_auc: 0.7410 - val_loss: 0.6636 -
val_precision: 0.6463 - val_sensitivity: 0.8618

Epoch 12/20

31/31 ————— 0s 156ms/step - accuracy: 0.8475 - auc:
0.9316 - loss: 0.3957 - precision: 0.8182 - sensitivity: 0.8949

Epoch 12: val_auc improved from 0.74100 to 0.75039, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 12: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 50s 2s/step - accuracy: 0.8315 - auc:
0.9212 - loss: 0.4053 - precision: 0.8008 - sensitivity: 0.8727 -

val_accuracy: 0.7063 - val_auc: 0.7504 - val_loss: 0.6513 -
val_precision: 0.6522 - val_sensitivity: 0.8537
Epoch 13/20
31/31 ————— 0s 155ms/step - accuracy: 0.8641 - auc:
0.9325 - loss: 0.3869 - precision: 0.8502 - sensitivity: 0.8780
Epoch 13: val_auc improved from 0.75039 to 0.75251, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 13: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 52s 2s/step - accuracy: 0.8580 - auc:
0.9290 - loss: 0.3845 - precision: 0.8400 - sensitivity: 0.8768 -
val_accuracy: 0.7103 - val_auc: 0.7525 - val_loss: 0.6827 -
val_precision: 0.6582 - val_sensitivity: 0.8455

Epoch 14/20

31/31 ————— 0s 158ms/step - accuracy: 0.8650 - auc:
0.9334 - loss: 0.3682 - precision: 0.8562 - sensitivity: 0.8712

Epoch 14: val_auc did not improve from 0.75251

31/31 ————— 7s 204ms/step - accuracy: 0.8652 - auc:
0.9312 - loss: 0.3703 - precision: 0.8577 - sensitivity: 0.8685 -
val_accuracy: 0.6865 - val_auc: 0.7466 - val_loss: 0.6929 -
val_precision: 0.6486 - val_sensitivity: 0.7805

Epoch 15/20

31/31 ————— 0s 171ms/step - accuracy: 0.8949 - auc:
0.9592 - loss: 0.3181 - precision: 0.8808 - sensitivity: 0.9075

Epoch 15: val_auc did not improve from 0.75251

31/31 ————— 6s 201ms/step - accuracy: 0.8876 - auc:
0.9529 - loss: 0.3319 - precision: 0.8836 - sensitivity: 0.8873 -
val_accuracy: 0.6865 - val_auc: 0.7499 - val_loss: 0.7168 -
val_precision: 0.6392 - val_sensitivity: 0.8211

Epoch 16/20

31/31 ————— 0s 158ms/step - accuracy: 0.8880 - auc:
0.9643 - loss: 0.3032 - precision: 0.8609 - sensitivity: 0.9073

Epoch 16: val_auc improved from 0.75251 to 0.75726, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

Epoch 16: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_best.keras

31/31 ————— 52s 2s/step - accuracy: 0.8907 - auc:
0.9595 - loss: 0.3064 - precision: 0.8843 - sensitivity: 0.8935 -
val_accuracy: 0.7103 - val_auc: 0.7573 - val_loss: 0.6947 -
val_precision: 0.6712 - val_sensitivity: 0.7967

Epoch 17/20

31/31 ————— 0s 153ms/step - accuracy: 0.9150 - auc:
0.9714 - loss: 0.2695 - precision: 0.9256 - sensitivity: 0.9083

```

Epoch 17: val_auc did not improve from 0.75726
31/31 ━━━━━━━━━━━ 6s 189ms/step - accuracy: 0.9091 - auc:
0.9650 - loss: 0.2787 - precision: 0.9029 - sensitivity: 0.9123 -
val_accuracy: 0.7341 - val_auc: 0.7555 - val_loss: 0.7052 -
val_precision: 0.7000 - val_sensitivity: 0.7967
Epoch 18/20
31/31 ━━━━━━━━━━━ 0s 163ms/step - accuracy: 0.9428 - auc:
0.9732 - loss: 0.2517 - precision: 0.9624 - sensitivity: 0.9246
Epoch 18: val_auc did not improve from 0.75726
31/31 ━━━━━━━━━━━ 6s 193ms/step - accuracy: 0.9336 - auc:
0.9733 - loss: 0.2524 - precision: 0.9312 - sensitivity: 0.9332 -
val_accuracy: 0.6984 - val_auc: 0.7516 - val_loss: 0.7329 -
val_precision: 0.6643 - val_sensitivity: 0.7724
Epoch 19/20
31/31 ━━━━━━━━━━━ 0s 156ms/step - accuracy: 0.9399 - auc:
0.9755 - loss: 0.2416 - precision: 0.9523 - sensitivity: 0.9282
Epoch 19: val_auc did not improve from 0.75726
31/31 ━━━━━━━━━━━ 6s 186ms/step - accuracy: 0.9397 - auc:
0.9774 - loss: 0.2347 - precision: 0.9412 - sensitivity: 0.9353 -
val_accuracy: 0.6825 - val_auc: 0.7569 - val_loss: 0.7490 -
val_precision: 0.6503 - val_sensitivity: 0.7561
Epoch 20/20
31/31 ━━━━━━━━━━━ 0s 163ms/step - accuracy: 0.9276 - auc:
0.9721 - loss: 0.2355 - precision: 0.9384 - sensitivity: 0.9085
Epoch 20: val_auc did not improve from 0.75726
31/31 ━━━━━━━━━━━ 11s 197ms/step - accuracy: 0.9397 - auc:
0.9756 - loss: 0.2285 - precision: 0.9506 - sensitivity: 0.9248 -
val_accuracy: 0.6627 - val_auc: 0.7508 - val_loss: 0.7265 -
val_precision: 0.6397 - val_sensitivity: 0.7073
Restoring model weights from the end of the best epoch: 16.

```

ResNet50 vs ResNet50 + CLAHE - Validation Comparison

Plain ResNet50 and ResNet50 + CLAHE are compared using validation performance. ROC-AUC measures overall discrimination, while sensitivity, specificity, precision and accuracy describe performance at the selected operating thresholds.

The comparison is interpreted as a trade-off across metrics, with particular attention to malignant sensitivity and false-negative cases.

```

# ResNet50 + CLAHE validation results

# Load saved best checkpoint
resnet_clahe_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_clahe_best.keras", compile=False
)

# Threshold selected ONLY on validation
resnet_clahe_threshold = find_best_threshold(resnet_clahe_model,

```

```

resnet_clahe_val_ds)

# Evaluate ONLY on validation
resnet_clahe_results = evaluate_model(
    resnet_clahe_model,
    resnet_clahe_val_ds,
    name="ResNet50 + CLAHE",
    threshold=resnet_clahe_threshold,
    split_name="Validation",
)

# Validation confusion matrix
plot_confusion_matrix(
    resnet_clahe_results["cm"],
    "ResNet50 + CLAHE - Validation Confusion Matrix",
    save_path=f"{BASE}/cm_resnet50_clahe_validation.png",
)

# Validation ROC curve
plot_roc_curve(
    resnet_clahe_results,
    title="ResNet50 + CLAHE - Validation ROC Curve",
    save_path=f"{BASE}/roc_resnet50_clahe_validation.png",
)

# Compare ResNet50 vs ResNet50 + CLAHE on VALIDATION only
resnet_clahe_comparison = pd.DataFrame([resnet_results,
    resnet_clahe_results])[
    ["model", "threshold", "accuracy", "sensitivity", "specificity",
    "precision", "auc"]
]

display(resnet_clahe_comparison)

Optimal threshold (Youden's J): 0.391 (val sensitivity=0.902, val
specificity=0.535)

```

```

=====
ResNet50 + CLAHE – Validation Results (threshold=0.391)
=====

```

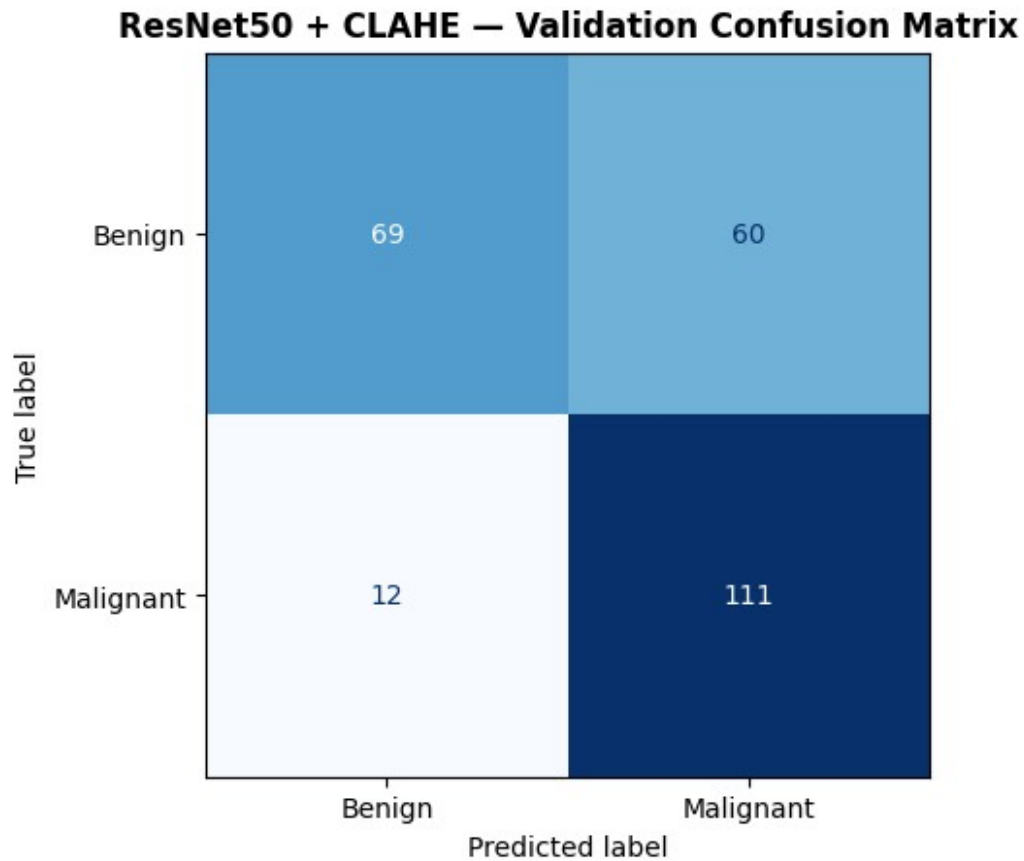
	precision	recall	f1-score	support
Benign	0.85	0.53	0.66	129
Malignant	0.65	0.90	0.76	123
accuracy			0.71	252
macro avg	0.75	0.72	0.71	252
weighted avg	0.75	0.71	0.70	252

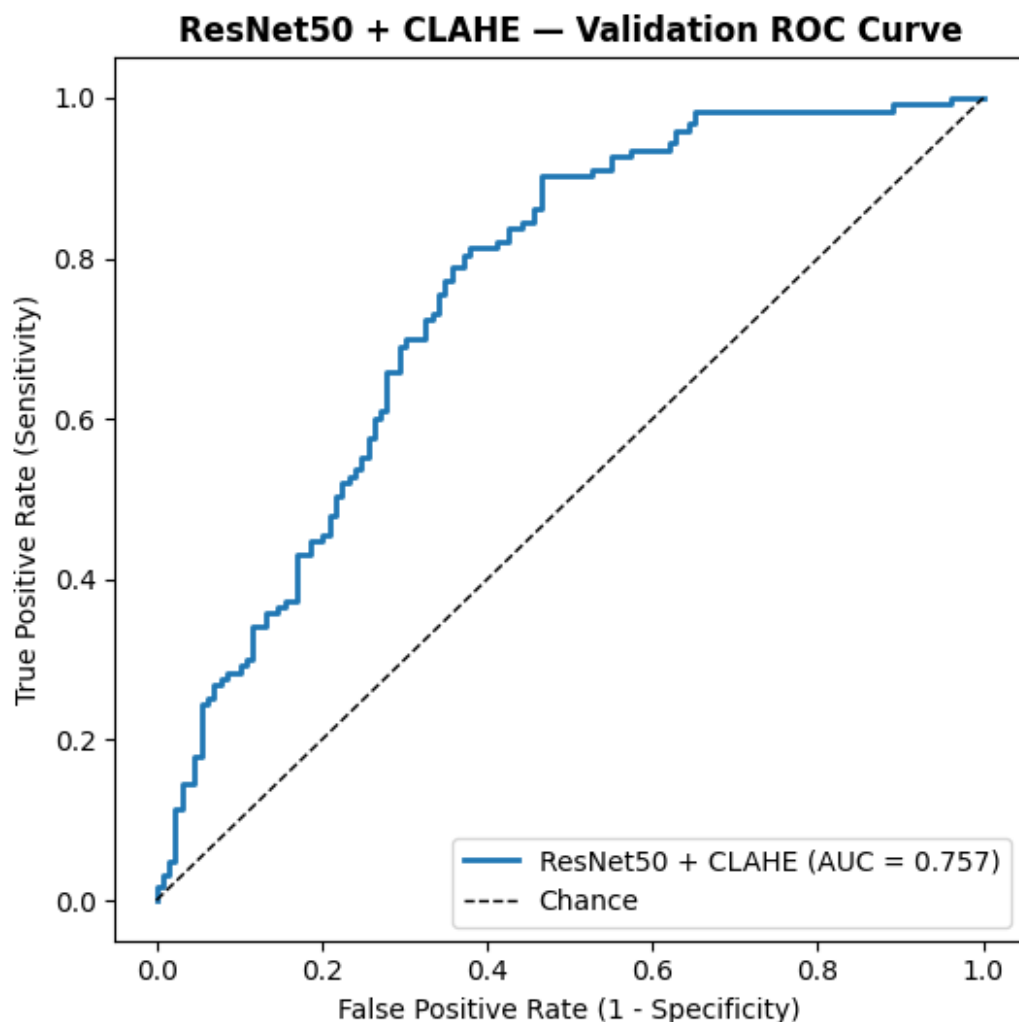
```

Accuracy      : 0.7143

```

Sensitivity : 0.9024 (missed 12 of 123 malignant cases)
Specificity : 0.5349
Precision : 0.6491
AUC : 0.7574





```
{
  "summary": {
    "name": "resnet_clahe_comparison",
    "rows": 2,
    "fields": [
      {
        "column": "model",
        "dtype": "string",
        "num_unique_values": 2,
        "samples": [
          "ResNet50 + CLAHE",
          "ResNet50"
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "threshold",
        "dtype": "number",
        "std": 0.006576093065034874,
        "min": 0.3814,
        "max": 0.3907,
        "num_unique_values": 2,
        "samples": [
          0.3814,
          0.3907
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "accuracy",
        "dtype": "number",
        "std": 0.005656854249492385,
        "min": 0.7063,
        "max": 0.7143,
        "num_unique_values": 2,
        "samples": [
          0.7143,
          0.7063
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "sensitivity",
        "dtype": "number",
        "std": 0.006576093065034874,
        "min": 0.3814,
        "max": 0.3907,
        "num_unique_values": 2,
        "samples": [
          0.3814,
          0.3907
        ],
        "semantic_type": "",
        "description": ""
      }
    ]
  }
}
```

```

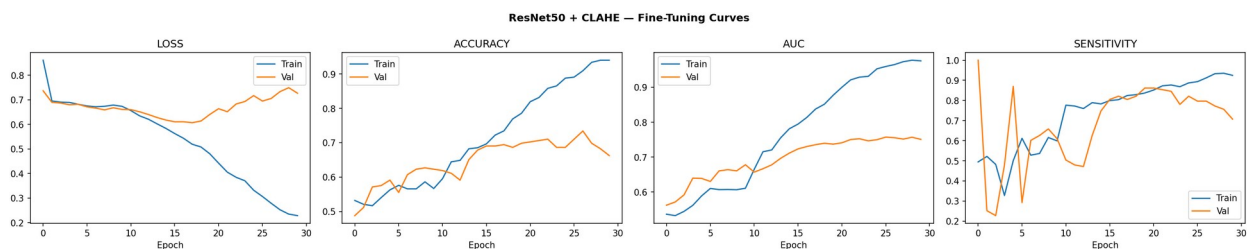
{"properties": {"dtype": "number", "std": 0.011525840533340711, "min": 0.9024, "max": 0.9187, "num_unique_values": 2, "samples": [0.9024, 0.9187], "semantic_type": ""}, {"description": "specificity", "column": "specificity", "properties": {"dtype": "number", "std": 0.021920310216782993, "min": 0.5039, "max": 0.5349, "num_unique_values": 2, "samples": [0.5039, 0.5349], "semantic_type": ""}, {"description": "precision", "column": "precision", "properties": {"dtype": "number", "std": 0.007566042558696089, "min": 0.6384, "max": 0.6491, "num_unique_values": 2, "samples": [0.6384, 0.6491], "semantic_type": ""}, {"description": "auc", "column": "auc", "properties": {"dtype": "number", "std": 0.012515790027001926, "min": 0.7574, "max": 0.7751, "num_unique_values": 2, "samples": [0.7574, 0.7751], "semantic_type": ""}}], "type": "dataframe", "variable_name": "resnet_clahe_comparison"}

```

Original training curves

```
from IPython.display import Image, display
```

```
display(Image(filename=f"{BASE}/curves_resnet50_clahe.png"))
```



Section 24: Grad-CAM - ResNet50 + CLAHE

Grad-CAM is used to inspect regions contributing to predictions from ResNet50 + CLAHE. Images follow the same **crop** → **CLAHE** → **resize** → **ResNet50 preprocessing** pipeline used during model development.

Heatmaps are restricted to the breast region to reduce activation from black or padded background areas. These visualisations provide qualitative interpretability evidence rather than tumour segmentation.

```

# ResNet50 + CLAHE Grad-CAM (no training needed)

import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications.resnet50 import preprocess_input

# Model + validation-selected threshold
model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_clahe_best.keras", compile=False
)
threshold = find_best_threshold(model, resnet_clahe_val_ds,
    verbose=False)
print(f"CLAHE validation threshold: {threshold:.4f}")

# Build Grad-CAM model once
layer = model.get_layer("conv4_block6_out")
grad_model = tf.keras.Model(model.input, [layer.output, model.output])

def breast_mask(img):
    binary = (img.mean(axis=-1) > 0.03).astype(np.uint8)
    n, labels, stats, _ = cv2.connectedComponentsWithStats(binary, 8)

    if n <= 1:
        return binary.astype(np.float32)

    largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
    mask = (labels == largest).astype(np.float32)
    return np.clip(cv2.GaussianBlur(mask, (7, 7), 0), 0, 1)

def prepare_image(path):
    img = tf.image.decode_jpeg(tf.io.read_file(path), channels=3)
    img = crop_to_content(img).numpy().astype(np.uint8)

    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    gray = cv2.createCLAHE(2.0, (8, 8)).apply(gray)
    rgb = cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)

    resized = tf.image.resize_with_pad(
        tf.cast(rgb, tf.float32), IMG_SIZE_224, IMG_SIZE_224
    )

    display = resized.numpy() / 255.0
    x = preprocess_input(resized)[None, ...]

    return display, x

```

```

def gradcam(x, display):
    with tf.GradientTape() as tape:
        conv, pred = grad_model(x, training=False)
        p = pred[:, 0]
        loss = tf.where(p >= threshold, p, 1.0 - p)

        grads = tape.gradient(loss, conv)
        weights = tf.reduce_mean(grads, axis=(0, 1, 2))
        heat = tf.squeeze(conv[0] @ weights[..., None])
        heat = tf.maximum(heat, 0)
        heat /= tf.reduce_max(heat) + 1e-8

        heat = cv2.resize(
            heat.numpy(), (IMG_SIZE_224, IMG_SIZE_224),
            interpolation=cv2.INTER_CUBIC
        )
        heat = np.clip(heat, 0, 1) * breast_mask(display)

    return heat, float(pred.numpy()[0, 0])

fig, axes = plt.subplots(4, 3, figsize=(12, 16))

for i, (path, true) in enumerate(zip(GRADCAM_SAMPLES,
GRADCAM_LABELS)):
    img, x = prepare_image(path)
    heat, prob = gradcam(x, img)

    pred = "MALIGNANT" if prob >= threshold else "BENIGN"
    correct = pred == true

    colour = plt.get_cmap("jet")(heat)[..., :3]
    alpha = heat[..., None] * 0.42
    overlay = img * (1 - alpha) + colour * alpha
    overlay[img.mean(axis=-1) <= 0.03] = img[img.mean(axis=-1) <=
0.03]

    axes[i, 0].imshow(img)
    axes[i, 0].set_title(f"True: {true}")

    axes[i, 1].imshow(heat, cmap="jet", vmin=0, vmax=1)
    axes[i, 1].set_title("Grad-CAM Heatmap")

    axes[i, 2].imshow(np.clip(overlay, 0, 1))
    axes[i, 2].set_title(
        f'Pred: {pred} ({prob:.2f}) [{"CORRECT" if correct else
"WRONG"}]',
        color="green" if correct else "red",
    )

    for ax in axes[i]:

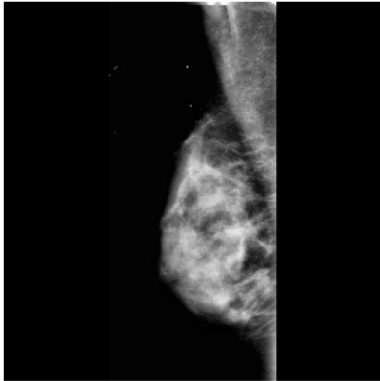
```

```
ax.axis("off")

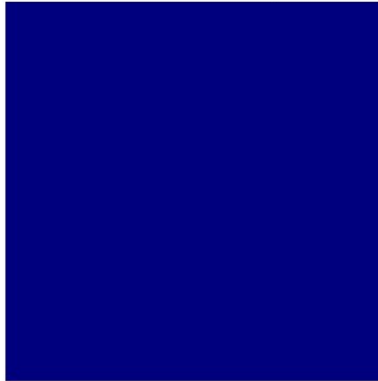
plt.suptitle("Grad-CAM - ResNet50 + CLAHE", fontweight="bold")
plt.tight_layout()
plt.savefig(f"{BASE}/gradcam_resnet50_clahe_clean.png", dpi=150,
bbox_inches="tight")
plt.show()

Optimal threshold (Youden's J): 0.391 (val sensitivity=0.902, val
specificity=0.535)
CLAHE validation threshold: 0.3907
```

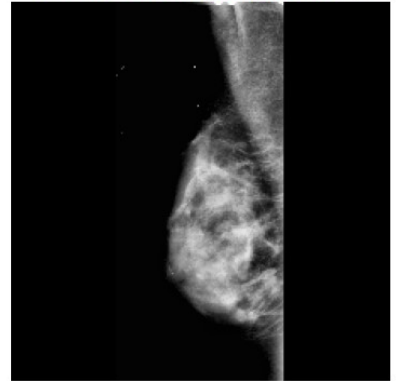
True: MALIGNANT



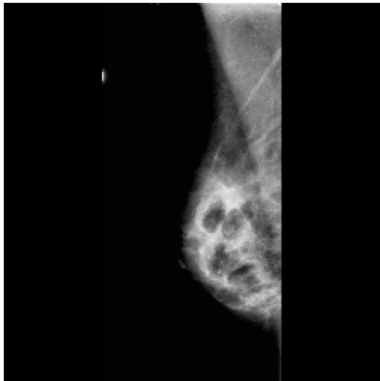
Grad-CAM — ResNet50 + CLAHE
Grad-CAM Heatmap



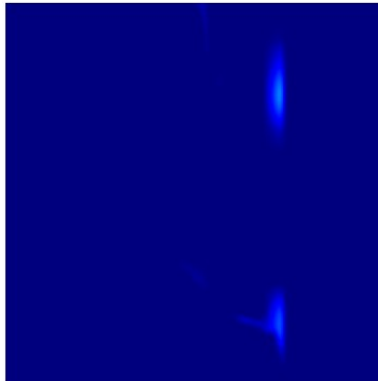
Pred: MALIGNANT (0.51) [CORRECT]



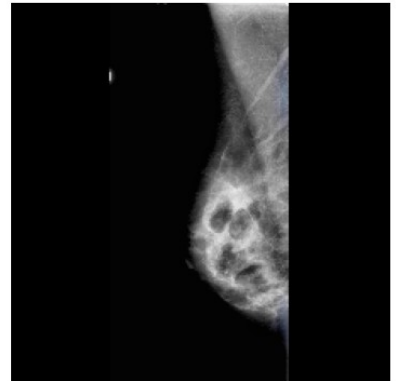
True: MALIGNANT



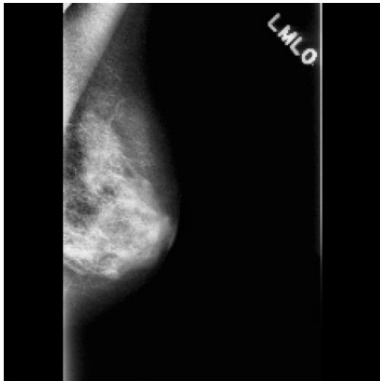
Grad-CAM Heatmap



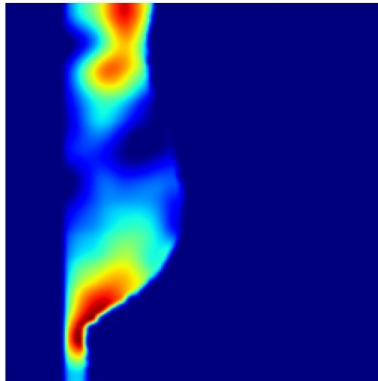
Pred: MALIGNANT (0.53) [CORRECT]



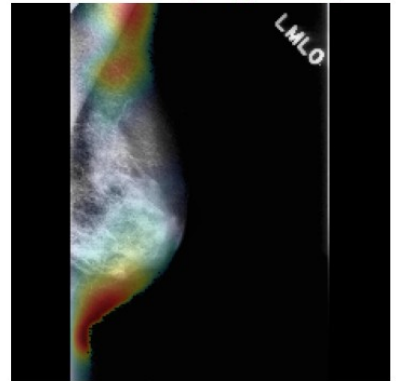
True: BENIGN



Grad-CAM Heatmap



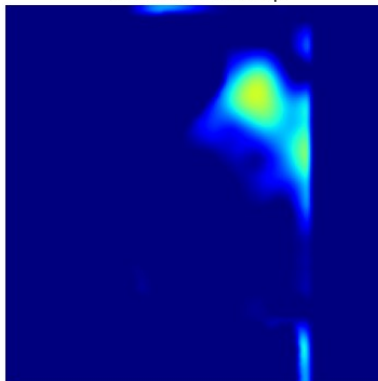
Pred: BENIGN (0.06) [CORRECT]



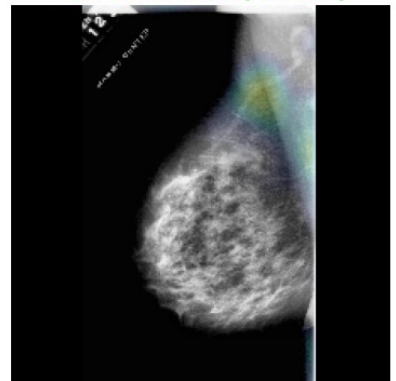
True: BENIGN



Grad-CAM Heatmap



Pred: BENIGN (0.32) [CORRECT]



Section 25: Principal Model Comparison and Final Model Selection

The six principal models are compared using validation-set accuracy, sensitivity, specificity, precision and ROC-AUC. This comparison provides the evidence used for final-model selection.

Although plain ResNet50 achieved the highest validation sensitivity, **ResNet50 + CLAHE** retained sensitivity above 90% while improving accuracy, specificity and precision. It is therefore selected as the final balanced high-sensitivity configuration for held-out evaluation.

```
# Master model comparison on validation set

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

results = [
    baseline_principal_results,
    reg128_results,
    reg224_results,
    vgg_results,
    resnet_results,
    resnet_clahe_results,
]

master_comparison = pd.DataFrame(results)[
    ["model", "threshold", "accuracy", "sensitivity", "specificity",
    "precision", "auc"]
].round(4)

display(master_comparison)

# Metric comparison
metrics = ["accuracy", "sensitivity", "specificity", "auc"]
x = np.arange(len(metrics))
width = 0.8 / len(master_comparison)

fig, ax = plt.subplots(figsize=(11, 5))

for i, row in master_comparison.iterrows():
    ax.bar(x + i * width, [row[m] for m in metrics], width,
    label=row["model"])

ax.set_xticks(x + width * (len(master_comparison) - 1) / 2)
ax.set_xticklabels([m.capitalize() for m in metrics])
ax.set_ylim(0, 1)
ax.set_ylabel("Score")
ax.set_title("Principal Model Comparison - Validation Set",
fontweight="bold")
```

```
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
```

```
plt.tight_layout()
```

```
plt.savefig(
```

```
    f"{BASE}/comparison_all_models_validation.png", dpi=150,
```

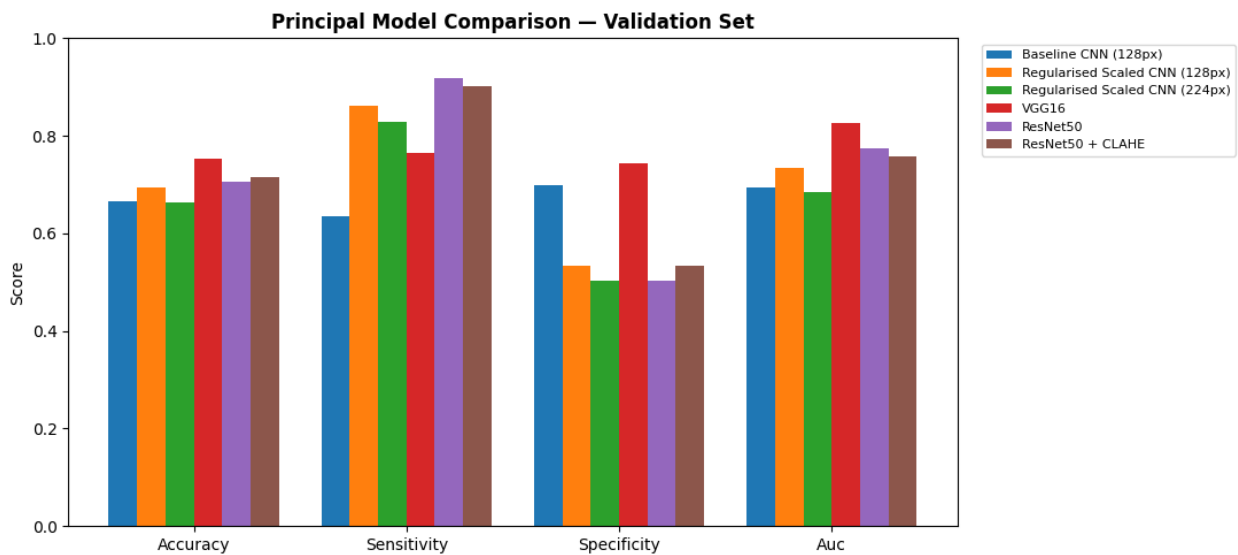
```
    bbox_inches="tight"
```

```
)
```

```
plt.show()
```

```
{"summary": "{\n  \"name\": \"master_comparison\",\n  \"rows\": 6,\n  \"fields\": [\n    {\n      \"column\": \"model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 6,\n        \"samples\": [\n          \"Baseline CNN (128px)\",\n          \"Regularised Scaled CNN (128px)\",\n          \"ResNet50 + CLAHE\"],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"threshold\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.09342720517422463,\n          \"min\": 0.3172,\n          \"max\": 0.5567,\n          \"num_unique_values\": 6,\n          \"samples\": [\n            0.5308,\n            0.5567,\n            0.3907],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"accuracy\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 0.03373808925630893,\n            \"min\": 0.6627,\n            \"max\": 0.754,\n            \"num_unique_values\": 6,\n            \"samples\": [\n              0.6667,\n              0.6944,\n              0.7143],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"column\": \"sensitivity\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 0.10585139425943649,\n              \"min\": 0.6341,\n              \"max\": 0.9187,\n              \"num_unique_values\": 6,\n              \"samples\": [\n                0.6341,\n                0.8618,\n                0.9024],\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"column\": \"specificity\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0.10602389196151339,\n                \"min\": 0.5039,\n                \"max\": 0.7442,\n                \"num_unique_values\": 4,\n                \"samples\": [\n                  0.5349,\n                  0.7442,\n                  0.6977],\n                \"semantic_type\": \"\",\n                \"description\": \"\",\n                \"column\": \"precision\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 0.04373663986483947,\n                  \"min\": 0.6145,\n                  \"max\": 0.7402,\n                  \"num_unique_values\": 6,\n                  \"samples\": [\n                    0.6667,\n                    0.6386,\n                    0.6491],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\",\n                  \"column\": \"auc\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 0.052472265690235526,\n                    \"min\": 0.6856,\n                    \"max\": 0.8254,\n                    \"num_unique_values\": 6,\n                    \"samples\": [\n                      0.6938,\n
```

```
0.7353,\n                0.7574\n            ],\n            \"semantic_type\":\n            \"\", \n            \"description\": \"\" \n            } \n            ] \n        }\n    }, \"type\": \"dataframe\", \"variable_name\": \"master_comparison\"}
```



Section 26: Multi-Seed Robustness Analysis

ResNet50 and ResNet50 + CLAHE are evaluated across random seeds **42, 7 and 123** to examine sensitivity to stochastic training variation. Performance is summarised using mean $\pm$ standard deviation across the main evaluation metrics.

This is treated as a post-selection robustness analysis rather than a statistical significance test.

Section 26.1: Three-Seed Evaluation

Existing checkpoints from each training seed are evaluated using thresholds derived independently from their corresponding validation predictions. Performance is then measured on the held-out test set for the robustness analysis.

```
# ResNet50 + CLAHE: 3-seed robustness check
```

```
import random, numpy as np, pandas as pd, tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

train_clahe = prepare_resnet_dataset(train_ds_224_clahe)
val_clahe = prepare_resnet_dataset(val_ds_224_clahe)
test_clahe = prepare_resnet_dataset(test_ds_224_clahe)

def build_model():
    base = ResNet50(weights="imagenet", include_top=False,
```

```

input_shape=(224, 224, 3))
base.trainable = False

x = layers.GlobalAveragePooling2D()(base.output)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.5)(x)
out = layers.Dense(1, activation="sigmoid")(x)

model = tf.keras.Model(base.input, out)

model.compile(
    Adam(1e-3),
    "binary_crossentropy",
    metrics=[
        "accuracy",
        tf.keras.metrics.AUC(name="auc"),
        tf.keras.metrics.Recall(name="sensitivity"),
        tf.keras.metrics.Precision(name="precision"),
    ],
)
return model, base

def run_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)
    tf.keras.backend.clear_session()

    model, base = build_model()

    # Stage A - frozen base
    model.fit(
        train_clahe,
        validation_data=val_clahe,
        epochs=10,
        class_weight=class_weights,
        verbose=1,
    )

    # Stage B - fine-tune top 30%
    base.trainable = True
    for layer in base.layers[: int(len(base.layers) * 0.7)]:
        layer.trainable = False

    model.compile(
        Adam(1e-5),
        "binary_crossentropy",
        metrics=[
            "accuracy",

```

```

        tf.keras.metrics.AUC(name="auc"),
        tf.keras.metrics.Recall(name="sensitivity"),
        tf.keras.metrics.Precision(name="precision"),
    ],
)

path = f"{MODELS_DIR}/resnet50_clahe_seed{seed}_best.keras"

model.fit(
    train_clahe,
    validation_data=val_clahe,
    epochs=20,
    class_weight=class_weights,
    callbacks=[
        EarlyStopping(
            monitor="val_auc", patience=7, mode="max",
restore_best_weights=True
        ),
        ModelCheckpoint(path, monitor="val_auc", mode="max",
save_best_only=True),
    ],
    verbose=1,
)

best = tf.keras.models.load_model(path, compile=False)

threshold = find_best_threshold(best, val_clahe, verbose=False)

result = evaluate_model(
    best,
    test_clahe,
    name=f"ResNet50 + CLAHE (seed {seed})",
    threshold=threshold,
    split_name="Held-Out Test",
    verbose=False,
)

result["seed"] = seed
tf.keras.backend.clear_session()

return result

# Run 3 seeds
df = pd.DataFrame([run_seed(seed) for seed in [42, 7, 123]])

cols = [
    "seed",
    "threshold",
    "accuracy",

```

```

    "sensitivity",
    "specificity",
    "precision",
    "auc",
]

resnet_clahe_multiseed_df = df[cols]

display(resnet_clahe_multiseed_df.round(4))

# Mean ± SD
metrics = ["accuracy", "sensitivity", "specificity", "precision",
"auc"]

summary = resnet_clahe_multiseed_df[metrics].agg(["mean", "std"])

for m in metrics:
    print(f"{m}: " f'{summary.loc["mean", m]:.4f} ± '
f'{summary.loc["std", m]:.4f}')

#####
RESNET50 + CLAHE – SEED 42
#####

=====
Stage A: frozen ResNet50 base
=====

Epoch 1/10
31/31 ————— 40s 736ms/step - accuracy: 0.5240 - auc:
0.5258 - loss: 0.8147 - precision: 0.5128 - sensitivity: 0.5449 -
val_accuracy: 0.5357 - val_auc: 0.6485 - val_loss: 0.6889 -
val_precision: 0.8000 - val_sensitivity: 0.0650
Epoch 2/10
31/31 ————— 4s 140ms/step - accuracy: 0.5271 - auc:
0.5461 - loss: 0.7017 - precision: 0.5161 - sensitivity: 0.5344 -
val_accuracy: 0.5516 - val_auc: 0.6320 - val_loss: 0.6799 -
val_precision: 0.6667 - val_sensitivity: 0.1626
Epoch 3/10
31/31 ————— 5s 149ms/step - accuracy: 0.5271 - auc:
0.5665 - loss: 0.6834 - precision: 0.5167 - sensitivity: 0.5157 -
val_accuracy: 0.6151 - val_auc: 0.6820 - val_loss: 0.6574 -
val_precision: 0.6140 - val_sensitivity: 0.5691
Epoch 4/10
31/31 ————— 5s 146ms/step - accuracy: 0.5516 - auc:
0.5855 - loss: 0.6807 - precision: 0.5450 - sensitivity: 0.5052 -
val_accuracy: 0.5198 - val_auc: 0.6786 - val_loss: 0.6871 -
val_precision: 0.5041 - val_sensitivity: 0.9919
Epoch 5/10

```

31/31 ————— 5s 142ms/step - accuracy: 0.5475 - auc:
0.5715 - loss: 0.6825 - precision: 0.5311 - sensitivity: 0.6409 -
val_accuracy: 0.5556 - val_auc: 0.6673 - val_loss: 0.6673 -
val_precision: 0.6222 - val_sensitivity: 0.2276

Epoch 6/10

31/31 ————— 5s 154ms/step - accuracy: 0.6190 - auc:
0.6648 - loss: 0.6488 - precision: 0.6039 - sensitivity: 0.6430 -
val_accuracy: 0.5794 - val_auc: 0.6736 - val_loss: 0.6581 -
val_precision: 0.6491 - val_sensitivity: 0.3008

Epoch 7/10

31/31 ————— 5s 140ms/step - accuracy: 0.6067 - auc:
0.6454 - loss: 0.6581 - precision: 0.5940 - sensitivity: 0.6200 -
val_accuracy: 0.6468 - val_auc: 0.6729 - val_loss: 0.6519 -
val_precision: 0.6250 - val_sensitivity: 0.6911

Epoch 8/10

31/31 ————— 4s 131ms/step - accuracy: 0.6231 - auc:
0.6778 - loss: 0.6426 - precision: 0.6050 - sensitivity: 0.6618 -
val_accuracy: 0.6151 - val_auc: 0.6857 - val_loss: 0.6465 -
val_precision: 0.6444 - val_sensitivity: 0.4715

Epoch 9/10

31/31 ————— 4s 137ms/step - accuracy: 0.6159 - auc:
0.6677 - loss: 0.6452 - precision: 0.6080 - sensitivity: 0.6054 -
val_accuracy: 0.5952 - val_auc: 0.6761 - val_loss: 0.6629 -
val_precision: 0.5528 - val_sensitivity: 0.8943

Epoch 10/10

31/31 ————— 5s 137ms/step - accuracy: 0.6088 - auc:
0.6713 - loss: 0.6433 - precision: 0.5886 - sensitivity: 0.6660 -
val_accuracy: 0.6151 - val_auc: 0.6928 - val_loss: 0.6514 -
val_precision: 0.7031 - val_sensitivity: 0.3659

=====
Stage B: fine-tuning
=====

Unfroze top 53 of 175 base layers.

Epoch 1/20

31/31 ————— 0s 578ms/step - accuracy: 0.5714 - auc:
0.6584 - loss: 0.7145 - precision: 0.5178 - sensitivity: 0.8823

Epoch 1: val_auc improved from None to 0.67792, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras

31/31 ————— 60s 955ms/step - accuracy: 0.6067 - auc:
0.6596 - loss: 0.6783 - precision: 0.5647 - sensitivity: 0.8559 -
val_accuracy: 0.5198 - val_auc: 0.6779 - val_loss: 0.6891 -
val_precision: 0.5556 - val_sensitivity: 0.0813

Epoch 2/20

31/31 ————— 0s 166ms/step - accuracy: 0.6343 - auc: 0.7059 - loss: 0.6328 - precision: 0.5916 - sensitivity: 0.7762
Epoch 2: val_auc improved from 0.67792 to 0.68375, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 2: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 9s 295ms/step - accuracy: 0.6496 - auc: 0.7158 - loss: 0.6227 - precision: 0.6141 - sensitivity: 0.7641 - val_accuracy: 0.5595 - val_auc: 0.6837 - val_loss: 0.6761 - val_precision: 0.7000 - val_sensitivity: 0.1707

Epoch 3/20

31/31 ————— 0s 170ms/step - accuracy: 0.6670 - auc: 0.7391 - loss: 0.6076 - precision: 0.6310 - sensitivity: 0.7315
Epoch 3: val_auc improved from 0.68375 to 0.68791, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 3: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 9s 281ms/step - accuracy: 0.6680 - auc: 0.7436 - loss: 0.5985 - precision: 0.6426 - sensitivity: 0.7244 - val_accuracy: 0.5714 - val_auc: 0.6879 - val_loss: 0.6618 - val_precision: 0.6596 - val_sensitivity: 0.2520

Epoch 4/20

31/31 ————— 0s 182ms/step - accuracy: 0.6946 - auc: 0.7594 - loss: 0.5935 - precision: 0.6729 - sensitivity: 0.7090
Epoch 4: val_auc improved from 0.68791 to 0.69333, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 4: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 10s 306ms/step - accuracy: 0.6844 - auc: 0.7476 - loss: 0.5974 - precision: 0.6673 - sensitivity: 0.7077 - val_accuracy: 0.6151 - val_auc: 0.6933 - val_loss: 0.6493 - val_precision: 0.6806 - val_sensitivity: 0.3984

Epoch 5/20

31/31 ————— 0s 173ms/step - accuracy: 0.7439 - auc: 0.8236 - loss: 0.5528 - precision: 0.7054 - sensitivity: 0.8028
Epoch 5: val_auc improved from 0.69333 to 0.69689, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 5: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

```
ahe_seed42_best.keras
31/31 ————— 9s 291ms/step - accuracy: 0.7242 - auc:
0.7980 - loss: 0.5629 - precision: 0.6983 - sensitivity: 0.7683 -
val_accuracy: 0.6071 - val_auc: 0.6969 - val_loss: 0.6421 -
val_precision: 0.6304 - val_sensitivity: 0.4715
Epoch 6/20
31/31 ————— 0s 169ms/step - accuracy: 0.7242 - auc:
0.8121 - loss: 0.5449 - precision: 0.7134 - sensitivity: 0.7475
Epoch 6: val_auc improved from 0.69689 to 0.70798, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras
31/31 ————— 11s 343ms/step - accuracy: 0.7344 - auc:
0.8169 - loss: 0.5407 - precision: 0.7102 - sensitivity: 0.7724 -
val_accuracy: 0.6548 - val_auc: 0.7080 - val_loss: 0.6393 -
val_precision: 0.6429 - val_sensitivity: 0.6585
Epoch 7/20
31/31 ————— 0s 173ms/step - accuracy: 0.7877 - auc:
0.8567 - loss: 0.5084 - precision: 0.7875 - sensitivity: 0.7864
Epoch 7: val_auc improved from 0.70798 to 0.71538, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras
31/31 ————— 8s 266ms/step - accuracy: 0.7722 - auc:
0.8584 - loss: 0.5039 - precision: 0.7550 - sensitivity: 0.7912 -
val_accuracy: 0.6508 - val_auc: 0.7154 - val_loss: 0.6438 -
val_precision: 0.6296 - val_sensitivity: 0.6911
Epoch 8/20
31/31 ————— 0s 165ms/step - accuracy: 0.8109 - auc:
0.9003 - loss: 0.4644 - precision: 0.7871 - sensitivity: 0.8457
Epoch 8: val_auc improved from 0.71538 to 0.72014, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras

Epoch 8: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed42_best.keras
31/31 ————— 12s 389ms/step - accuracy: 0.7978 - auc:
0.8778 - loss: 0.4767 - precision: 0.7750 - sensitivity: 0.8267 -
val_accuracy: 0.6587 - val_auc: 0.7201 - val_loss: 0.6377 -
val_precision: 0.6370 - val_sensitivity: 0.6992
Epoch 9/20
31/31 ————— 0s 164ms/step - accuracy: 0.8117 - auc:
0.8895 - loss: 0.4585 - precision: 0.8016 - sensitivity: 0.8182
```

Epoch 9: val_auc improved from 0.72014 to 0.72358, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 9: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 10s 325ms/step - accuracy: 0.8110 - auc: 0.8864 - loss: 0.4566 - precision: 0.7964 - sensitivity: 0.8246 - val_accuracy: 0.6865 - val_auc: 0.7236 - val_loss: 0.6466 - val_precision: 0.6549 - val_sensitivity: 0.7561

Epoch 10/20

31/31 ————— 0s 165ms/step - accuracy: 0.8143 - auc: 0.9002 - loss: 0.4364 - precision: 0.8347 - sensitivity: 0.7990

Epoch 10: val_auc did not improve from 0.72358

31/31 ————— 6s 197ms/step - accuracy: 0.8407 - auc: 0.9183 - loss: 0.4144 - precision: 0.8358 - sensitivity: 0.8392 - val_accuracy: 0.6746 - val_auc: 0.7208 - val_loss: 0.6487 - val_precision: 0.6376 - val_sensitivity: 0.7724

Epoch 11/20

31/31 ————— 0s 172ms/step - accuracy: 0.8299 - auc: 0.9204 - loss: 0.4010 - precision: 0.8516 - sensitivity: 0.7989

Epoch 11: val_auc improved from 0.72358 to 0.72884, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 11: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 14s 451ms/step - accuracy: 0.8304 - auc: 0.9148 - loss: 0.4042 - precision: 0.8323 - sensitivity: 0.8184 - val_accuracy: 0.6786 - val_auc: 0.7288 - val_loss: 0.6805 - val_precision: 0.6329 - val_sensitivity: 0.8130

Epoch 12/20

31/31 ————— 0s 171ms/step - accuracy: 0.8431 - auc: 0.9110 - loss: 0.4120 - precision: 0.8343 - sensitivity: 0.8585

Epoch 12: val_auc improved from 0.72884 to 0.73057, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 12: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

31/31 ————— 10s 313ms/step - accuracy: 0.8519 - auc: 0.9255 - loss: 0.3885 - precision: 0.8394 - sensitivity: 0.8622 - val_accuracy: 0.6865 - val_auc: 0.7306 - val_loss: 0.6745 - val_precision: 0.6467 - val_sensitivity: 0.7886

Epoch 13/20

31/31 ————— 0s 170ms/step - accuracy: 0.8759 - auc: 0.9468 - loss: 0.3444 - precision: 0.8562 - sensitivity: 0.8794

Epoch 13: val_auc did not improve from 0.73057
31/31 ————— 6s 202ms/step - accuracy: 0.8611 - auc: 0.9343 - loss: 0.3591 - precision: 0.8657 - sensitivity: 0.8476 - val_accuracy: 0.6627 - val_auc: 0.7281 - val_loss: 0.6855 - val_precision: 0.6462 - val_sensitivity: 0.6829
Epoch 14/20
31/31 ————— 0s 179ms/step - accuracy: 0.9075 - auc: 0.9575 - loss: 0.3085 - precision: 0.9143 - sensitivity: 0.8886
Epoch 14: val_auc improved from 0.73057 to 0.74220, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 14: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras
31/31 ————— 9s 277ms/step - accuracy: 0.8958 - auc: 0.9555 - loss: 0.3151 - precision: 0.9002 - sensitivity: 0.8852 - val_accuracy: 0.6786 - val_auc: 0.7422 - val_loss: 0.7029 - val_precision: 0.6382 - val_sensitivity: 0.7886
Epoch 15/20
31/31 ————— 0s 170ms/step - accuracy: 0.9129 - auc: 0.9747 - loss: 0.2684 - precision: 0.9221 - sensitivity: 0.9002
Epoch 15: val_auc improved from 0.74220 to 0.75399, saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras

Epoch 15: finished saving model to /content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_clahe_seed42_best.keras
31/31 ————— 8s 268ms/step - accuracy: 0.9081 - auc: 0.9700 - loss: 0.2746 - precision: 0.9165 - sensitivity: 0.8935 - val_accuracy: 0.7024 - val_auc: 0.7540 - val_loss: 0.7238 - val_precision: 0.6463 - val_sensitivity: 0.8618
Epoch 16/20
31/31 ————— 0s 173ms/step - accuracy: 0.9369 - auc: 0.9762 - loss: 0.2606 - precision: 0.9328 - sensitivity: 0.9342
Epoch 16: val_auc did not improve from 0.75399
31/31 ————— 7s 205ms/step - accuracy: 0.9265 - auc: 0.9743 - loss: 0.2518 - precision: 0.9321 - sensitivity: 0.9165 - val_accuracy: 0.6865 - val_auc: 0.7434 - val_loss: 0.6944 - val_precision: 0.6618 - val_sensitivity: 0.7317
Epoch 17/20
31/31 ————— 0s 169ms/step - accuracy: 0.9193 - auc: 0.9799 - loss: 0.2326 - precision: 0.9146 - sensitivity: 0.9175
Epoch 17: val_auc did not improve from 0.75399
31/31 ————— 7s 215ms/step - accuracy: 0.9142 - auc: 0.9783 - loss: 0.2402 - precision: 0.9072 - sensitivity: 0.9186 - val_accuracy: 0.6706 - val_auc: 0.7282 - val_loss: 0.8420 - val_precision: 0.6250 - val_sensitivity: 0.8130

Epoch 18/20
31/31 _____ 0s 188ms/step - accuracy: 0.9454 - auc:
0.9809 - loss: 0.2217 - precision: 0.9643 - sensitivity: 0.9174
Epoch 18: val_auc did not improve from 0.75399
31/31 _____ 7s 234ms/step - accuracy: 0.9438 - auc:
0.9803 - loss: 0.2115 - precision: 0.9629 - sensitivity: 0.9207 -
val_accuracy: 0.6587 - val_auc: 0.7347 - val_loss: 0.8723 -
val_precision: 0.6121 - val_sensitivity: 0.8211
Epoch 19/20
31/31 _____ 0s 166ms/step - accuracy: 0.9440 - auc:
0.9847 - loss: 0.1894 - precision: 0.9474 - sensitivity: 0.9376
Epoch 19: val_auc did not improve from 0.75399
31/31 _____ 6s 199ms/step - accuracy: 0.9438 - auc:
0.9863 - loss: 0.1886 - precision: 0.9473 - sensitivity: 0.9374 -
val_accuracy: 0.6825 - val_auc: 0.7398 - val_loss: 0.7898 -
val_precision: 0.6503 - val_sensitivity: 0.7561
Epoch 20/20
31/31 _____ 0s 168ms/step - accuracy: 0.9629 - auc:
0.9927 - loss: 0.1493 - precision: 0.9690 - sensitivity: 0.9550
Epoch 20: val_auc did not improve from 0.75399
31/31 _____ 6s 204ms/step - accuracy: 0.9632 - auc:
0.9913 - loss: 0.1550 - precision: 0.9703 - sensitivity: 0.9541 -
val_accuracy: 0.6786 - val_auc: 0.7392 - val_loss: 0.8357 -
val_precision: 0.6438 - val_sensitivity: 0.7642
Restoring model weights from the end of the best epoch: 15.

Seed 42: AUC=0.7378, Sensitivity=0.8483, Accuracy=0.6094, Time=5.5 min

RESNET50 + CLAHE - SEED 7
#####

=====
Stage A: frozen ResNet50 base
=====

Epoch 1/10
31/31 _____ 28s 523ms/step - accuracy: 0.4944 - auc:
0.4966 - loss: 0.9339 - precision: 0.4835 - sensitivity: 0.4885 -
val_accuracy: 0.4881 - val_auc: 0.5890 - val_loss: 0.7127 -
val_precision: 0.4879 - val_sensitivity: 0.9837
Epoch 2/10
31/31 _____ 5s 152ms/step - accuracy: 0.5557 - auc:
0.5974 - loss: 0.6771 - precision: 0.5489 - sensitivity: 0.5157 -
val_accuracy: 0.6111 - val_auc: 0.6807 - val_loss: 0.6773 -
val_precision: 0.5668 - val_sensitivity: 0.8618
Epoch 3/10
31/31 _____ 5s 139ms/step - accuracy: 0.5312 - auc:
0.5526 - loss: 0.6948 - precision: 0.5208 - sensitivity: 0.5219 -
val_accuracy: 0.6429 - val_auc: 0.6858 - val_loss: 0.6753 -
val_precision: 0.6025 - val_sensitivity: 0.7886

Epoch 4/10
31/31 ————— 4s 138ms/step - accuracy: 0.5710 - auc:
0.5845 - loss: 0.6850 - precision: 0.5626 - sensitivity: 0.5532 -
val_accuracy: 0.5397 - val_auc: 0.6715 - val_loss: 0.6723 -
val_precision: 0.6522 - val_sensitivity: 0.1220

Epoch 5/10
31/31 ————— 5s 153ms/step - accuracy: 0.5638 - auc:
0.6027 - loss: 0.6759 - precision: 0.5542 - sensitivity: 0.5553 -
val_accuracy: 0.6032 - val_auc: 0.6569 - val_loss: 0.6767 -
val_precision: 0.5642 - val_sensitivity: 0.8211

Epoch 6/10
31/31 ————— 5s 148ms/step - accuracy: 0.6200 - auc:
0.6533 - loss: 0.6644 - precision: 0.6141 - sensitivity: 0.6013 -
val_accuracy: 0.6389 - val_auc: 0.7010 - val_loss: 0.6582 -
val_precision: 0.5920 - val_sensitivity: 0.8374

Epoch 7/10
31/31 ————— 4s 141ms/step - accuracy: 0.6098 - auc:
0.6448 - loss: 0.6605 - precision: 0.5984 - sensitivity: 0.6159 -
val_accuracy: 0.5317 - val_auc: 0.6670 - val_loss: 0.7029 -
val_precision: 0.5105 - val_sensitivity: 0.9837

Epoch 8/10
31/31 ————— 5s 144ms/step - accuracy: 0.5894 - auc:
0.6068 - loss: 0.6759 - precision: 0.5797 - sensitivity: 0.5846 -
val_accuracy: 0.6230 - val_auc: 0.7098 - val_loss: 0.6587 -
val_precision: 0.6628 - val_sensitivity: 0.4634

Epoch 9/10
31/31 ————— 5s 148ms/step - accuracy: 0.6006 - auc:
0.6450 - loss: 0.6584 - precision: 0.5917 - sensitivity: 0.5929 -
val_accuracy: 0.6151 - val_auc: 0.7051 - val_loss: 0.6522 -
val_precision: 0.6585 - val_sensitivity: 0.4390

Epoch 10/10
31/31 ————— 5s 133ms/step - accuracy: 0.6047 - auc:
0.6514 - loss: 0.6540 - precision: 0.5924 - sensitivity: 0.6159 -
val_accuracy: 0.5913 - val_auc: 0.7044 - val_loss: 0.6526 -
val_precision: 0.6724 - val_sensitivity: 0.3171

=====
Stage B: fine-tuning
=====

Unfroze top 53 of 175 base layers.

Epoch 1/20
31/31 ————— 0s 514ms/step - accuracy: 0.5738 - auc:
0.6292 - loss: 0.6626 - precision: 0.5590 - sensitivity: 0.5731

Epoch 1: val_auc improved from None to 0.70369, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl

```

ahe_seed7_best.keras
31/31 ————— 52s 868ms/step - accuracy: 0.5955 - auc:
0.6470 - loss: 0.6552 - precision: 0.5835 - sensitivity: 0.6054 -
val_accuracy: 0.6151 - val_auc: 0.7037 - val_loss: 0.6449 -
val_precision: 0.7031 - val_sensitivity: 0.3659
Epoch 2/20
31/31 ————— 0s 161ms/step - accuracy: 0.6553 - auc:
0.7202 - loss: 0.6210 - precision: 0.6416 - sensitivity: 0.6689
Epoch 2: val_auc did not improve from 0.70369
31/31 ————— 6s 193ms/step - accuracy: 0.6313 - auc:
0.7053 - loss: 0.6246 - precision: 0.6157 - sensitivity: 0.6555 -
val_accuracy: 0.6071 - val_auc: 0.6965 - val_loss: 0.6374 -
val_precision: 0.6333 - val_sensitivity: 0.4634
Epoch 3/20
31/31 ————— 0s 177ms/step - accuracy: 0.6696 - auc:
0.7229 - loss: 0.6180 - precision: 0.6311 - sensitivity: 0.7037
Epoch 3: val_auc did not improve from 0.70369
31/31 ————— 7s 210ms/step - accuracy: 0.6987 - auc:
0.7537 - loss: 0.6035 - precision: 0.6833 - sensitivity: 0.7161 -
val_accuracy: 0.6151 - val_auc: 0.6992 - val_loss: 0.6329 -
val_precision: 0.6477 - val_sensitivity: 0.4634
Epoch 4/20
31/31 ————— 0s 172ms/step - accuracy: 0.6849 - auc:
0.7703 - loss: 0.5881 - precision: 0.6629 - sensitivity: 0.7083
Epoch 4: val_auc improved from 0.70369 to 0.71573, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 4: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras
31/31 ————— 8s 271ms/step - accuracy: 0.6803 - auc:
0.7596 - loss: 0.5898 - precision: 0.6621 - sensitivity: 0.7077 -
val_accuracy: 0.6468 - val_auc: 0.7157 - val_loss: 0.6195 -
val_precision: 0.6491 - val_sensitivity: 0.6016
Epoch 5/20
31/31 ————— 0s 181ms/step - accuracy: 0.7168 - auc:
0.8101 - loss: 0.5578 - precision: 0.6922 - sensitivity: 0.7507
Epoch 5: val_auc improved from 0.71573 to 0.72752, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras
31/31 ————— 9s 296ms/step - accuracy: 0.7068 - auc:
0.7944 - loss: 0.5654 - precision: 0.6853 - sensitivity: 0.7411 -
val_accuracy: 0.6627 - val_auc: 0.7275 - val_loss: 0.6112 -
val_precision: 0.6462 - val_sensitivity: 0.6829

```

Epoch 6/20
31/31 ————— 0s 177ms/step - accuracy: 0.7328 - auc:
0.8213 - loss: 0.5413 - precision: 0.6847 - sensitivity: 0.7948
Epoch 6: val_auc improved from 0.72752 to 0.73502, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 6: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras
31/31 ————— 13s 428ms/step - accuracy: 0.7252 - auc:
0.8084 - loss: 0.5539 - precision: 0.7027 - sensitivity: 0.7599 -
val_accuracy: 0.6944 - val_auc: 0.7350 - val_loss: 0.6059 -
val_precision: 0.6667 - val_sensitivity: 0.7480

Epoch 7/20
31/31 ————— 0s 165ms/step - accuracy: 0.7608 - auc:
0.8420 - loss: 0.5301 - precision: 0.7542 - sensitivity: 0.7721
Epoch 7: val_auc improved from 0.73502 to 0.73807, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras
31/31 ————— 14s 448ms/step - accuracy: 0.7559 - auc:
0.8397 - loss: 0.5259 - precision: 0.7353 - sensitivity: 0.7829 -
val_accuracy: 0.6984 - val_auc: 0.7381 - val_loss: 0.6044 -
val_precision: 0.6767 - val_sensitivity: 0.7317

Epoch 8/20
31/31 ————— 0s 167ms/step - accuracy: 0.7734 - auc:
0.8746 - loss: 0.4919 - precision: 0.7467 - sensitivity: 0.8100
Epoch 8: val_auc improved from 0.73807 to 0.74961, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras

Epoch 8: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed7_best.keras
31/31 ————— 9s 291ms/step - accuracy: 0.7589 - auc:
0.8569 - loss: 0.5002 - precision: 0.7387 - sensitivity: 0.7850 -
val_accuracy: 0.7024 - val_auc: 0.7496 - val_loss: 0.5991 -
val_precision: 0.6765 - val_sensitivity: 0.7480

Epoch 9/20
31/31 ————— 0s 170ms/step - accuracy: 0.7668 - auc:
0.8628 - loss: 0.4956 - precision: 0.7621 - sensitivity: 0.7787
Epoch 9: val_auc did not improve from 0.74961

31/31 ————— 6s 201ms/step - accuracy: 0.7681 - auc:
0.8678 - loss: 0.4844 - precision: 0.7500 - sensitivity: 0.7891 -
val_accuracy: 0.6905 - val_auc: 0.7442 - val_loss: 0.6038 -
val_precision: 0.6692 - val_sensitivity: 0.7236

Epoch 10/20
31/31 ————— 0s 177ms/step - accuracy: 0.8127 - auc: 0.8950 - loss: 0.4516 - precision: 0.7788 - sensitivity: 0.8500
Epoch 10: val_auc did not improve from 0.74961
31/31 ————— 7s 227ms/step - accuracy: 0.8172 - auc: 0.9003 - loss: 0.4439 - precision: 0.7976 - sensitivity: 0.8392 - val_accuracy: 0.7063 - val_auc: 0.7448 - val_loss: 0.6105 - val_precision: 0.6713 - val_sensitivity: 0.7805
Epoch 11/20
31/31 ————— 0s 186ms/step - accuracy: 0.8019 - auc: 0.9006 - loss: 0.4384 - precision: 0.7870 - sensitivity: 0.8065
Epoch 11: val_auc did not improve from 0.74961
31/31 ————— 7s 225ms/step - accuracy: 0.8243 - auc: 0.9108 - loss: 0.4249 - precision: 0.8165 - sensitivity: 0.8267 - val_accuracy: 0.6865 - val_auc: 0.7411 - val_loss: 0.6188 - val_precision: 0.6667 - val_sensitivity: 0.7154
Epoch 12/20
31/31 ————— 0s 182ms/step - accuracy: 0.8807 - auc: 0.9433 - loss: 0.3826 - precision: 0.8663 - sensitivity: 0.8929
Epoch 12: val_auc did not improve from 0.74961
31/31 ————— 7s 237ms/step - accuracy: 0.8652 - auc: 0.9376 - loss: 0.3814 - precision: 0.8534 - sensitivity: 0.8747 - val_accuracy: 0.7024 - val_auc: 0.7443 - val_loss: 0.6506 - val_precision: 0.6600 - val_sensitivity: 0.8049
Epoch 13/20
31/31 ————— 0s 187ms/step - accuracy: 0.8541 - auc: 0.9259 - loss: 0.3938 - precision: 0.8133 - sensitivity: 0.8809
Epoch 13: val_auc did not improve from 0.74961
31/31 ————— 7s 220ms/step - accuracy: 0.8621 - auc: 0.9364 - loss: 0.3671 - precision: 0.8554 - sensitivity: 0.8643 - val_accuracy: 0.6984 - val_auc: 0.7390 - val_loss: 0.6890 - val_precision: 0.6497 - val_sensitivity: 0.8293
Epoch 14/20
31/31 ————— 0s 170ms/step - accuracy: 0.8750 - auc: 0.9532 - loss: 0.3317 - precision: 0.8647 - sensitivity: 0.8784
Epoch 14: val_auc did not improve from 0.74961
31/31 ————— 7s 205ms/step - accuracy: 0.8764 - auc: 0.9505 - loss: 0.3368 - precision: 0.8745 - sensitivity: 0.8727 - val_accuracy: 0.6905 - val_auc: 0.7394 - val_loss: 0.6612 - val_precision: 0.6642 - val_sensitivity: 0.7398
Epoch 15/20
31/31 ————— 0s 175ms/step - accuracy: 0.8862 - auc: 0.9590 - loss: 0.3141 - precision: 0.8706 - sensitivity: 0.8844
Epoch 15: val_auc did not improve from 0.74961
31/31 ————— 7s 205ms/step - accuracy: 0.9091 - auc: 0.9703 - loss: 0.2901 - precision: 0.9114 - sensitivity: 0.9019 - val_accuracy: 0.6984 - val_auc: 0.7406 - val_loss: 0.7086 - val_precision: 0.6577 - val_sensitivity: 0.7967
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 8.

Seed 7: AUC=0.7177, Sensitivity=0.7172, Accuracy=0.6150, Time=4.3 min

RESNET50 + CLAE - SEED 123
#####

=====
Stage A: frozen ResNet50 base
=====

Epoch 1/10

31/31 ————— 28s 516ms/step - accuracy: 0.4709 - auc:
0.4747 - loss: 0.9163 - precision: 0.4606 - sensitivity: 0.4760 -
val_accuracy: 0.5833 - val_auc: 0.6191 - val_loss: 0.6808 -
val_precision: 0.5592 - val_sensitivity: 0.6911

Epoch 2/10

31/31 ————— 5s 157ms/step - accuracy: 0.5444 - auc:
0.5647 - loss: 0.6884 - precision: 0.5310 - sensitivity: 0.5908 -
val_accuracy: 0.6032 - val_auc: 0.6490 - val_loss: 0.6772 -
val_precision: 0.5772 - val_sensitivity: 0.6992

Epoch 3/10

31/31 ————— 4s 135ms/step - accuracy: 0.5546 - auc:
0.5769 - loss: 0.6834 - precision: 0.5511 - sensitivity: 0.4843 -
val_accuracy: 0.6190 - val_auc: 0.6470 - val_loss: 0.6744 -
val_precision: 0.5944 - val_sensitivity: 0.6911

Epoch 4/10

31/31 ————— 4s 136ms/step - accuracy: 0.5689 - auc:
0.6208 - loss: 0.6684 - precision: 0.5638 - sensitivity: 0.5261 -
val_accuracy: 0.6349 - val_auc: 0.6642 - val_loss: 0.6698 -
val_precision: 0.5906 - val_sensitivity: 0.8211

Epoch 5/10

31/31 ————— 5s 149ms/step - accuracy: 0.6129 - auc:
0.6433 - loss: 0.6628 - precision: 0.6012 - sensitivity: 0.6200 -
val_accuracy: 0.5952 - val_auc: 0.6622 - val_loss: 0.6671 -
val_precision: 0.5574 - val_sensitivity: 0.8293

Epoch 6/10

31/31 ————— 5s 155ms/step - accuracy: 0.5822 - auc:
0.6260 - loss: 0.6657 - precision: 0.5665 - sensitivity: 0.6221 -
val_accuracy: 0.5437 - val_auc: 0.6504 - val_loss: 0.6716 -
val_precision: 0.5952 - val_sensitivity: 0.2033

Epoch 7/10

31/31 ————— 4s 140ms/step - accuracy: 0.5730 - auc:
0.6155 - loss: 0.6679 - precision: 0.5673 - sensitivity: 0.5365 -
val_accuracy: 0.6032 - val_auc: 0.6548 - val_loss: 0.6629 -
val_precision: 0.5906 - val_sensitivity: 0.6098

Epoch 8/10

31/31 ————— 5s 150ms/step - accuracy: 0.5975 - auc:
0.6415 - loss: 0.6576 - precision: 0.5887 - sensitivity: 0.5887 -
val_accuracy: 0.6349 - val_auc: 0.6712 - val_loss: 0.6559 -
val_precision: 0.6202 - val_sensitivity: 0.6504

Epoch 9/10
31/31 ————— 5s 141ms/step - accuracy: 0.5975 - auc:
0.6580 - loss: 0.6494 - precision: 0.5906 - sensitivity: 0.5783 -
val_accuracy: 0.6429 - val_auc: 0.6799 - val_loss: 0.6476 -
val_precision: 0.6138 - val_sensitivity: 0.7236

Epoch 10/10
31/31 ————— 5s 136ms/step - accuracy: 0.5935 - auc:
0.6486 - loss: 0.6527 - precision: 0.5842 - sensitivity: 0.5866 -
val_accuracy: 0.6508 - val_auc: 0.6911 - val_loss: 0.6482 -
val_precision: 0.6471 - val_sensitivity: 0.6260

=====
Stage B: fine-tuning
=====

Unfroze top 53 of 175 base layers.

Epoch 1/20
31/31 ————— 0s 502ms/step - accuracy: 0.5938 - auc:
0.6658 - loss: 0.6606 - precision: 0.5495 - sensitivity: 0.8355
Epoch 1: val_auc improved from None to 0.69856, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 1: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 59s 1s/step - accuracy: 0.5955 - auc:
0.6541 - loss: 0.6624 - precision: 0.5592 - sensitivity: 0.8184 -
val_accuracy: 0.6111 - val_auc: 0.6986 - val_loss: 0.6382 -
val_precision: 0.6437 - val_sensitivity: 0.4553

Epoch 2/20
31/31 ————— 0s 171ms/step - accuracy: 0.6323 - auc:
0.7015 - loss: 0.6344 - precision: 0.6051 - sensitivity: 0.7563
Epoch 2: val_auc improved from 0.69856 to 0.70224, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 2: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 12s 391ms/step - accuracy: 0.6486 - auc:
0.7117 - loss: 0.6295 - precision: 0.6142 - sensitivity: 0.7578 -
val_accuracy: 0.6270 - val_auc: 0.7022 - val_loss: 0.6323 -
val_precision: 0.6593 - val_sensitivity: 0.4878

Epoch 3/20
31/31 ————— 0s 178ms/step - accuracy: 0.6594 - auc:
0.7411 - loss: 0.6114 - precision: 0.6261 - sensitivity: 0.7478
Epoch 3: val_auc improved from 0.70224 to 0.70442, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 3: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 15s 496ms/step - accuracy: 0.6599 - auc:
0.7354 - loss: 0.6103 - precision: 0.6332 - sensitivity: 0.7244 -
val_accuracy: 0.6270 - val_auc: 0.7044 - val_loss: 0.6287 -
val_precision: 0.6593 - val_sensitivity: 0.4878

Epoch 4/20

31/31 ————— 0s 179ms/step - accuracy: 0.7245 - auc:
0.7964 - loss: 0.5767 - precision: 0.7002 - sensitivity: 0.7765

Epoch 4: val_auc improved from 0.70442 to 0.70631, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 4: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 17s 566ms/step - accuracy: 0.7048 - auc:
0.7821 - loss: 0.5805 - precision: 0.6799 - sensitivity: 0.7495 -
val_accuracy: 0.6151 - val_auc: 0.7063 - val_loss: 0.6252 -
val_precision: 0.6327 - val_sensitivity: 0.5041

Epoch 5/20

31/31 ————— 0s 174ms/step - accuracy: 0.7513 - auc:
0.8092 - loss: 0.5646 - precision: 0.7237 - sensitivity: 0.7850

Epoch 5: val_auc improved from 0.70631 to 0.70949, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 5: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 13s 404ms/step - accuracy: 0.7303 - auc:
0.7938 - loss: 0.5703 - precision: 0.7040 - sensitivity: 0.7745 -
val_accuracy: 0.6468 - val_auc: 0.7095 - val_loss: 0.6143 -
val_precision: 0.6250 - val_sensitivity: 0.6911

Epoch 6/20

31/31 ————— 0s 169ms/step - accuracy: 0.7954 - auc:
0.8690 - loss: 0.5103 - precision: 0.7849 - sensitivity: 0.8103

Epoch 6: val_auc did not improve from 0.70949

31/31 ————— 6s 202ms/step - accuracy: 0.7477 - auc:
0.8343 - loss: 0.5296 - precision: 0.7266 - sensitivity: 0.7766 -
val_accuracy: 0.6468 - val_auc: 0.7087 - val_loss: 0.6121 -
val_precision: 0.6197 - val_sensitivity: 0.7154

Epoch 7/20

31/31 ————— 0s 183ms/step - accuracy: 0.7603 - auc:
0.8344 - loss: 0.5248 - precision: 0.7290 - sensitivity: 0.7645

Epoch 7: val_auc improved from 0.70949 to 0.72783, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 7: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 11s 342ms/step - accuracy: 0.7559 - auc:
0.8369 - loss: 0.5221 - precision: 0.7449 - sensitivity: 0.7620 -
val_accuracy: 0.6667 - val_auc: 0.7278 - val_loss: 0.6013 -
val_precision: 0.6489 - val_sensitivity: 0.6911

Epoch 8/20

31/31 ————— 0s 169ms/step - accuracy: 0.7865 - auc:
0.8761 - loss: 0.4835 - precision: 0.7416 - sensitivity: 0.8358

Epoch 8: val_auc did not improve from 0.72783

31/31 ————— 6s 201ms/step - accuracy: 0.7794 - auc:
0.8689 - loss: 0.4874 - precision: 0.7594 - sensitivity: 0.8038 -
val_accuracy: 0.6786 - val_auc: 0.7204 - val_loss: 0.6084 -
val_precision: 0.6329 - val_sensitivity: 0.8130

Epoch 9/20

31/31 ————— 0s 182ms/step - accuracy: 0.7973 - auc:
0.8867 - loss: 0.4683 - precision: 0.7585 - sensitivity: 0.8411

Epoch 9: val_auc did not improve from 0.72783

31/31 ————— 7s 228ms/step - accuracy: 0.7967 - auc:
0.8845 - loss: 0.4659 - precision: 0.7652 - sensitivity: 0.8434 -
val_accuracy: 0.6944 - val_auc: 0.7150 - val_loss: 0.6184 -
val_precision: 0.6402 - val_sensitivity: 0.8537

Epoch 10/20

31/31 ————— 0s 176ms/step - accuracy: 0.8523 - auc:
0.9173 - loss: 0.4249 - precision: 0.8475 - sensitivity: 0.8597

Epoch 10: val_auc improved from 0.72783 to 0.72793, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 10: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 10s 328ms/step - accuracy: 0.8264 - auc:
0.8963 - loss: 0.4477 - precision: 0.8084 - sensitivity: 0.8455 -
val_accuracy: 0.6786 - val_auc: 0.7279 - val_loss: 0.6182 -
val_precision: 0.6382 - val_sensitivity: 0.7886

Epoch 11/20

31/31 ————— 0s 168ms/step - accuracy: 0.8479 - auc:
0.9225 - loss: 0.4101 - precision: 0.8060 - sensitivity: 0.8821

Epoch 11: val_auc did not improve from 0.72793

31/31 ————— 6s 200ms/step - accuracy: 0.8315 - auc:
0.9188 - loss: 0.4114 - precision: 0.8230 - sensitivity: 0.8351 -
val_accuracy: 0.6984 - val_auc: 0.7191 - val_loss: 0.6513 -
val_precision: 0.6442 - val_sensitivity: 0.8537

Epoch 12/20

31/31 ————— 0s 176ms/step - accuracy: 0.8598 - auc:
0.9414 - loss: 0.3690 - precision: 0.8268 - sensitivity: 0.8892

Epoch 12: val_auc improved from 0.72793 to 0.73076, saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 12: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 12s 382ms/step - accuracy: 0.8672 - auc:
0.9404 - loss: 0.3677 - precision: 0.8511 - sensitivity: 0.8831 -
val_accuracy: 0.6825 - val_auc: 0.7308 - val_loss: 0.6751 -
val_precision: 0.6319 - val_sensitivity: 0.8374

Epoch 13/20

31/31 ————— 0s 183ms/step - accuracy: 0.8744 - auc:
0.9419 - loss: 0.3561 - precision: 0.8691 - sensitivity: 0.8818

Epoch 13: val_auc improved from 0.73076 to 0.73747, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 13: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 29s 644ms/step - accuracy: 0.8672 - auc:
0.9401 - loss: 0.3537 - precision: 0.8497 - sensitivity: 0.8852 -
val_accuracy: 0.6746 - val_auc: 0.7375 - val_loss: 0.6338 -
val_precision: 0.6434 - val_sensitivity: 0.7480

Epoch 14/20

31/31 ————— 0s 166ms/step - accuracy: 0.8894 - auc:
0.9608 - loss: 0.3150 - precision: 0.8925 - sensitivity: 0.8846

Epoch 14: val_auc did not improve from 0.73747

31/31 ————— 6s 198ms/step - accuracy: 0.8887 - auc:
0.9607 - loss: 0.3100 - precision: 0.8870 - sensitivity: 0.8852 -
val_accuracy: 0.6746 - val_auc: 0.7354 - val_loss: 0.6858 -
val_precision: 0.6358 - val_sensitivity: 0.7805

Epoch 15/20

31/31 ————— 0s 188ms/step - accuracy: 0.9267 - auc:
0.9731 - loss: 0.2735 - precision: 0.9204 - sensitivity: 0.9349

Epoch 15: val_auc did not improve from 0.73747

31/31 ————— 7s 229ms/step - accuracy: 0.9275 - auc:
0.9704 - loss: 0.2727 - precision: 0.9198 - sensitivity: 0.9332 -
val_accuracy: 0.6706 - val_auc: 0.7368 - val_loss: 0.7027 -
val_precision: 0.6449 - val_sensitivity: 0.7236

Epoch 16/20

31/31 ————— 0s 187ms/step - accuracy: 0.9426 - auc:
0.9789 - loss: 0.2456 - precision: 0.9428 - sensitivity: 0.9385

Epoch 16: val_auc improved from 0.73747 to 0.73842, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 16: finished saving model to

/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 25s 732ms/step - accuracy: 0.9254 - auc:
0.9701 - loss: 0.2596 - precision: 0.9338 - sensitivity: 0.9123 -
val_accuracy: 0.6627 - val_auc: 0.7384 - val_loss: 0.7371 -
val_precision: 0.6284 - val_sensitivity: 0.7561

Epoch 17/20

31/31 ————— 0s 166ms/step - accuracy: 0.9304 - auc:
0.9836 - loss: 0.2192 - precision: 0.9226 - sensitivity: 0.9342
Epoch 17: val_auc improved from 0.73842 to 0.73892, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 17: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 14s 461ms/step - accuracy: 0.9326 - auc:
0.9832 - loss: 0.2180 - precision: 0.9293 - sensitivity: 0.9332 -
val_accuracy: 0.6944 - val_auc: 0.7389 - val_loss: 0.7525 -
val_precision: 0.6716 - val_sensitivity: 0.7317

Epoch 18/20

31/31 ————— 0s 169ms/step - accuracy: 0.9474 - auc:
0.9859 - loss: 0.1962 - precision: 0.9631 - sensitivity: 0.9310
Epoch 18: val_auc improved from 0.73892 to 0.74113, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 18: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 19s 605ms/step - accuracy: 0.9387 - auc:
0.9830 - loss: 0.1999 - precision: 0.9356 - sensitivity: 0.9395 -
val_accuracy: 0.6905 - val_auc: 0.7411 - val_loss: 0.7749 -
val_precision: 0.6642 - val_sensitivity: 0.7398

Epoch 19/20

31/31 ————— 0s 173ms/step - accuracy: 0.9518 - auc:
0.9872 - loss: 0.1781 - precision: 0.9672 - sensitivity: 0.9377
Epoch 19: val_auc did not improve from 0.74113

31/31 ————— 7s 219ms/step - accuracy: 0.9448 - auc:
0.9853 - loss: 0.1859 - precision: 0.9512 - sensitivity: 0.9353 -
val_accuracy: 0.6706 - val_auc: 0.7356 - val_loss: 0.8750 -
val_precision: 0.6266 - val_sensitivity: 0.8049

Epoch 20/20

31/31 ————— 0s 181ms/step - accuracy: 0.9615 - auc:
0.9940 - loss: 0.1548 - precision: 0.9697 - sensitivity: 0.9541
Epoch 20: val_auc improved from 0.74113 to 0.74853, saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

Epoch 20: finished saving model to
/content/drive/MyDrive/cbis_ddsm/models_patientsplit_dedup/resnet50_cl
ahe_seed123_best.keras

31/31 ————— 20s 635ms/step - accuracy: 0.9632 - auc:
0.9906 - loss: 0.1548 - precision: 0.9663 - sensitivity: 0.9582 -
val_accuracy: 0.6786 - val_auc: 0.7485 - val_loss: 0.8169 -
val_precision: 0.6522 - val_sensitivity: 0.7317
Restoring model weights from the end of the best epoch: 20.

Seed 123: AUC=0.7333, Sensitivity=0.8207, Accuracy=0.6565, Time=6.8
min

=====
ResNet50 + CLAHE – 3 SEEDS
=====

```
{"summary": "{\n  \"name\": \"          \",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"seed\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 59,\n        \"min\": 7,\n        \"max\": 123,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          42,\n          7,\n          123\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"threshold\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.05398175617743462,\n        \"min\": 0.3698,\n        \"max\": 0.465,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.465,\n          0.4615,\n          0.3698\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02572942543729517,\n        \"min\": 0.6094,\n        \"max\": 0.6565,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.6094,\n          0.615,\n          0.6565\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sensitivity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.06911490432605695,\n        \"min\": 0.7172,\n        \"max\": 0.8483,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.8483,\n          0.7172,\n          0.8207\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"specificity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.05611844616523162,\n        \"min\": 0.4491,\n        \"max\": 0.5463,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.5463,\n          0.4491\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"precision\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02150124027430356,\n        \"min\": 0.5083,\n        \"max\": 0.5484,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.5083,\n          0.5149,\n          0.5083\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"auc\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02150124027430356,\n        \"min\": 0.5083,\n        \"max\": 0.5484,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.5083,\n          0.5149,\n          0.5083\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sensitivity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.06911490432605695,\n        \"min\": 0.7172,\n        \"max\": 0.8483,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.8483,\n          0.7172,\n          0.8207\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"specificity\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.05611844616523162,\n        \"min\": 0.4491,\n        \"max\": 0.5463,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.5463,\n          0.4491\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"precision\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02150124027430356,\n        \"min\": 0.5083,\n        \"max\": 0.5484,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.5083,\n          0.5149,\n          0.5083\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"auc\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02150124027430356,\n        \"min\": 0.5083,\n        \"max\": 0.5484,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.5083,\n          0.5149,\n          0.5083\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}
```

```

\ "std\ ": 0.010548459603183768,\n          \ "min\ ": 0.7177,\n
\ "max\ ": 0.7378,\n          \ "num_unique_values\ ": 3,\n
\ "samples\ ": [\n          0.7378,\n          0.7177\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\n          }\n
n      }\n      ]\n      }", "type": "dataframe"}

```

Mean $\pm$ Std across 3 runs:

```

accuracy      : 0.6270  $\pm$  0.0257
sensitivity    : 0.7954  $\pm$  0.0691
specificity    : 0.5139  $\pm$  0.0561
precision     : 0.5239  $\pm$  0.0215
auc           : 0.7296  $\pm$  0.0105

```

Reload existing 3-seed results (no retraining)

```

import pandas as pd
import tensorflow as tf

rows = []

for seed in [42, 7, 123]:
    model = tf.keras.models.load_model(
        f"{MODELS_DIR}/resnet50_clahe_seed{seed}_best.keras",
        compile=False
    )

    threshold = find_best_threshold(model, resnet_clahe_val_ds,
        verbose=False)

    result = evaluate_model(
        model,
        resnet_clahe_test_ds,
        name=f"ResNet50 + CLAHE (seed {seed})",
        threshold=threshold,
        split_name="Held-Out Test",
        verbose=False,
    )

    result["seed"] = seed
    rows.append(result)

resnet_clahe_multiseed_df = pd.DataFrame(rows)[
    ["seed", "threshold", "accuracy", "sensitivity", "specificity",
    "precision", "auc"]
]

display(resnet_clahe_multiseed_df.round(4))

metrics = ["accuracy", "sensitivity", "specificity", "precision",
    "auc"]

```

```

summary = resnet_clahe_multiseed_df[metrics].agg(["mean", "std"])

print("\nMean ± Std across 3 runs:")
for m in metrics:
    print(f'{m}: {summary.loc["mean", m]:.4f} ± {summary.loc["std", m]:.4f}')

{"summary": "{\n  \"name\": \"      print(f'{m}: {summary\", \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"seed\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 59,\n    \"min\": 7, \"max\": 123,\n    \"num_unique_values\": 3, \"samples\": [\n      42, \"\n      7, \"\n      123\n    ], \"semantic_type\": \"\", \"\n  \"description\": \"\"\n    }, {\n      \"column\": \"threshold\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.05398175617743462, \"min\": 0.3698, \"\n    \"max\": 0.465, \"num_unique_values\": 3, \"\n  \"samples\": [\n    0.465, \"\n    0.4615, \"\n    0.3698\n  ], \"semantic_type\": \"\", \"\n  \"description\": \"\"\n    }, {\n      \"column\": \"accuracy\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.02572942543729517, \"min\": 0.6094, \"\n    \"max\": 0.6565, \"num_unique_values\": 3, \"\n  \"samples\": [\n    0.6094, \"\n    0.615, \"\n    0.6565\n  ], \"semantic_type\": \"\", \"\n  \"description\": \"\"\n    }, {\n      \"column\": \"sensitivity\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.06911490432605695, \"min\": 0.7172, \"\n    \"max\": 0.8483, \"num_unique_values\": 3, \"\n  \"samples\": [\n    0.8483, \"\n    0.7172, \"\n    0.8207\n  ], \"semantic_type\": \"\", \"\n  \"description\": \"\"\n    }, {\n      \"column\": \"specificity\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.05611844616523162, \"min\": 0.4491, \"\n    \"max\": 0.5463, \"num_unique_values\": 2, \"\n  \"samples\": [\n    0.5463, \"\n    0.4491\n  ], \"\n  \"semantic_type\": \"\", \"description\": \"\"\n    }, {\n      \"column\": \"precision\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.02150124027430356, \"min\": 0.5083, \"\n    \"max\": 0.5484, \"num_unique_values\": 3, \"\n  \"samples\": [\n    0.5083, \"\n    0.5149\n  ], \"semantic_type\": \"\", \"\n  \"description\": \"\"\n    }, {\n      \"column\": \"auc\", \"\n  \"properties\": {\n    \"dtype\": \"number\", \"std\": 0.010548459603183768, \"min\": 0.7177, \"\n    \"max\": 0.7378, \"num_unique_values\": 3, \"\n  \"samples\": [\n    0.7378, \"\n    0.7177\n  ], \"\n  \"semantic_type\": \"\", \"description\": \"\"\n    }\n  ]\n}", "type": "dataframe"}

```

Mean $\pm$ Std across 3 runs:
accuracy: 0.6270 ± 0.0257
sensitivity: 0.7954 ± 0.0691
specificity: 0.5139 ± 0.0561
precision: 0.5239 ± 0.0215
auc: 0.7296 ± 0.0105

Section 26.2: Multi-Seed Comparison

Performance across the three seeds is summarised using mean $\pm$ standard deviation. ResNet50 + CLAHE shows slightly higher mean accuracy, specificity, precision and ROC-AUC, while plain ResNet50 retains slightly higher mean sensitivity.

The results provide evidence about training stability and are not interpreted as statistical significance.

```
# 3-seed results with progress output

import os
import pandas as pd
import tensorflow as tf

def evaluate_seeds(prefix, val_ds, test_ds):
    rows = []

    for i, seed in enumerate([42, 7, 123], 1):
        version = "final" if prefix == "resnet50" and seed == 42 else
"best"

        print(f"\n[{i}/3] {prefix} - Seed {seed}")
        print(f"    Loading {version} checkpoint...")

        model = tf.keras.models.load_model(
            os.path.join(MODELS_DIR,
f"{prefix}_seed{seed}_{version}.keras"),
            compile=False,
        )

        print("    Finding validation threshold...")

        threshold = find_best_threshold(model, val_ds, verbose=True)

        print(f"    Threshold = {threshold:.4f}")
        print("    Evaluating held-out test set...")

        result = evaluate_model(
            model,
```

```

        test_ds,
        name=f"{prefix} seed {seed}",
        threshold=threshold,
        split_name="Held-Out Test",
        verbose=True,
    )

    result["seed"] = seed
    rows.append(result)

    print(f"    ✓ Seed {seed} complete")

    del model
    tf.keras.backend.clear_session()

    return pd.DataFrame(rows)[
        [
            "seed",
            "threshold",
            "accuracy",
            "sensitivity",
            "specificity",
            "precision",
            "auc",
        ]
    ]
]

```

```

print("\n===== RESNET50 =====")
resnet_multiseed_df = evaluate_seeds("resnet50", resnet_val_ds,
resnet_test_ds)

```

```

print("\n===== RESNET50 + CLAHE =====")
resnet_clahe_multiseed_df = evaluate_seeds(
    "resnet50_clahe", resnet_clahe_val_ds, resnet_clahe_test_ds
)

```

```

print("\n===== FINAL RESULTS =====")

```

```

print("\nResNet50")
display(resnet_multiseed_df.round(4))

```

```

print("\nResNet50 + CLAHE")
display(resnet_clahe_multiseed_df.round(4))

```

```

===== RESNET50 =====

```

```

[1/3] resnet50 – Seed 42
      Loading final checkpoint...
      Finding validation threshold...

```

Optimal threshold (Youden's J): 0.504 (val sensitivity=0.724, val specificity=0.667)

Threshold = 0.5042

Evaluating held-out test set...

=====
resnet50 seed 42 – Held-Out Test Results (threshold=0.504)
=====

	precision	recall	f1-score	support
Benign	0.75	0.59	0.66	216
Malignant	0.54	0.71	0.61	145
accuracy			0.64	361
macro avg	0.65	0.65	0.64	361
weighted avg	0.67	0.64	0.64	361

Accuracy : 0.6399

Sensitivity : 0.7103 (missed 42 of 145 malignant cases)

Specificity : 0.5926

Precision : 0.5393

AUC : 0.7142

✓ Seed 42 complete

[2/3] resnet50 – Seed 7

Loading best checkpoint...

Finding validation threshold...

Optimal threshold (Youden's J): 0.177 (val sensitivity=0.935, val specificity=0.481)

Threshold = 0.1766

Evaluating held-out test set...

=====
resnet50 seed 7 – Held-Out Test Results (threshold=0.177)
=====

	precision	recall	f1-score	support
Benign	0.82	0.38	0.51	216
Malignant	0.48	0.88	0.62	145
accuracy			0.58	361
macro avg	0.65	0.63	0.57	361
weighted avg	0.68	0.58	0.56	361

Accuracy : 0.5762

Sensitivity : 0.8759 (missed 18 of 145 malignant cases)

Specificity : 0.375

Precision : 0.4847

AUC : 0.7326

✓ Seed 7 complete

```
[3/3] resnet50 – Seed 123
  Loading best checkpoint...
  Finding validation threshold...
Optimal threshold (Youden's J): 0.424 (val sensitivity=0.919, val
specificity=0.504)
  Threshold = 0.4235
  Evaluating held-out test set...
```

```
=====
resnet50 seed 123 – Held-Out Test Results (threshold=0.424)
=====
```

	precision	recall	f1-score	support
Benign	0.83	0.46	0.59	216
Malignant	0.52	0.86	0.65	145
accuracy			0.62	361
macro avg	0.67	0.66	0.62	361
weighted avg	0.71	0.62	0.61	361

```
Accuracy      : 0.6205
Sensitivity   : 0.8621 (missed 20 of 145 malignant cases)
Specificity   : 0.4583
Precision     : 0.5165
AUC           : 0.7123
  ✓ Seed 123 complete
```

```
===== RESNET50 + CLAHE =====
```

```
[1/3] resnet50_clahe – Seed 42
  Loading best checkpoint...
  Finding validation threshold...
Optimal threshold (Youden's J): 0.465 (val sensitivity=0.878, val
specificity=0.543)
  Threshold = 0.4650
  Evaluating held-out test set...
```

```
=====
resnet50_clahe seed 42 – Held-Out Test Results (threshold=0.465)
=====
```

	precision	recall	f1-score	support
Benign	0.82	0.45	0.58	216
Malignant	0.51	0.85	0.64	145
accuracy			0.61	361
macro avg	0.66	0.65	0.61	361
weighted avg	0.69	0.61	0.60	361

Accuracy : 0.6094
Sensitivity : 0.8483 (missed 22 of 145 malignant cases)
Specificity : 0.4491
Precision : 0.5083
AUC : 0.7378
✓ Seed 42 complete

[2/3] resnet50_clahe – Seed 7

Loading best checkpoint...

Finding validation threshold...

Optimal threshold (Youden's J): 0.461 (val sensitivity=0.821, val specificity=0.636)

Threshold = 0.4615

Evaluating held-out test set...

=====
resnet50_clahe seed 7 – Held-Out Test Results (threshold=0.461)
=====

	precision	recall	f1-score	support
Benign	0.74	0.55	0.63	216
Malignant	0.51	0.72	0.60	145
accuracy			0.61	361
macro avg	0.63	0.63	0.61	361
weighted avg	0.65	0.61	0.62	361

Accuracy : 0.615
Sensitivity : 0.7172 (missed 41 of 145 malignant cases)
Specificity : 0.5463
Precision : 0.5149
AUC : 0.7177
✓ Seed 7 complete

[3/3] resnet50_clahe – Seed 123

Loading best checkpoint...

Finding validation threshold...

Optimal threshold (Youden's J): 0.370 (val sensitivity=0.837, val specificity=0.574)

Threshold = 0.3698

Evaluating held-out test set...

=====
resnet50_clahe seed 123 – Held-Out Test Results (threshold=0.370)
=====

	precision	recall	f1-score	support
Benign	0.82	0.55	0.66	216
Malignant	0.55	0.82	0.66	145

accuracy			0.66	361
macro avg	0.68	0.68	0.66	361
weighted avg	0.71	0.66	0.66	361

Accuracy : 0.6565
 Sensitivity : 0.8207 (missed 26 of 145 malignant cases)
 Specificity : 0.5463
 Precision : 0.5484
 AUC : 0.7333
 ✓ Seed 123 complete

===== FINAL RESULTS =====

ResNet50

```

{"summary": "{\n  \"name\": \"display(resnet_clahe_multiseed_df\", \n  \"rows\": 3, \n  \"fields\": [\n    {\n      \"column\": \"seed\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 59, \n        \"min\": 7, \n        \"max\": 123, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          42, \n          7, \n          123\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n        \"threshold\": \n      }, \n      \"column\": \"\n      \"properties\": {\n        \"dtype\": \"\n        \"number\", \n        \"std\": 0.17068189710686954, \n        \"min\": 0.1766, \n        \"max\": 0.5042, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.5042, \n          0.1766, \n          0.4235\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n        \"column\": \"\n        \"properties\": {\n          \"dtype\": \"\n          \"number\", \n          \"std\": 0.03265103367429582, \n          \"min\": 0.5762, \n          \"max\": 0.6399, \n          \"num_unique_values\": 3, \n          \"samples\": [\n            0.6399, \n            0.5762, \n            0.6205\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          \"column\": \"\n          \"properties\": {\n            \"dtype\": \"\n            \"number\", \n            \"std\": 0.09188492803501559, \n            \"min\": 0.7103, \n            \"max\": 0.8759, \n            \"num_unique_values\": 3, \n            \"samples\": [\n              0.7103, \n              0.8759, \n              0.8621\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\", \n            \"column\": \"\n            \"properties\": {\n              \"dtype\": \"\n              \"number\", \n              \"std\": 0.10979157526877917, \n              \"min\": 0.375, \n              \"max\": 0.5926, \n              \"num_unique_values\": 3, \n              \"samples\": [\n                0.5926, \n                0.375, \n                0.4583\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\", \n              \"column\": \"\n              \"properties\": {\n                \"dtype\": \"\n                \"number\", \n                \"std\": 0.027423347716863444, \n                \"min\": 0.4847, \n                \"max\": 0.5393, \n                \"num_unique_values\": 3, \n                \"samples\": [\n                  0.5393, \n                  0.4847, \n                  \n                ]\n              }\n            }\n          }\n        }\n      }\n    }\n  ]\n}

```

```
0.5165\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\":\n      \"auc\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.011212047092302117,\n        \"min\": 0.7123,\n        \"max\": 0.7326,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          0.7142,\n          0.7326,\n          0.7123\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}
```

ResNet50 + CLAHE

```
{\"summary\":{\n  \"name\": \"display(resnet_clahe_multiseed_df\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"seed\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 59,\n        \"min\": 7,\n        \"max\": 123,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          42,\n          7,\n          123\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\":\n        \"threshold\",\n        \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 0.05398175617743462,\n          \"min\": 0.3698,\n          \"max\": 0.465,\n          \"num_unique_values\": 3,\n          \"samples\": [\n            0.465,\n            0.4615,\n            0.3698\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\":\n          \"accuracy\",\n          \"properties\": {\n            \"dtype\":\n            \"number\",\n            \"std\": 0.02572942543729517,\n            \"min\": 0.6094,\n            \"max\": 0.6565,\n            \"num_unique_values\": 3,\n            \"samples\": [\n              0.6094,\n              0.615,\n              0.6565\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\":\n            \"sensitivity\",\n            \"properties\": {\n              \"dtype\":\n              \"number\",\n              \"std\": 0.06911490432605695,\n              \"min\": 0.7172,\n              \"max\": 0.8483,\n              \"num_unique_values\": 3,\n              \"samples\": [\n                0.8483,\n                0.7172,\n                0.8207\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\":\n              \"specificity\",\n              \"properties\": {\n                \"dtype\":\n                \"number\",\n                \"std\": 0.05611844616523162,\n                \"min\": 0.4491,\n                \"max\": 0.5463,\n                \"num_unique_values\": 2,\n                \"samples\": [\n                  0.5463,\n                  0.4491\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              }\n            },\n            {\n              \"column\": \"precision\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 0.02150124027430356,\n                \"min\": 0.5083,\n                \"max\": 0.5484,\n                \"num_unique_values\": 3,\n                \"samples\": [\n                  0.5083,\n                  0.5149\n                ],\n                \"semantic_type\":\n                \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"auc\",\n                \"properties\": {\n                  \"dtype\":\n                  \"number\",\n                  \"std\": 0.010548459603183768,\n                  \"min\":
```

```

0.7177,\n          \"max\": 0.7378,\n          \"num_unique_values\": 3,\n\"samples\": [\n          0.7378,\n          0.7177\n          ],\n\"semantic_type\": \"\", \n          \"description\": \"\" \n          }\n          }\n          }\", \"type\": \"dataframe\"}

# Multi-seed robustness summary

metrics = [\"accuracy\", \"sensitivity\", \"specificity\", \"precision\",
\"auc\"]

comparison = pd.DataFrame(
    {
        \"ResNet50 Mean\": resnet_multiseed_df[metrics].mean(),
        \"ResNet50 Std\": resnet_multiseed_df[metrics].std(),
        \"ResNet50 + CLAHE Mean\":
resnet_clahe_multiseed_df[metrics].mean(),
        \"ResNet50 + CLAHE Std\":
resnet_clahe_multiseed_df[metrics].std(),
    }
)

display(comparison.round(4))

resnet_auc = resnet_multiseed_df[\"auc\"].mean()
clahe_auc = resnet_clahe_multiseed_df[\"auc\"].mean()

print(f\"Mean AUC difference (CLAHE - plain): \" f\"{clahe_auc -
resnet_auc:+.4f}\")

print(
    \"\\nThree-seed results are reported as a robustness check, \"
    \"not as a formal statistical significance test.\"
)

{\"summary\": \"{\\n  \\\"name\\\": \"\\\")\\n  \\\"rows\\\": 5,\\n  \\\"fields\\\": [\\n
{\\n
    \\\"column\\\": \\\"ResNet50 Mean\\\",\\n    \\\"properties\\\": {\\n
    \\\"dtype\\\": \\\"number\\\",\\n    \\\"std\\\": 0.14183232353733757,\\n
    \\\"min\\\": 0.4753,\\n    \\\"max\\\": 0.8161,\\n
    \\\"num_unique_values\\\": 5,\\n    \\\"samples\\\": [\\n    0.8161,\\n
    0.7197,\\n    0.4753\\n    ],\\n    \\\"semantic_type\\\":
    \\\"\\\",\\n    \\\"description\\\": \\\"\\\"\\n    }\\n    },\\n    {\\n
    \\\"column\\\": \\\"ResNet50 Std\\\",\\n    \\\"properties\\\": {\\n
    \\\"dtype\\\": \\\"number\\\",\\n    \\\"std\\\": 0.04342044449334898,\\n
    \\\"min\\\": 0.0112,\\n    \\\"max\\\": 0.1098,\\n
    \\\"num_unique_values\\\": 5,\\n    \\\"samples\\\": [\\n    0.0919,\\n
    0.0112,\\n    0.1098\\n    ],\\n    \\\"semantic_type\\\":
    \\\"\\\",\\n    \\\"description\\\": \\\"\\\"\\n    }\\n    },\\n    {\\n
    \\\"column\\\": \\\"ResNet50 + CLAHE Mean\\\",\\n    \\\"properties\\\": {\\n
    \\\"dtype\\\": \\\"number\\\",\\n    \\\"std\\\": 0.12420339367344194,\\n
    \\\"min\\\": 0.5139,\\n    \\\"max\\\": 0.7954,\\n

```



```

        resnet_clahe_results,
    ]
)

display(principal_results.round(4))

{"summary":{"\n  \"name\": \"display(principal_results\", \"rows\": 6, \"fields\": [\n    {\n      \"column\": \"Model\", \"properties\": {\n        \"dtype\": \"string\", \"num_unique_values\": 6, \"samples\": [\n          \"Baseline CNN (128px)\", \"Regularised Scaled CNN (128px)\", \"ResNet50 + CLAHE\", \n        ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"Threshold\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.09342720517422463, \"min\": 0.3172, \"max\": 0.5567, \"num_unique_values\": 6, \"samples\": [\n            0.5308, 0.5567, 0.3907 \n          ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"Accuracy\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.03373808925630893, \"min\": 0.6627, \"max\": 0.754, \"num_unique_values\": 6, \"samples\": [\n            0.6667, 0.6944, 0.7143 \n          ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"Sensitivity\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.10585139425943649, \"min\": 0.6341, \"max\": 0.9187, \"num_unique_values\": 6, \"samples\": [\n            0.6341, 0.8618, 0.9024 \n          ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"Specificity\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.10602389196151339, \"min\": 0.5039, \"max\": 0.7442, \"num_unique_values\": 4, \"samples\": [\n            0.5349, 0.7442, 0.6977 \n          ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"Precision\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.04373663986483947, \"min\": 0.6145, \"max\": 0.7402, \"num_unique_values\": 6, \"samples\": [\n            0.6667, 0.6386, 0.6491 \n          ], \"semantic_type\": \"\", \"description\": \"\" \n      }, {\n        \"column\": \"AUC\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 0.052472265690235526, \"min\": 0.6856, \"max\": 0.8254, \"num_unique_values\": 6, \"samples\": [\n            0.7353, 0.7574, 0.6938, \n          ], \"semantic_type\":

```

```
\",\",\\n      \\\"description\\\": \\\"\\\"\\n      }\\n      }\\n    ]\\n}\\\", \"type\": \"dataframe\"}
```

Section 27.2: Final Selected Model - Held-Out Test Evaluation

Following validation-based model selection, **ResNet50 + CLAHE** is evaluated on the held-out test set using the frozen validation-derived threshold of **0.3907**.

This represents the formal held-out performance of the selected final model.

```
# Final model: held-out test evaluation

final_model = tf.keras.models.load_model(
    f'{MODELS_DIR}/resnet50_clahe_best.keras', compile=False
)

# Freeze threshold selected from VALIDATION
final_threshold = resnet_clahe_threshold

final_test_results = evaluate_model(
    final_model,
    resnet_clahe_test_ds,
    name="ResNet50 + CLAHE",
    threshold=final_threshold,
    split_name="Held-Out Test",
)

final_test_table = pd.DataFrame(
    [
        {
            "Model": final_test_results["model"],
            "Threshold": final_test_results["threshold"],
            "Accuracy": final_test_results["accuracy"],
            "Sensitivity": final_test_results["sensitivity"],
            "Specificity": final_test_results["specificity"],
            "Precision": final_test_results["precision"],
            "AUC": final_test_results["auc"],
        }
    ]
).round(4)

display(final_test_table)
```

```
=====
ResNet50 + CLAHE – Held-Out Test Results (threshold=0.391)
=====
```

	precision	recall	f1-score	support
Benign	0.84	0.50	0.63	216

Malignant	0.54	0.86	0.66	145
accuracy			0.65	361
macro avg	0.69	0.68	0.65	361
weighted avg	0.72	0.65	0.64	361

Accuracy : 0.6482
Sensitivity : 0.8621 (missed 20 of 145 malignant cases)
Specificity : 0.5046
Precision : 0.5388
AUC : 0.7457

```
{"summary":{"\n  \"name\": \"final_test_table\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"ResNet50 + CLAHE\",\n          \"\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"Threshold\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.3907,\n            \"max\": 0.3907,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.3907\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          \"column\": \"Accuracy\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.6482,\n            \"max\": 0.6482,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.6482\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          \"column\": \"Sensitivity\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.8621,\n            \"max\": 0.8621,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.8621\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          \"column\": \"Specificity\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.5046,\n            \"max\": 0.5046,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.5046\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          \"column\": \"Precision\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.5388,\n            \"max\": 0.5388,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.5388\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          \"column\": \"AUC\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": null,\n            \"min\": 0.7457,\n            \"max\": 0.7457,\n            \"num_unique_values\": 1,\n            \"samples\": [\n              0.7457\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          }\n        ],\n        \"type\": \"dataframe\", \"variable_name\": \"final_test_table\"}
```

```

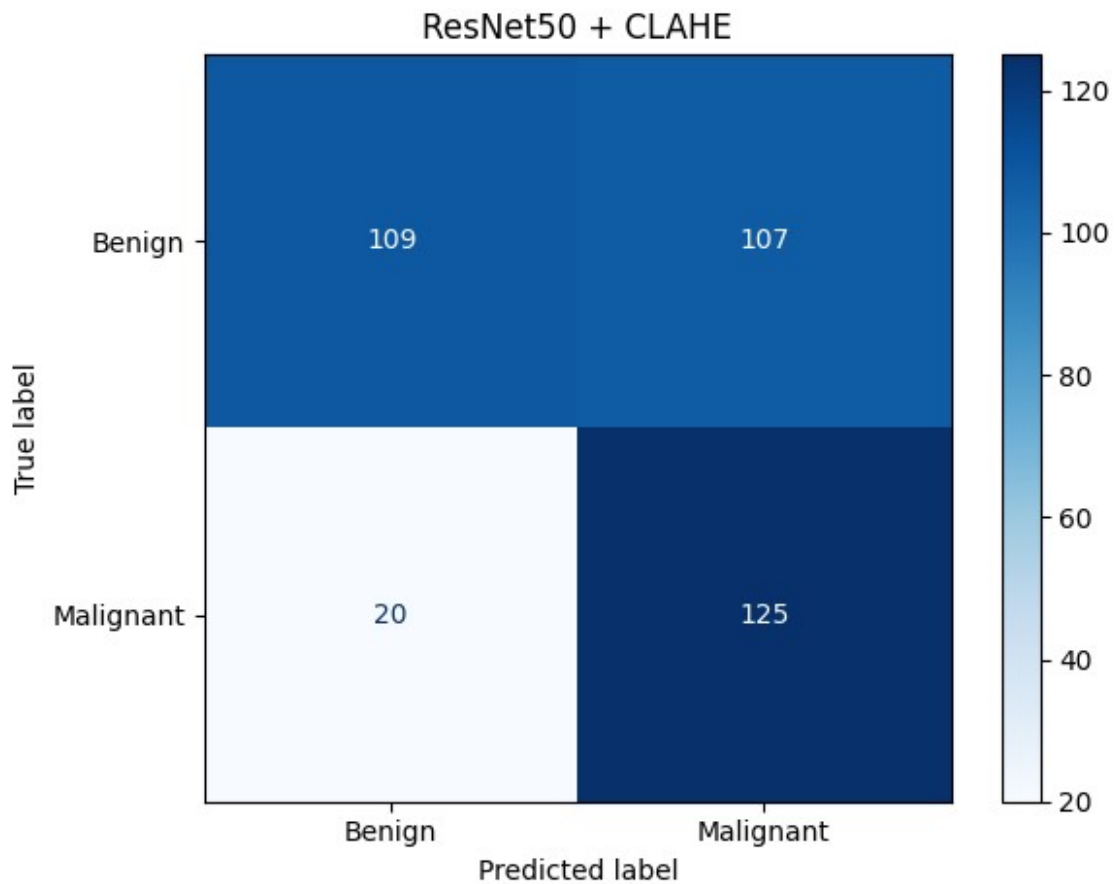
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

# Final held-out test confusion matrix
# [[TN, FP],
#  [FN, TP]]
cm = np.array([[109, 107], [20, 125]])

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=["Benign", "Malignant"]
)

disp.plot(cmap="Blues", values_format="d")
plt.title("ResNet50 + CLAHE")
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.tight_layout()
plt.show()

```



Section 27.3: Validation ROC Curves and Confusion Matrices

ROC curves and confusion matrices summarise the validation behaviour of the six principal models. These visualisations present the validation evidence used during model comparison and selection.

```
# ROC curves for all principal models

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc

# Load models
model_files = {
    "Baseline CNN (128px)": "baseline_cnn_best.keras",
    "Regularised Scaled CNN (128px)":
"regularised_scaled_cnn_128_best.keras",
    "Regularised Scaled CNN (224px)":
"regularised_scaled_cnn_224_best.keras",
    "VGG16": "vgg16_best.keras",
    "ResNet50": "resnet50_best.keras",
    "ResNet50 + CLAHE": "resnet50_clahe_best.keras",
}

models = {
    name: tf.keras.models.load_model(f"{MODELS_DIR}/{file}",
compile=False)
    for name, file in model_files.items()
}

# Transfer-learning validation datasets
if "vgg_val_ds" not in globals():
    vgg_val_ds = prepare_vgg_dataset(val_ds_224)

if "resnet_val_ds" not in globals():
    resnet_val_ds = prepare_resnet_dataset(val_ds_224)

if "resnet_clahe_val_ds" not in globals():
    resnet_clahe_val_ds = prepare_resnet_dataset(val_ds_224_clahe)

datasets = {
    "Baseline CNN (128px)": val_ds_128,
    "Regularised Scaled CNN (128px)": val_ds_128,
    "Regularised Scaled CNN (224px)": val_ds_224,
    "VGG16": vgg_val_ds,
    "ResNet50": resnet_val_ds,
    "ResNet50 + CLAHE": resnet_clahe_val_ds,
}
```

```

# ROC curves
plt.figure(figsize=(8, 6))

for name, model in models.items():
    ds = datasets[name]

    y_prob = model.predict(ds, verbose=0).ravel()
    y_true = np.concatenate([y.numpy().ravel() for _, y in
ds]).astype(int)

    fpr, tpr, _ = roc_curve(y_true, y_prob)
    score = auc(fpr, tpr)

    plt.plot(fpr, tpr, linewidth=2, label=f"{name} (AUC =
{score:.3f})")

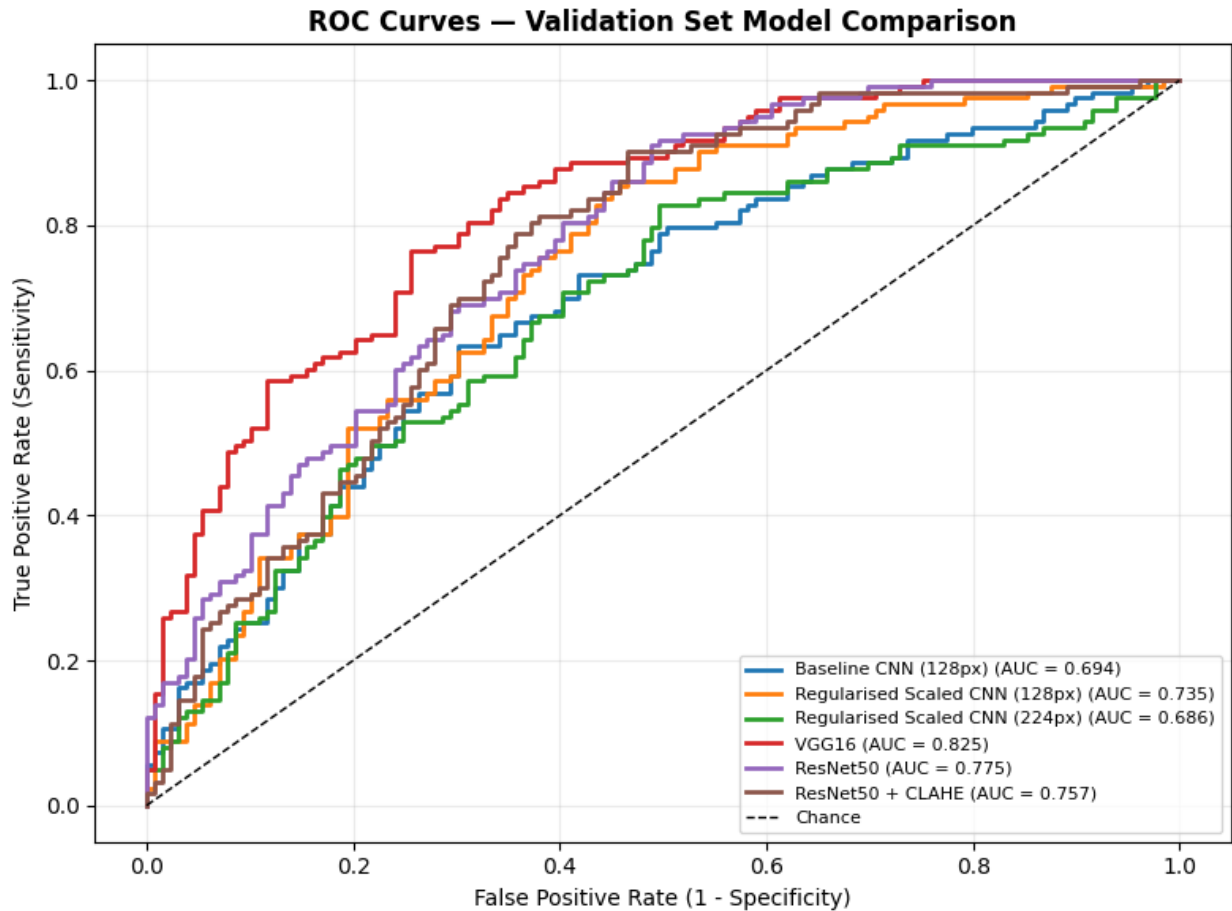
plt.plot([0, 1], [0, 1], "k--", linewidth=1, label="Chance")

plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Sensitivity)")
plt.title("ROC Curves - Validation Set Model Comparison",
fontweight="bold")
plt.legend(loc="lower right", fontsize=8)
plt.grid(alpha=0.25)
plt.tight_layout()

plt.savefig(f"{BASE}/roc_all_models_validation.png", dpi=300,
bbox_inches="tight")
plt.show()

All current corrected principal models loaded.

```



```
# Confusion matrices for principal models (validation set)
```

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
```

```
# Current validation results
```

```
current_results = [
    baseline_principal_results,
    reg128_results,
    reg224_results,
    vgg_results,
    resnet_results,
    resnet_clahe_results,
]
```

```
# Create figure
```

```
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
```

```
# Plot each confusion matrix
for ax, result in zip(axes.flat, current_results):
    disp = ConfusionMatrixDisplay(
        confusion_matrix=result["cm"], display_labels=["Benign",
        "Malignant"]
    )

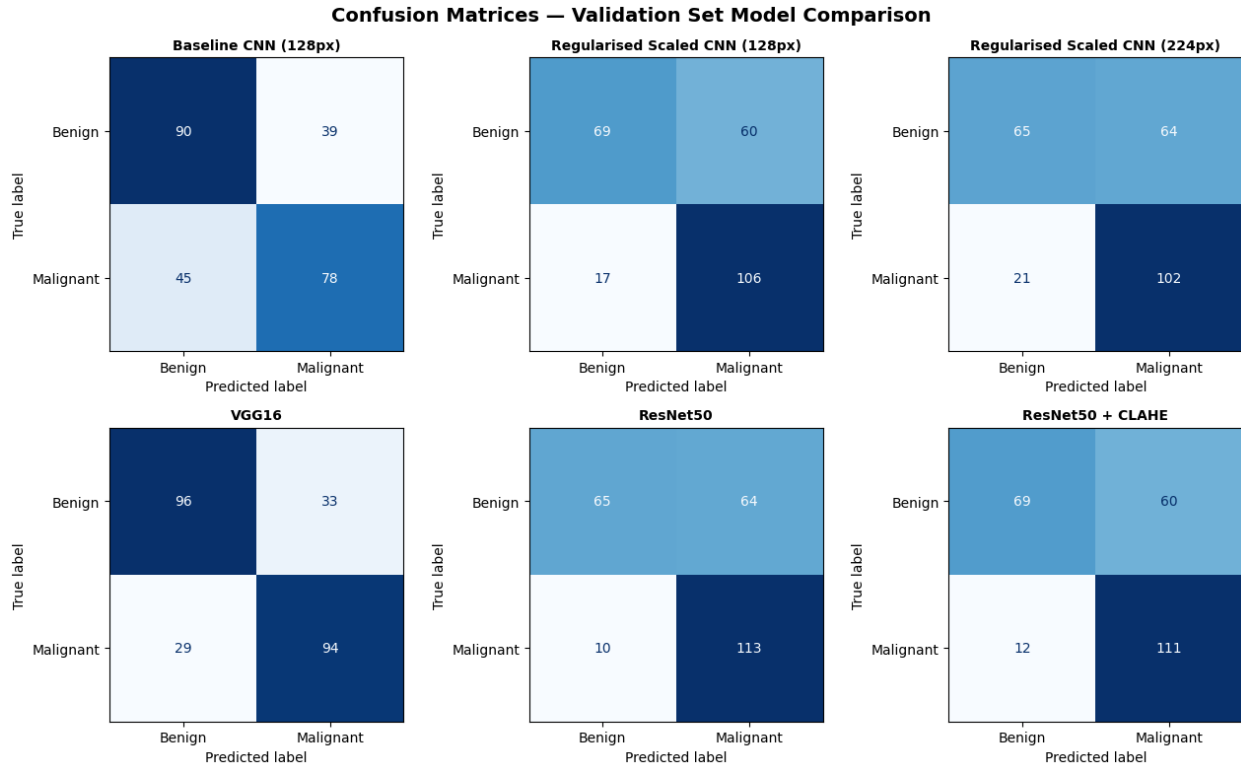
    disp.plot(ax=ax, cmap="Blues", colorbar=False, values_format="d")
    ax.set_title(result["model"], fontweight="bold", fontsize=10)

# Format
plt.suptitle(
    "Confusion Matrices - Validation Set Model Comparison",
    fontsize=14,
    fontweight="bold",
)

plt.tight_layout()

# Save
plt.savefig(
    f"{BASE}/confusion_matrices_all_models_validation.png", dpi=300,
    bbox_inches="tight"
)

plt.show()
```



Section 27.4: Multi-Seed Robustness Summary

The three-seed results for ResNet50 and ResNet50 + CLAHE are summarised as mean $\pm$ standard deviation across the main evaluation metrics. This provides a compact view of performance stability across repeated training runs.

Figure 5.3: multi-seed mean +/- SD comparison

```
import numpy as np
import matplotlib.pyplot as plt

metrics = ["Accuracy", "Sensitivity", "Specificity", "Precision",
"AUC"]

metric_cols = ["accuracy", "sensitivity", "specificity", "precision",
"auc"]

# Calculate directly from regenerated results
resnet_mean = resnet_multiseed_df[metric_cols].mean().values
resnet_std = resnet_multiseed_df[metric_cols].std().values

clahe_mean = resnet_clahe_multiseed_df[metric_cols].mean().values
clahe_std = resnet_clahe_multiseed_df[metric_cols].std().values

x = np.arange(len(metrics))
width = 0.35
```

```

plt.figure(figsize=(9, 6))

plt.bar(x - width / 2, resnet_mean, width, yerr=resnet_std, capsize=4,
label="ResNet50")

plt.bar(
    x + width / 2,
    clahe_mean,
    width,
    yerr=clahe_std,
    capsize=4,
    label="ResNet50 + CLAHE",
)

plt.xticks(x, metrics)
plt.ylabel("Mean Held-Out Test Score")
plt.ylim(0, 1)

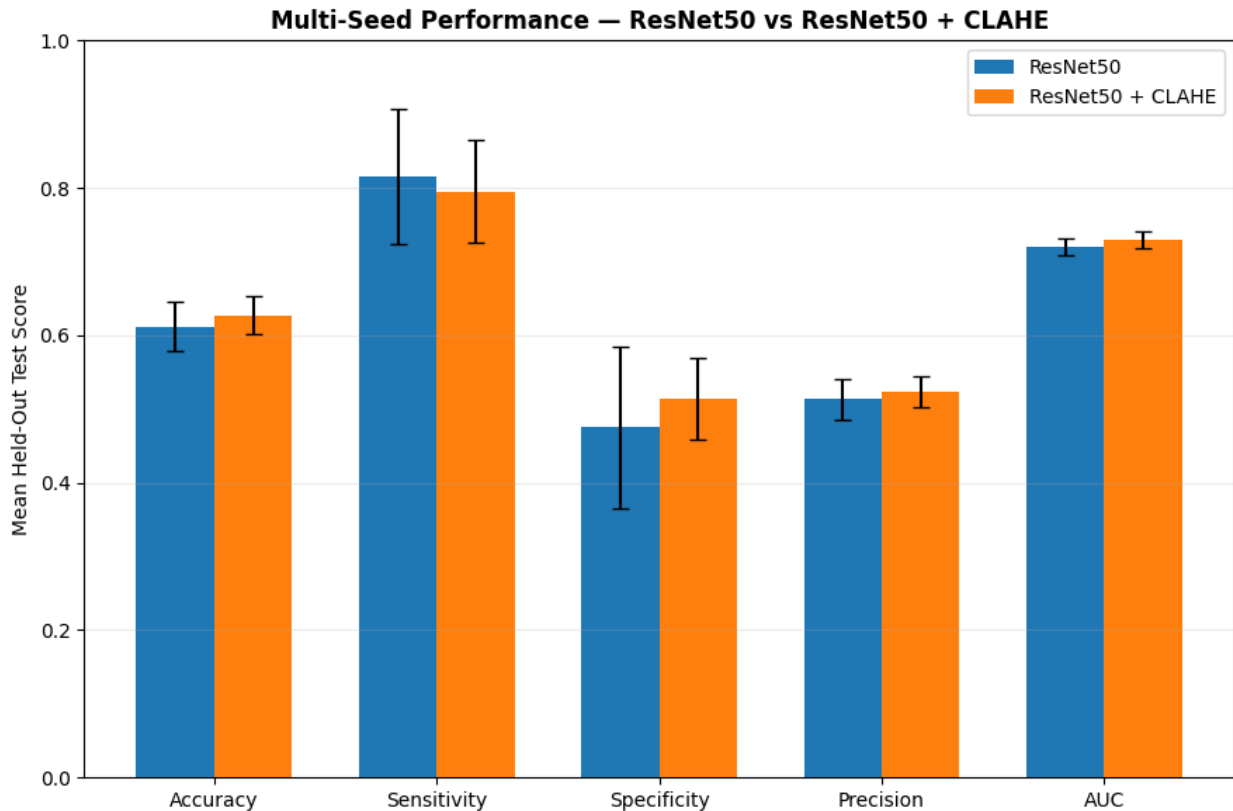
plt.title("Multi-Seed Performance - ResNet50 vs ResNet50 + CLAHE",
fontweight="bold")

plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()

plt.savefig(
    f"{BASE}/multiseed_resnet_vs_clahe_corrected.png", dpi=300,
    bbox_inches="tight"
)

plt.show()

```



Section 27.5: Grad-CAM Case Analysis - Final Selected Model

Grad-CAM is used to qualitatively examine the selected **ResNet50 + CLAHE** model using one true positive, true negative, false negative and false positive case from the held-out test set.

The same preprocessing and Grad-CAM configuration used for the final model are reproduced. These examples illustrate model attention for correct and incorrect predictions and are interpreted as post-hoc explanations rather than lesion segmentation.

```
# Figure 5.4: Grad-CAM case analysis (TP, TN, FN, FP)

import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications.resnet50 import preprocess_input

# Load model + validation-selected threshold
model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_clahe_best.keras", compile=False
)

threshold = find_best_threshold(model, resnet_clahe_val_ds,
                               verbose=False)
```

```

# Grad-CAM model
layer = model.get_layer("conv4_block6_out")
grad_model = tf.keras.Model(model.input, [layer.output, model.output])

def gradcam(path):
    # Same CLAHE preprocessing as model
    img = tf.image.decode_jpeg(tf.io.read_file(path), channels=3)
    img = crop_to_content(img).numpy().astype(np.uint8)

    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    gray = cv2.createCLAHE(2.0, (8, 8)).apply(gray)
    rgb = cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)

    resized = tf.image.resize_with_pad(
        tf.cast(rgb, tf.float32), IMG_SIZE_224, IMG_SIZE_224
    )

    display = resized.numpy() / 255.0
    x = preprocess_input(resized)[None, ...]

    # Explain predicted class
    with tf.GradientTape() as tape:
        conv, pred = grad_model(x, training=False)
        p = pred[:, 0]
        loss = tf.where(p >= threshold, p, 1.0 - p)

    grads = tape.gradient(loss, conv)
    weights = tf.reduce_mean(grads, axis=(1, 2))
    heat = tf.reduce_sum(conv[0] * weights[0], axis=-1)
    heat = tf.maximum(heat, 0).numpy()

    heat = cv2.resize(heat, (IMG_SIZE_224, IMG_SIZE_224),
        interpolation=cv2.INTER_CUBIC)
    heat = np.clip(heat, 0, None)

    # Remove attribution outside breast
    heat *= display.mean(axis=-1) > 0.03

    if heat.max() > 0:
        heat /= heat.max()

    prob = float(pred.numpy()[0, 0])
    pred_label = "MALIGNANT" if prob >= threshold else "BENIGN"

    return display, heat, pred_label, prob

# Find first TP, TN, FN and FP deterministically
cases = {
    "True Positive": None,

```

```

    "True Negative": None,
    "False Negative": None,
    "False Positive": None,
}

for path, true in zip(X_test, y_test):
    img, heat, pred, prob = gradcam(path)
    true = "MALIGNANT" if int(true) == 1 else "BENIGN"

    if true == "MALIGNANT" and pred == "MALIGNANT":
        key = "True Positive"
    elif true == "BENIGN" and pred == "BENIGN":
        key = "True Negative"
    elif true == "MALIGNANT":
        key = "False Negative"
    else:
        key = "False Positive"

    if cases[key] is None:
        cases[key] = (img, heat, true, pred, prob)

    if all(v is not None for v in cases.values()):
        break

# Plot
fig, axes = plt.subplots(4, 3, figsize=(12, 16))

for i, (case_type, case) in enumerate(cases.items()):
    img, heat, true, pred, prob = case

    colour = plt.get_cmap("jet")(heat)[..., :3]
    alpha = heat[..., None] * 0.35
    overlay = np.clip(img * (1 - alpha) + colour * alpha, 0, 1)

    axes[i, 0].imshow(img)
    axes[i, 0].set_title(f"{case_type}\nTrue: {true}")

    axes[i, 1].imshow(img)
    axes[i, 1].imshow(heat, cmap="jet", alpha=0.55, vmin=0, vmax=1)
    axes[i, 1].set_title("Grad-CAM")

    axes[i, 2].imshow(overlay)
    axes[i, 2].set_title(f"Pred: {pred} ({prob:.2f})")

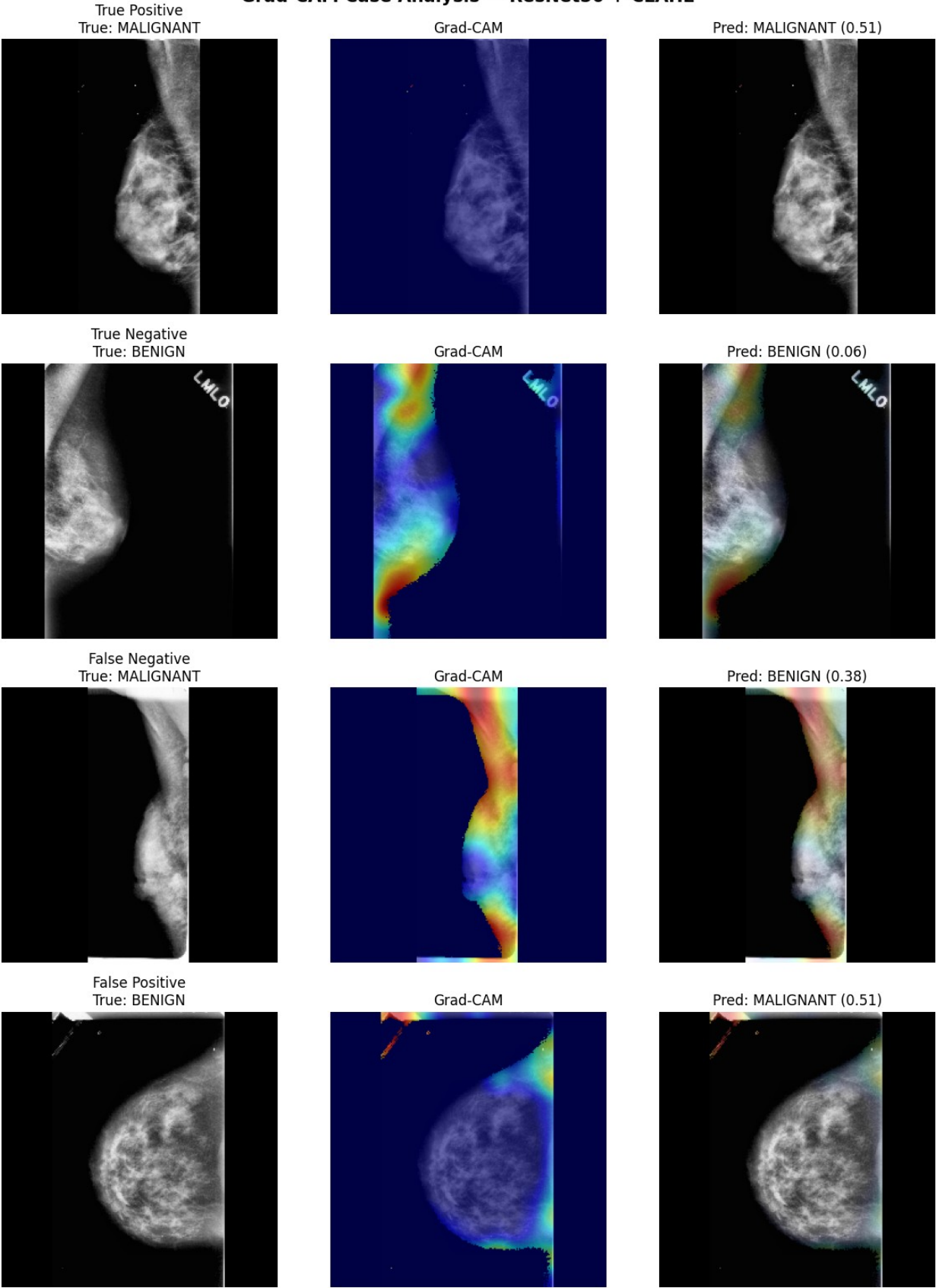
    for ax in axes[i]:
        ax.axis("off")

plt.suptitle(
    "Grad-CAM Case Analysis - ResNet50 + CLAHE", fontsize=15,

```

```
fontweight="bold"  
)  
plt.tight_layout()  
  
plt.savefig(f"{BASE}/gradcam_case_analysis_corrected.png", dpi=300,  
bbox_inches="tight")  
  
plt.show()  
  
print(f"Threshold: {threshold:.4f}")  
print(f"Saved: {BASE}/gradcam_case_analysis_corrected.png")
```

Grad-CAM Case Analysis — ResNet50 + CLAHE



Threshold: 0.3907

Saved:

/content/drive/MyDrive/cbis_ddsm/gradcam_case_analysis_corrected.png

Section 28: McNemar Significance Test

An exact McNemar test provides a paired comparison between ResNet50 and ResNet50 + CLAHE on the same held-out test cases. Each model uses its independently derived validation threshold.

The resulting $p = 0.1112$ indicates no statistically significant evidence of a difference in paired classification correctness at $\alpha = 0.05$.

```
# McNemar test: ResNet50 vs ResNet50 + CLAHE

import numpy as np
import tensorflow as tf
from scipy.stats import binomtest

# Rebuild datasets if needed

if "resnet_val_ds" not in globals():
    resnet_val_ds = prepare_resnet_dataset(val_ds_224)

if "resnet_test_ds" not in globals():
    resnet_test_ds = prepare_resnet_dataset(test_ds_224)

if "resnet_clahe_val_ds" not in globals():
    resnet_clahe_val_ds = prepare_resnet_dataset(val_ds_224_clahe)

if "resnet_clahe_test_ds" not in globals():
    resnet_clahe_test_ds = prepare_resnet_dataset(test_ds_224_clahe)

# Load principal models

resnet_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_best.keras", compile=False
)

resnet_clahe_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/resnet50_clahe_best.keras", compile=False
)

# Select thresholds using validation set only

RESNET_THRESHOLD = find_best_threshold(resnet_model, resnet_val_ds,
    verbose=False)

CLAHE_THRESHOLD = find_best_threshold(
    resnet_clahe_model, resnet_clahe_val_ds, verbose=False)
```

```

)

print(f"ResNet50 threshold: {RESNET_THRESHOLD:.4f}")
print(f"CLAHE threshold: {CLAHE_THRESHOLD:.4f}")

# Test labels and predictions

def get_labels(ds):
    return np.concatenate([y.numpy().ravel() for _, y in
ds]).astype(int)

y_plain = get_labels(resnet_test_ds)
y_clahe = get_labels(resnet_clahe_test_ds)

assert len(y_plain) == 361
assert len(y_clahe) == 361
assert np.array_equal(y_plain, y_clahe)

y_true = y_plain

p_resnet = resnet_model.predict(resnet_test_ds, verbose=0).ravel()
p_clahe = resnet_clahe_model.predict(resnet_clahe_test_ds,
verbose=0).ravel()

pred_resnet = (p_resnet >= RESNET_THRESHOLD).astype(int)
pred_clahe = (p_clahe >= CLAHE_THRESHOLD).astype(int)

# McNemar paired counts

r_correct = pred_resnet == y_true
c_correct = pred_clahe == y_true

both_correct = int(np.sum(r_correct & c_correct))
resnet_only = int(np.sum(r_correct & ~c_correct))
clahe_only = int(np.sum(~r_correct & c_correct))
both_wrong = int(np.sum(~r_correct & ~c_correct))

discordant = resnet_only + clahe_only

p_value = (
    binomtest(
        min(resnet_only, clahe_only), discordant, p=0.5,
alternative="two-sided"
    ).pvalue
    if discordant > 0
    else 1.0
)

```

Results

```
print("\nMcNemar Test - Held-Out Test Set")
print("=" * 45)
print(f"Both correct           : {both_correct}")
print(f"ResNet50 only correct  : {resnet_only}")
print(f"CLAHE only correct      : {clahe_only}")
print(f"Both wrong              : {both_wrong}")
print(f"Discordant cases        : {discordant}")
print(f"Exact p-value           : {p_value:.6f}")

if p_value < 0.05:
    print("Result: statistically significant difference (p < 0.05).")
else:
    print("Result: no statistically significant difference (p >= 0.05).")
```

Loading corrected principal models...

WARNING:tensorflow:5 out of the last 21 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7cddc53eec00> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

ResNet50 validation-selected threshold: 0.3814

ResNet50 + CLAHE validation-selected threshold: 0.3907

Plain test cases: 361

CLAHE test cases: 361

ResNet50 test accuracy from predictions: 0.6122

ResNet50 + CLAHE test accuracy from predictions: 0.6482

McNemar Test: ResNet50 vs ResNet50 + CLAHE

```
=====
Both correct           : 199
ResNet50 only correct  : 22
CLAHE only correct     : 35
Both wrong            : 105
Discordant cases       : 57

Exact McNemar p-value   : 0.111161
```

Result: no statistically significant difference between the paired model predictions ($p \geq 0.05$).

Section 29: Quantitative Grad-CAM Lesion-Mask Evaluation

Grad-CAM activation from the final ResNet50 + CLAHE model is compared with the malignant lesion masks provided by CBIS-DDSM. ROI geometry and mask alignment are first verified before localisation is evaluated across the malignant held-out cases.

This analysis examines whether model attention spatially corresponds with annotated abnormalities; Grad-CAM is not treated as a segmentation method.

```
!pip install pydicom -q

2.4/2.4 MB 15.1 MB/s eta
0:00:00

# Lesion mask sanity check

import os, cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_BASE = "/content/drive/MyDrive/cbis_ddsm"
MASK_COL = "ROI mask file path"

# DICOM metadata + real extracted paths
dicom_info = pd.read_csv(f"{DATA_BASE}/csv/dicom_info.csv")
dicom_info["real_path"] = (
    DATA_BASE
    + "/"
    + dicom_info["image_path"].astype(str).str.replace("CBIS-DDSM/",
    "", regex=False)
)

roi_info = dicom_info[dicom_info["SeriesDescription"] == "ROI mask
images"]

roi_lookup = {}
for _, r in roi_info.iterrows():
    path = r["real_path"]
    if os.path.exists(path):
        roi_lookup.setdefault(str(r["PatientID"]).strip(),
        []).append(path)

def resolve_roi_image(csv_path):
```

```

case_id = str(csv_path).replace("\\", "/").split("/")[0]
for path in roi_lookup.get(case_id, []):
    if cv2.imread(path, cv2.IMREAD_GRAYSCALE) is not None:
        return path
raise FileNotFoundError(f"No readable ROI for {case_id}")

sample = (
    test_lesion_csv[(test_lesion_csv["label"] == 1) &
test_lesion_csv[MASK_COL].notna()]
    .drop_duplicates(["full_path", MASK_COL])
    .head(4)
)

fig, axes = plt.subplots(len(sample), 4, figsize=(12, 3 *
len(sample)))
if len(sample) == 1:
    axes = axes[None, :]

for i, (_, row) in enumerate(sample.iterrows()):
    mamm = cv2.imread(row["full_path"], cv2.IMREAD_GRAYSCALE)
    roi = cv2.imread(resolve_roi_image(row[MASK_COL]),
cv2.IMREAD_GRAYSCALE)

    _, mask = cv2.threshold(roi, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
    if (mask > 0).mean() > 0.5:
        mask = 255 - mask

    overlay_mask = cv2.resize(
        mask, (mamm.shape[1], mamm.shape[0]),
interpolation=cv2.INTER_NEAREST
    )

    axes[i, 0].imshow(mamm, cmap="gray")
    axes[i, 0].set_title("Mammogram")

    axes[i, 1].imshow(roi, cmap="gray")
    axes[i, 1].set_title("ROI Mask Image")

    axes[i, 2].imshow(mask, cmap="gray")
    axes[i, 2].set_title("Thresholded Mask")

    axes[i, 3].imshow(mamm, cmap="gray")
    axes[i, 3].imshow(
        np.ma.masked_where(overlay_mask == 0, overlay_mask),
cmap="Reds", alpha=0.5
    )
    axes[i, 3].set_title("Raw Overlay")

```

```
    for ax in axes[i]:
        ax.axis("off")

plt.tight_layout()
plt.savefig(
    f"{BASE}/lesion_mask_sanity_check_corrected.png", dpi=200,
    bbox_inches="tight"
)
plt.show()

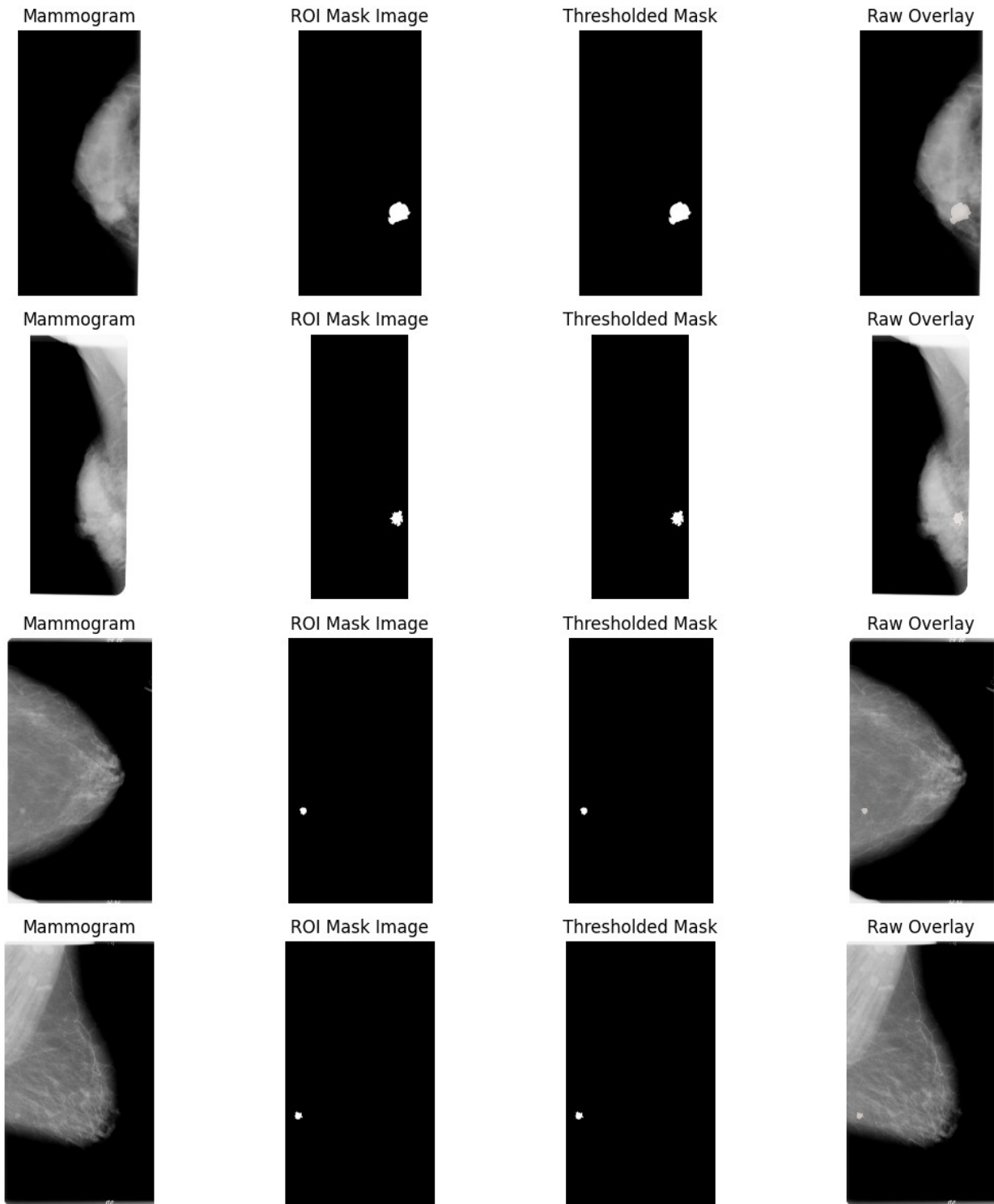
dicom_info rows: 10237
ROI mask metadata rows: 3247
ROI cases with real extracted files: 3247
Selected lesion rows: 4

Resolved ROI 1:
/content/drive/MyDrive/cbis_ddsm/jpeg/1.3.6.1.4.1.9590.100.1.2.3082058
6311062570442302321942433426184/1-083.jpg

Resolved ROI 2:
/content/drive/MyDrive/cbis_ddsm/jpeg/1.3.6.1.4.1.9590.100.1.2.3814401
41511137044327302306604206077287/1-084.jpg

Resolved ROI 3:
/content/drive/MyDrive/cbis_ddsm/jpeg/1.3.6.1.4.1.9590.100.1.2.2121430
28513012144941507232513982203672/2-085.jpg

Resolved ROI 4:
/content/drive/MyDrive/cbis_ddsm/jpeg/1.3.6.1.4.1.9590.100.1.2.1540304
3813402510742192372832381918984/2-086.jpg
```



Section 29.1: Quantitative Grad-CAM Setup

The final ResNet50 + CLAHE model and Grad-CAM utilities are prepared for lesion-level analysis. These functions reproduce the final model preprocessing and map Grad-CAM activation and lesion annotations into the same 224×224 coordinate system.

```

# ROI/crop geometry diagnostic

import cv2
import matplotlib.pyplot as plt

row = test_lesion_csv[test_lesion_csv["label"] == 1].iloc[0]

case_id = str(row["ROI mask file path"]).replace("\\", "/").split("/")
[0]

case_meta = dicom_info[dicom_info["PatientID"].astype(str).str.strip()
== case_id]

def get_case_image(series):
    rows = case_meta[case_meta["SeriesDescription"] == series]
    if rows.empty:
        return None, None
    path = rows.iloc[0]["real_path"]
    return cv2.imread(path, cv2.IMREAD_GRAYSCALE), path

full_img = cv2.imread(row["full_path"], cv2.IMREAD_GRAYSCALE)
crop_img, crop_path = get_case_image("cropped images")
mask_img, mask_path = get_case_image("ROI mask images")

print("CASE:", case_id)
print("Full:", full_img.shape)
print("Crop:", None if crop_img is None else crop_img.shape)
print("Mask:", None if mask_img is None else mask_img.shape)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

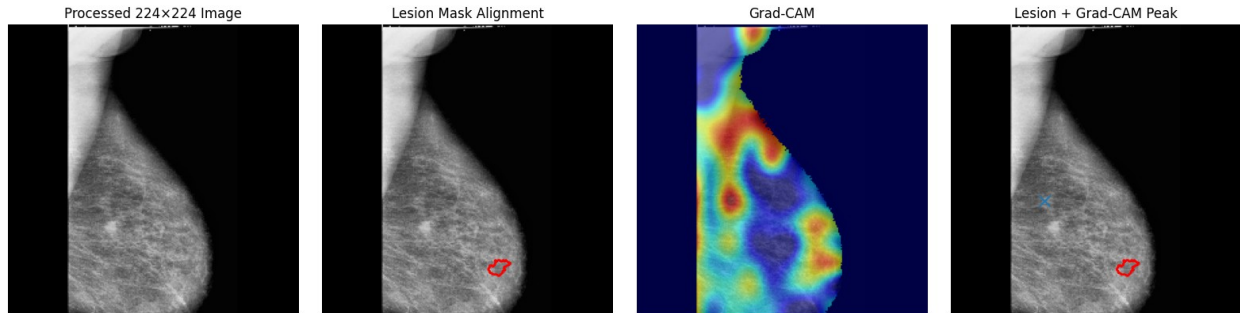
for ax, img, title in zip(
    axes,
    [full_img, crop_img, mask_img],
    ["Full Mammogram", "Cropped Lesion", "ROI Mask"],
):
    if img is not None:
        ax.imshow(img, cmap="gray")
        ax.set_title(title)
        ax.axis("off")

plt.tight_layout()
plt.show()

Case index: 0
Full path:
/content/drive/MyDrive/cbis_ddsm/jpeg/1.3.6.1.4.1.9590.100.1.2.3347043
28611391643805508991083300190349/1-108.jpg
Mask pixels: 114

```

```
Mask area fraction: 0.0022720026317983866
Malignancy score: 0.9514608383178711
Grad-CAM peak: (np.int64(135), np.int64(72))
Peak inside mask: False
```



Section 29.2: ROI and Crop Geometry Verification

The full mammogram, cropped lesion image and ROI mask are inspected to verify the relationship between the original CBIS-DDSM annotations and the preprocessing geometry used by the classification pipeline.

```
# Quantitative Grad-CAM lesion localisation

rows = []

malignant_indices = np.where(y == 1)[0]

for n, i in enumerate(malignant_indices, 1):
    mask = union_malignant_mask_224(str(X_test[i]))

    heat, prob = cam_exact(X[i])

    # Location of maximum Grad-CAM activation
    py, px = np.unravel_index(np.argmax(heat), heat.shape)

    peak_inside = bool(mask[py, px])

    # Fraction of total Grad-CAM activation inside lesion
    total_heat = heat.sum()
    lesion_heat = heat[mask].sum()

    heat_fraction = lesion_heat / total_heat if total_heat > 0 else
0.0

    # Fraction of image occupied by lesion
    area_fraction = mask.mean()

    # Attention enrichment relative to lesion size
    enrichment = heat_fraction / area_fraction if area_fraction > 0
```

```

else np.nan

    rows.append(
        {
            "test_index": int(i),
            "malignancy_probability": prob,
            "peak_inside_lesion": peak_inside,
            "lesion_heat_fraction": heat_fraction,
            "lesion_area_fraction": area_fraction,
            "attention_enrichment": enrichment,
        }
    )

    if n % 25 == 0 or n == len(malignant_indices):
        print(f"Processed {n}/{len(malignant_indices)} " "malignant cases")

# Results

localisation_df = pd.DataFrame(rows)
n = len(localisation_df)
peak_n = int(localisation_df["peak_inside_lesion"].sum())
valid_enrichment = localisation_df["attention_enrichment"].dropna()
enriched_n = int((valid_enrichment > 1).sum())

print("\n===== GRAD-CAM LESION LOCALISATION =====")
print(f"Malignant cases analysed: {n}")

print(f"Peak inside lesion: " f"{peak_n}/{n} " f"({100 * peak_n / n:.1f}%)")

print(f"Peak outside lesion: " f"{n - peak_n}/{n}")

print(
    "Mean lesion heat fraction: "
    f"{localisation_df['lesion_heat_fraction'].mean():.4f}"
)

print(
    "Median lesion heat fraction: "
    f"{localisation_df['lesion_heat_fraction'].median():.4f}"
)

print(
    "Mean lesion area fraction: "

```

```

        f"{localisation_df['lesion_area_fraction'].mean():.4f}"
    )
    print(
        "Median lesion area fraction: "
        f"{localisation_df['lesion_area_fraction'].median():.4f}"
    )

    print("Mean attention enrichment: " f"{valid_enrichment.mean():.3f}x")

    print("Median attention enrichment: "
        f"{valid_enrichment.median():.3f}x")

    print(
        f"Enrichment > 1: "
        f"{enriched_n}/{len(valid_enrichment)} "
        f"({100 * enriched_n / len(valid_enrichment):.1f}%) "
    )

# Save results

localisation_df.to_csv(f"{BASE}/gradcam_lesion_localisation_corrected.
csv", index=False)

display(localisation_df.head())

print("\nSaved:", f"{BASE}/gradcam_lesion_localisation_corrected.csv")

Processed 25/145 malignant cases
Processed 50/145 malignant cases
Processed 75/145 malignant cases
Processed 100/145 malignant cases
Processed 125/145 malignant cases
Processed 145/145 malignant cases

===== GRAD-CAM LESION LOCALISATION =====
Malignant cases analysed: 145
Peak inside lesion: 1/145 (0.7%)
Peak outside lesion: 144/145
Mean lesion heat fraction: 0.0059
Median lesion heat fraction: 0.0002
Mean lesion area fraction: 0.0034
Median lesion area fraction: 0.0026
Mean attention enrichment: 1.526x
Median attention enrichment: 0.038x
Enrichment > 1: 44/145 (30.3%)

{"summary": "{\n  \"name\": \"\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"test_index\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 0,\n
```

```

{"max": 4, "num_unique_values": 5, "samples": [1, 4, 2], "semantic_type": "", "description": "", "column": "malignancy_probability", "properties": {"dtype": "number", "std": 0.273128337888232, "min": 0.37956997752189636, "max": 0.9514603018760681, "num_unique_values": 5, "samples": [0.5279043316841125, 0.9303671717643738, 0.44774746894836426]}, "semantic_type": "", "description": "", "column": "peak_inside_lesion", "properties": {"dtype": "boolean", "num_unique_values": 1, "samples": [false]}, "semantic_type": "", "description": "", "column": "lesion_heat_fraction", "properties": {"dtype": "float32", "num_unique_values": 4, "samples": [0.0]}, "semantic_type": "", "description": "", "column": "lesion_area_fraction", "properties": {"dtype": "number", "std": 0.0016454137636972596, "min": 0.00047831632653061223, "max": 0.004643654336734694, "num_unique_values": 5, "samples": [0.0027901785714285715]}, "semantic_type": "", "description": "", "column": "attention_enrichment", "properties": {"dtype": "number", "std": 1.0178712366357916, "min": 0.0, "max": 1.8612485478321712, "num_unique_values": 4, "samples": [0.0]}, "semantic_type": "", "description": "", "column": ""}, {"type": "dataframe"}

```

Saved:

/content/drive/MyDrive/cbis_ddsm/gradcam_lesion_localisation_corrected.csv

Section 30: Export Example Test Images

This optional utility exports a small set of benign and malignant test mammograms for demonstration purposes. It is not part of model training, model selection or evaluation and can be skipped when reproducing the main experimental pipeline.

```

import os, shutil

sample_dir = '/content/sample_images'
os.makedirs(sample_dir, exist_ok=True)

# Grab 3 malignant (label=1) and 3 benign (label=0) paths from your

```

```

test set
malignant_paths = [p for p, l in zip(X_test, y_test) if l == 1][:3]
benign_paths     = [p for p, l in zip(X_test, y_test) if l == 0][:3]

for i, p in enumerate(malignant_paths, start=1):
    shutil.copy(p, f'{sample_dir}/malignant_{i}.jpg')
for i, p in enumerate(benign_paths, start=1):
    shutil.copy(p, f'{sample_dir}/benign_{i}.jpg')

print("Saved 6 sample images to", sample_dir)

!zip -r /content/sample_images.zip /content/sample_images
print("Now download sample_images.zip from the Colab file browser
(folder icon, left sidebar).")

```

```

Saved 6 sample images to /content/sample_images
  adding: content/sample_images/ (stored 0%)
  adding: content/sample_images/malignant_3.jpg (deflated 5%)
  adding: content/sample_images/benign_1.jpg (deflated 10%)
  adding: content/sample_images/benign_3.jpg (deflated 8%)
  adding: content/sample_images/malignant_1.jpg (deflated 13%)
  adding: content/sample_images/benign_2.jpg (deflated 6%)
  adding: content/sample_images/malignant_2.jpg (deflated 8%)
Now download sample_images.zip from the Colab file browser (folder
icon, left sidebar).

```